

Αριστοτέλειο Πανεπιστήμιο Θεσσαλονίκης
Πολυτεχνική Σχολή
Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τομέας Ηλεκτρονικής και Υπολογιστών

Προηγμένη Επεξεργασία Αρχείων Καταγραφής

Advanced Processing of Log Files

Άρτεμις Βερνάδου

ΑΕΜ: 10401

Επιβλέπων:
Καθ. Αναστάσιος Γούναρης
Τμήμα Πληροφορικής

Περίληψη

Τα αρχεία καταγραφών (log files) αποτελούν πολύτιμη πηγή πληροφοριών για την κατανόηση της λειτουργίας των πληροφοριακών συστημάτων, ωστόσο η μη δομημένη και ετερογενής φύση τους καθιστά την ανάλυσή τους ιδιαίτερα περίπλοκη. Η παρούσα διπλωματική εργασία στοχεύει στη δημιουργία ενός ολοκληρωμένου συστήματος για την ολιστική επεξεργασία και ανάλυση αυτών των αρχείων. Η προτεινόμενη προσέγγιση περιλαμβάνει την αποδόμηση των καταγραφών (log parsing) μέσω του μοντέλου LILAC (Log Parsing using LLMs with Adaptive Parsing Cache) [37], το οποίο αναγνωρίζει τόσο τις μεταβλητές όσο και τα σταθερά πρότυπα (templates) που τις διέπουν. Στη συνέχεια, υλοποιείται μία λογική έτσι ώστε τα δεδομένα που προκύπτουν να μετασχηματιστούν σε κατάλληλη μορφή για την εφαρμογή αλγορίθμων εξόρυξης διαδικασιών (process mining), με τελικό στάδιο την αξιοποίηση του εργαλείου εξόρυξης MINERful [6]. Συνολικά, η διπλωματική επιδιώκει να ενοποιήσει τα επιμέρους στάδια επεξεργασίας σε μια ενιαία ροή, λαμβάνοντας παράλληλα υπόψη την ετερογένεια των σύγχρονων συστημάτων, προσφέροντας έτσι μια συστημική καινοτομία. Με τον τρόπο αυτό, διευκολύνεται η βαθύτερη κατανόηση της συμπεριφοράς των συστημάτων και της εκτέλεσης των διεργασιών, γεφυρώνοντας το χάσμα μεταξύ των ακατέργαστων καταγραφών (raw logs) και της εξόρυξης διαδικασιών.

Abstract

Log files constitute a valuable source of information for understanding the operation of information systems; however, their unstructured and heterogeneous nature makes their analysis particularly complex. This thesis aims to develop an integrated system for the holistic processing and analysis of these files. The proposed approach includes log parsing via the LILAC model (Log Parsing using LLMs with Adaptive Parsing Cache) [37], which identifies both the variables and the static templates that govern them. Subsequently, logic is implemented to transform the resulting data into an appropriate format for the application of process mining algorithms, with the final stage utilizing the MINERful [6] mining tool. Overall, this thesis seeks to unify the individual processing stages into a single workflow, while also taking into account the heterogeneity of modern systems, thus offering a systemic innovation. In this way, it facilitates a deeper understanding of system behavior and process execution, bridging the gap between raw logs and process mining.

Ευχαριστίες

Θα ήθελα να ευχαριστήσω θερμά τον καθηγητή μου κ. Αναστάσιο Γούναρη για την ευκαιρία που μου έδωσε να ασχοληθώ με το συγκεκριμένο θέμα, καθώς και για την καθοδήγηση που μου παρείχε καθ' όλη τη διάρκεια εκπόνησης της διπλωματικής εργασίας. Η εμπιστοσύνη του και η ελευθερία που μου έδωσε στη διαμόρφωσή της συνέβαλαν ουσιαστικά στην ολοκλήρωσή της. Η συνεχής υποστήριξη, η επιστημονική του καθοδήγηση και η διαρκής παρουσία του αποτέλεσαν καθοριστικούς παράγοντες για την εξέλιξη της εργασίας, ενώ παράλληλα συνέβαλαν στην ανάπτυξη της κριτικής μου σκέψης και της ερευνητικής μου προσέγγισης.

Ιδιαίτερες ευχαριστίες οφείλω στους γονείς μου, Βάσω και Ποθητό, για τη διαρκή συμπαράσταση, την υπομονή και την ενθάρρυνση που μου προσέφεραν καθ' όλη τη διάρκεια των σπουδών μου. Η στήριξή τους, σε κάθε επίπεδο, υπήρξε καθοριστική για την ολοκλήρωση αυτής της προσπάθειας.

Τέλος, ευχαριστώ τους κοντινούς μου ανθρώπους για την παρουσία και τη στήριξή τους σε αυτή τη διαδρομή.

Περιεχόμενα

1 Εισαγωγή **6**
- 1.1 Γενικά . . . 6
- 1.2 Η Σημασία και ο Ρόλος των Αρχείων Καταγραφής (Log Files) στα Σύγχρονα Πληροφοριακά Συστήματα . . . 6
- 1.3 Ερευνητικές Προκλήσεις και Περιορισμοί των Υπάρχοντων Προσεγγίσεων 8
- 1.4 Στόχοι και Συμβολή της Διπλωματικής Εργασίας . . . 8
- 1.5 Δομή της Διπλωματικής Εργασίας . . . 9

2 Θεωρητικό Υπόβαθρο και Σχετικές Έννοιες **11**
- 2.1 Δεδομένα και Αρχεία Καταγραφής . . . 11
  - 2.1.1 Ορισμός και Χαρακτηριστικά Αρχείων Καταγραφής . . . 11
  - 2.1.2 Δομημένα και Αδόμητα Δεδομένα . . . 12
  - 2.1.3 Προκλήσεις Ανάλυσης και Ετερογένεια . . . 13
  - 2.1.4 Event Logs και το Πρότυπο XES . . . 14
- 2.2 Επεξεργασία Αρχείων Καταγραφής . . . 15
  - 2.2.1 Ανάλυση log μηνυμάτων (Log Parsing) . . . 15
  - 2.2.2 Μετατροπή σε Δομημένα Γεγονότα (Event Logs) . . . 18
  - 2.2.3 Εξόρυξη Δεδομένων . . . 18

3 Σχετικές Προσεγγίσεις - Εργαλεία **24**
- 3.1 Εργαλεία Log Parsing . . . 24
  - 3.1.1 Logstash . . . 24
  - 3.1.2 Drain . . . 25
  - 3.1.3 LogPPT . . . 26
  - 3.1.4 DivLog . . . 26
  - 3.1.5 ByteBrain . . . 27
- 3.2 Εργαλεία Process Mining . . . 28
  - 3.2.1 Declare4py . . . 28
  - 3.2.2 RuM . . . 29
  - 3.2.3 ASP-Based Declarative Process Mining . . . 30
  - 3.2.4 SIESTA Declare Mining . . . 31
- 3.3 Εργαλεία Πολλαπλών Διαδικασιών . . . 32
  - 3.3.1 Graylog . . . 32
  - 3.3.2 SigNoz . . . 33
  - 3.3.3 Elasticsearch . . . 33
- 3.4 Περιορισμοί . . . 34

4 Εφαρμογή Δηλωτικής Εξόρυξης Διαδικασιών σε Τυχαία Αρχεία Log: Μια Ολοκληρωμένη Προσέγγιση **36**
- 4.1 Επισκόπηση Υλοποίησης . . . 36
- 4.2 Παρουσίαση και Αξιοποίηση του LILAC στην Προτεινόμενη Μεθοδολογία 37
  - 4.2.1 Βασική Αρχιτεκτονική του LILAC . . . 37
  - 4.2.2 Τροποποιήσεις του LILAC . . . 39

4.2.3 Περιγραφή της Ενσωμάτωσης του LILAC στον κώδικα 40
4.3 Μετατροπή των αποτελεσμάτων parsing σε event logs και αρχεία XES . . 41
4.3.1 Διαχωρισμός πεδίων και δημιουργία CSV 42
4.3.2 Μετατροπή CSV αρχείου σε XES 44
4.4 Δηλωτική Εξόρυξη Διαδικασιών με χρήση του εργαλείου MINERful . . . 44
4.4.1 Βασική Λειτουργία του MINERful 44
4.4.2 Περιγραφή της Ενσωμάτωσης του MINERful στον κώδι-κα 46

5 Πειραματική Αξιολόγηση **48**
5.1 Πειραματική Διάταξη και Παράμετροι 48
5.1.1 Σύνολα Δεδομένων 49
5.2 Αξιολόγηση LILAC Parsing 49
5.2.1 F1 Template Accuracy και Parsing Accuracy 50
5.2.2 F1 Grouping Accuracy και Grouping Accuracy 53
5.2.3 Βέλτιστος συνδυασμός παραμέτρων 57
5.3 Αξιολόγηση Δημιουργίας Ιχνών για τα Αρχεία XES 61
5.4 Αξιολόγηση DECLARE Mining 63
5.4.1 Επίδραση της παραμέτρου avgIntervals 64
5.4.2 Επίδραση του Support Threshold 65
5.4.3 Συνολική Αποτίμηση της Εξόρυξης Περιορισμών 66
5.5 Χρόνος Εκτέλεσης 68
5.6 Αποτελέσματα και Συζήτηση 70

6 Συμπεράσματα, Περιορισμοί και Μελλοντική Έρευνα **72**
6.1 Σύνοψη 72
6.2 Περιορισμοί και Αδυναμίες 73
6.3 Κατευθύνσεις Μελλοντικής Έρευνας 73

A Παραδείγματα Χρήσης Εργαλείων Log Parsing **75**
A.1 Ενδεικτική Χρήση του Logstash 75
A.2 Ενδεικτική Χρήση του Drain 76
A.3 Ενδεικτική Χρήση του LogPPT 77

B Παράδειγμα Ροής της Υλοποίησης **78**

1 Εισαγωγή

1.1 Γενικά

Τα αρχεία καταγραφής (log files) αποτελούν ιδιαίτερα σημαντική πηγή πληροφορίας, καθώς περιγράφουν σχεδόν κάθε γεγονός που συμβαίνει σε ένα σύστημα. Η ανάλυση των αρχείων αυτών συχνά προϋποθέτει πολύπλοκες μεθοδολογίες, δεδομένου ότι είναι συνήθως εκτενή σε μέγεθος και ενδέχεται να παρουσιάζουν δομές οι οποίες δεν είναι άμεσα κατανοητές χωρίς τη χρήση εξειδικευμένων αλγορίθμων ή λογισμικών εργαλείων. Επιπλέον, λόγω της ετερογένειας των αρχείων καταγραφής, καθίσταται ιδιαίτερα δύσκολη η ανάπτυξη γενικών μεθόδων επεξεργασίας, ενώ ταυτόχρονα απαιτούνται πολλαπλές και διαφορετικές διαδικασίες για την αποδοτική αξιοποίησή τους.

Από τα παραδοσιακά έως και τα πιο σύγχρονα συστήματα, η καταγραφή των συμβάντων θεωρείται απαραίτητη για την παρατήρηση και κατανόηση της λειτουργίας τους. Η ανάλυση των αρχείων καταγραφής, αξιοποιείται από πλήθος επιστημονικών και τεχνολογικών τομέων τόσο για σχεδιαστικούς σκοπούς όσο και για την απόκτηση εξειδικευμένης γνώσης και την υποστήριξη κρίσιμων λειτουργιών. Η ανάλυση των logs εξυπηρετεί πολλούς στόχους που συμβάλλουν στη βελτίωση και εξέλιξη των συστημάτων, με κυριότερους τον εντοπισμό ανωμαλιών, μοτίβων και συσχετίσεων, τη συμπεριφορική ανάλυση συστημάτων, τον εντοπισμό επιθέσεων και αστοχιών, την κατανόηση και διάγνωση σφαλμάτων, τον υπολογισμό της απόδοσης, τη βελτιστοποίηση λειτουργιών, την εξόρυξη διαδικασιών, καθώς και την παρατήρηση της συμπεριφοράς χρηστών [18].

Συνοψίζοντας, η πολυπλοκότητα που χαρακτηρίζει την επεξεργασία και ανάλυση των αρχείων καταγραφής καθιστά αναγκαία τη βαθύτερη έρευνα και τη συνεχή επιστημονική συνεισφορά στον συγκεκριμένο κλάδο, καθώς τα ευρήματα που προκύπτουν από αυτήν παρουσιάζουν ευρύ φάσμα εφαρμογών και υψηλή πρακτική αξία. Στο πλαίσιο των σύγχρονων συστημάτων, η ανάλυση των αρχείων καταγραφής αποτελεί ένα ιδιαίτερα χρήσιμο εργαλείο, υποστηρίζοντας ουσιώδεις διεργασίες που είναι κρίσιμες για την επιστημονική και τεχνολογική κοινότητα.

1.2 Η Σημασία και ο Ρόλος των Αρχείων Καταγραφής (Log Files) στα Σύγχρονα Πληροφοριακά Συστήματα

Η ανάλυση των αρχείων καταγραφής αποτελεί θεμελιώδες εργαλείο για τη συστηματική μελέτη της λειτουργίας και της συμπεριφοράς σύγχρονων πληροφοριακών συστημάτων. Τα αρχεία αυτά συνιστούν βασική πηγή δεδομένων εκτέλεσης, καθώς καταγράφουν με ακρίβεια την εξέλιξη των εσωτερικών διεργασιών και των αλληλεπιδράσεων ενός συστήματος στον χρόνο, προσφέροντας πολύτιμα δεδομένα. Στο πλαίσιο αυτό, τα αρχεία καταγραφής δεν αντιμετωπίζονται απλώς ως βοηθητικά τεχνικά τεκμήρια, αλλά ως κεντρικό μέσο παρατήρησης, ανάλυσης και αξιολόγησης πολύπλοκων υπολογιστικών και πληροφοριακών υποδομών.

Αναλυτικότερα, η συνεισφορά των αρχείων καταγραφής γίνεται εμφανής μέσα από τη

χρησιμότητα και τις λειτουργίες που καθίστανται εφικτές χάρη σε αυτά. Τα αρχεία καταγραφής περιέχουν κρίσιμες πληροφορίες για τη λειτουργική κατάσταση, τις μεταβολές και τα συμβάντα ενός συστήματος, αποτελώντας σημαντική πηγή δεδομένων για την κατανόηση της πραγματικής του συμπεριφοράς κατά την εκτέλεση. Η συστηματική ανάλυσή τους επιτρέπει μια πληρέστερη κατανόηση της αλληλουχίας των γεγονότων, καθώς και τη συσχέτιση επιμέρους συμβάντων που, όταν εξετάζονται συνολικά, αποτυπώνουν με μεγαλύτερη σαφήνεια τη λειτουργία του συστήματος. Ως αποτέλεσμα, τα αρχεία καταγραφής συμβάλλουν ουσιαστικά στη διάγνωση προβλημάτων, στην αξιολόγηση της απόδοσης και στην αναγνώριση μοτίβων που συνδέονται είτε με την κανονική είτε με την αποκλίνουσα λειτουργία. Επιπλέον, τα αρχεία καταγραφής έχουν καθοριστικό ρόλο στα συστήματα ασφάλειας, καθώς μέσω αυτών καθίσταται δυνατός ο εντοπισμός ανωμαλιών, επιθέσεων και λειτουργικών σφαλμάτων, αλλά και η ανακατασκευή της ακριβούς ακολουθίας συμβάντων που οδήγησαν σε ένα περιστατικό. Παράλληλα, επειδή παράγονται αυτόματα κατά τη λειτουργία του συστήματος και μπορούν να υποβληθούν σε επεξεργασία σε πραγματικό χρόνο, αποτελούν ιδιαίτερα κατάλληλη βάση για συστηματικές και αυτοματοποιημένες τεχνικές ανάλυσης, καθώς και για αξιοποίηση ως είσοδοι σε μοντέλα Μηχανικής Μάθησης (ML) και Τεχνητής Νοημοσύνης (AI) [26].

Σε γενικότερο πλαίσιο, η ορθά και συστηματικά εφαρμοσμένη ανάλυση των αρχείων καταγραφής αποτελεί θεμέλιο για την αξιόπιστη λειτουργία και αξιολόγηση των σύγχρονων πληροφοριακών συστημάτων. Μέσω αυτής επιτυγχάνεται η ακριβής παρατήρηση της απόδοσης ενός συστήματος, η τεκμηριωμένη αξιολόγηση πειραματικών δοκιμών και η δυνατότητα αναπαραγωγής και επαλήθευσης των αποτελεσμάτων. Παράλληλα, ενισχύονται ιδιότητες όπως η διαφάνεια των διαδικασιών, η αξιοπιστία και η εμπιστοσύνη στα παραγόμενα συμπεράσματα, στοιχεία που το paper [25] αναδεικνύει ως κρίσιμα για πειραματικές μελέτες μεγάλης κλίμακας και συστήματα που αξιοποιούν δεδομένα. Κατά συνέπεια, η επεξεργασία και ανάλυση των αρχείων καταγραφής δεν αποτελεί απλώς ένα βοηθητικό στάδιο, αλλά έναν κεντρικό πυλώνα πάνω στον οποίο στηρίζονται η εγκυρότητα των συμπερασμάτων, η αξιοπιστία των συστημάτων και η λήψη τεκμηριωμένων αποφάσεων.

Σε επίπεδο εφαρμογών, τα αποτελέσματα που προκύπτουν από τη συστηματική ανάλυση των αρχείων καταγραφής μπορούν να αξιοποιηθούν σε ευρύ φάσμα επιστημονικών και τεχνολογικών τομέων. Ενδεικτικά, η ανάλυση των logs υποστηρίζει την παρακολούθηση και βελτιστοποίηση διαδικασιών σε ιατρικά και ερευνητικά συστήματα, τη μελέτη της συμπεριφοράς χρηστών σε ψηφιακές πλατφόρμες και εφαρμογές μάρκετινγκ, καθώς και την κατανόηση και βελτίωση εσωτερικών λειτουργιών σε εμπορικά και επιχειρησιακά περιβάλλοντα. Παράλληλα, σε συστήματα πληροφορικής και υποδομές μεγάλης κλίμακας, η ανάλυση αυτή συμβάλλει στην αυτοματοποίηση λειτουργιών, την ανίχνευση απειλών και την προληπτική συντήρηση, με εφαρμογές που εκτείνονται από λειτουργικά συστήματα έως ενεργειακά συστήματα. Τέλος, σε περιβάλλοντα ψηφιακής ψυχαγωγίας, όπως τα διαδικτυακά παιχνίδια και τα μέσα μαζικής δικτύωσης, η ανάλυση των αρχείων καταγραφής επιτρέπει τόσο τον εντοπισμό κακόβουλων ή μη εξουσιοδοτημένων συμπεριφορών όσο και τη συστηματική μελέτη των προτύπων αλληλεπίδρασης των χρηστών.

1.3 Ερευνητικές Προκλήσεις και Περιορισμοί των Υπάρχοντων Προσεγγίσεων

Η επεξεργασία των αρχείων καταγραφής συνιστά μια σύνθετη και πολυπαραγοντική διαδικασία. Η ανάγκη για αποτελεσματική ανάλυσή τους αυξάνεται διαρκώς, παράλληλα με την εκθετική αύξηση του όγκου των παραγόμενων log δεδομένων και την αυξανόμενη πολυπλοκότητα των σύγχρονων κατανεμημένων συστημάτων. Η ύπαρξη πολλαπλών και ετερογενών πηγών καταγραφής οδηγεί όχι μόνο σε δεδομένα μεγάλης κλίμακας, αλλά και σε έντονη δομική και σημασιολογική ανομοιογένεια, καθώς δεν υφίσταται ενιαίο πρότυπο μορφοποίησης και αναπαράστασης. Σε συνδυασμό με τη συνεχή παραγωγή δεδομένων σε πραγματικό χρόνο, προκύπτει η ανάγκη για διαδικασίες επεξεργασίας που να είναι ταυτόχρονα ακριβείς και κλιμακούμενες. Η απουσία τυποποίησης και κοινών πλαισίων αξιολόγησης δυσχεραίνει επιπλέον τη σύγκριση των ερευνητικών προσεγγίσεων, γεγονός που αναδεικνύεται ως βασικός περιορισμός στη σχετική βιβλιογραφία [35].

Ένα από τα σημαντικότερα προβλήματα αφορά το γεγονός ότι η προεπεξεργασία των logs παραμένει σε μεγάλο βαθμό χειροκίνητη, χρονοβόρα και εξαρτώμενη από εξειδικευμένη τεχνογνωσία. Παράλληλα, απαιτείται συμμόρφωση με κανονισμούς προστασίας προσωπικών δεδομένων, γεγονός που καθιστά τη διαδικασία ακόμη πιο απαιτητική. Η εγγενής ετερογένεια των logs, ως αποτέλεσμα της παραγωγής τους από διαφορετικά συστήματα και εφαρμογές, δημιουργεί προκλήσεις στην κανονικοποίηση και ενοποίησή τους, καθώς και στη δημιουργία γενικών μεθόδων για την επεξεργασία τους. Επιπλέον, η πληθώρα πεδίων και παραμέτρων καθιστά μη προφανή την επιλογή των πλέον κρίσιμων χαρακτηριστικών προς ανάλυση. Ακόμη, η αποτελεσματική μοντελοποίηση των log δεδομένων και η εξαγωγή ουσιαστικών προτύπων από αυτά απαιτούν μεθόδους ικανές να διαχειριστούν τη δομική και σημασιολογική πολυπλοκότητά τους.

Πέρα από τις δυσκολίες της αρχικής προεπεξεργασίας, κρίσιμο ζήτημα αποτελεί και η αξιοπιστία των ενδιάμεσων σταδίων μετασχηματισμού. Από τη δομική ανάλυση έως τη μετατροπή των δεδομένων σε κατάλληλη αναπαράσταση γεγονότων, κάθε βήμα μπορεί να εισαγάγει σφάλματα, απλοποιήσεις ή μεροληπτικές παραδοχές που επηρεάζουν το τελικό αποτέλεσμα. Συνεπώς, η ακρίβεια των πρώτων σταδίων επεξεργασίας είναι καθοριστική για την εγκυρότητα των παραγόμενων μοντέλων και συμπερασμάτων. Για τον λόγο αυτό, αναδεικνύεται η ανάγκη για μεθοδολογική αυστηρότητα, σαφή ορισμό των σταδίων μετασχηματισμού και διαφανή τεκμηρίωση της διαδικασίας, ώστε οι προσεγγίσεις ανάλυσης αρχείων καταγραφής να παραμένουν επιστημονικά τεκμηριωμένες και αξιόπιστες [12].

1.4 Στόχοι και Συμβολή της Διπλωματικής Εργασίας

Η παρούσα εργασία στοχεύει να συμβάλει στην αντιμετώπιση των προαναφερθέντων προκλήσεων, εστιάζοντας στη συστηματοποίηση των βασικών σταδίων επεξεργασίας αρχείων καταγραφής. Όπως αναφέρθηκε, η ανάλυση των logs αποτελεί μια σύνθετη διαδικασία

που περιλαμβάνει διακριτά και αλληλοεξαρτώμενα στάδια, καθένα από τα οποία απαιτεί ουσιαστική κατανόηση τόσο της δομής των δεδομένων όσο και του υποκείμενου συστήματος. Παράλληλα, η εγγενής ετερογένεια των αρχείων καταγραφής δυσχεραίνει την εφαρμογή καθολικών μεθόδων, με αποτέλεσμα πολλές υπάρχουσες προσεγγίσεις να είναι προσαρμοσμένες σε συγκεκριμένα περιβάλλοντα.

Στο πλαίσιο αυτό, η εργασία αξιοποιεί τεχνικές log parsing για την εξαγωγή κατηγοριών γεγονότων από ακατέργαστα αρχεία καταγραφής. Συγκεκριμένα, χρησιμοποιεί τον αλγόριθμο LILAC, τον οποίο προσαρμόζει με στόχο τη δυνατότητα εφαρμογής του σε αρχεία μορφής .log ανεξαρτήτως προέλευσης. Στη συνέχεια, τα παραγόμενα αποτελέσματα μετασχηματίζονται μέσω κατάλληλων διαδικασιών σε δομημένη αναπαράσταση, ώστε να καθίστανται συμβατά με συστήματα εξόρυξης γνώσης. Ακολούθως, εφαρμόζεται το σύστημα MINERful για την εξαγωγή κανόνων τύπου DECLARE, δηλαδή δηλωτικών περιορισμών που περιγράφουν σχέσεις και εξαρτήσεις μεταξύ γεγονότων.

Η συμβολή της εργασίας έγκειται στη διαμόρφωση μιας ενιαίας και συνεκτικής ροής επεξεργασίας, η οποία εκτείνεται από το ακατέργαστο αρχείο καταγραφής έως την εξαγωγή δομημένων περιορισμών. Με τον τρόπο αυτό, προτείνεται ένα γενικό πλαίσιο που επιτρέπει τη διαχείριση logs διαφορετικής προέλευσης, μειώνοντας την ανάγκη για αποσπασματικές και χειροκίνητες παρεμβάσεις στα επιμέρους στάδια.

Συνολικά, η εργασία επιδιώκει να λειτουργήσει ως ενοποιητικός μηχανισμός μεταξύ parsing, μετασχηματισμού δεδομένων και δηλωτικής εξόρυξης διαδικασιών, προσφέροντας μια συστηματική και επαναχρησιμοποιήσιμη προσέγγιση στην ανάλυση αρχείων καταγραφής.

1.5 Δομή της Διπλωματικής Εργασίας

Η παρούσα διπλωματική εργασία οργανώνεται σε έξι κεφάλαια, τα οποία αναπτύσσονται ως εξής:

Στο Κεφάλαιο 2 παρουσιάζεται το θεωρητικό πλαίσιο της εργασίας. Εισάγονται βασικοί ορισμοί και θεμελιώδεις έννοιες που αφορούν τα αρχεία καταγραφής, ενώ παράλληλα αναλύονται σε εννοιολογικό επίπεδο τα κύρια στάδια επεξεργασίας και ανάλυσής τους.

Στο Κεφάλαιο 3 πραγματοποιείται ανασκόπηση της σχετικής βιβλιογραφίας. Εξετάζονται υφιστάμενα συστήματα και εργαλεία επεξεργασίας log δεδομένων, αναλύεται η λειτουργία τους και συγκρίνονται οι δυνατότητές τους ως προς το εύρος των υποστηριζόμενων σταδίων, από την αρχική επεξεργασία έως την εξαγωγή γνώσης και προτύπων συμπεριφοράς.

Το Κεφάλαιο 4 περιγράφει την προτεινόμενη μεθοδολογία και την υλοποίηση του συστήματος. Παρουσιάζονται οι αλγόριθμοι που αξιοποιούνται στα επιμέρους στάδια (parsing, μετασχηματισμός, εξόρυξη περιορισμών), καθώς και η συνολική αρχιτεκτονική και η ροή επεξεργασίας των δεδομένων.

Στο Κεφάλαιο 5 παρουσιάζεται η πειραματική διαδικασία και η αξιολόγηση της προσέγγισης. Αναλύονται τα σύνολα δεδομένων που χρησιμοποιήθηκαν, περιγράφονται τα σενάρια δοκιμών και συζητούνται τα αποτελέσματα που προέκυψαν.

Τέλος, το Κεφάλαιο 6 συνοψίζει τα κύρια συμπεράσματα της εργασίας, αποτιμά τη συμβολή της, αναγνωρίζει τους περιορισμούς της προτεινόμενης λύσης και προτείνει κατευθύνσεις για μελλοντική έρευνα και πιθανή εφαρμογή σε πραγματικά συστήματα.

2 Θεωρητικό Υπόβαθρο και Σχετικές Έννοιες

Το παρόν κεφάλαιο παρουσιάζει το θεωρητικό υπόβαθρο που είναι απαραίτητο για την κατανόηση του αντικειμένου της εργασίας και εισάγει βασικές έννοιες που σχετίζονται με τα αρχεία καταγραφής και την επεξεργασία τους. Αρχικά παρουσιάζονται θεμελιώδεις έννοιες σχετικά με τα αρχεία καταγραφής, τα χαρακτηριστικά τους και τη δομή των δεδομένων που περιέχουν. Στη συνέχεια αναλύονται οι βασικές αρχές και διαδικασίες που χρησιμοποιούνται για την επεξεργασία και τον μετασχηματισμό των log δεδομένων. Τέλος, εξετάζεται η σχέση των αρχείων καταγραφής με τον ευρύτερο χώρο της εξόρυξης δεδομένων και διαδικασιών.

2.1 Δεδομένα και Αρχεία Καταγραφής

2.1.1 Ορισμός και Χαρακτηριστικά Αρχείων Καταγραφής

Σύμφωνα με το Πανεπιστήμιο του Cambridge [2], ένα **αρχείο καταγραφής (log file)** είναι ένα αρχείο που περιέχει καταγραφές, δηλαδή περιγραφές των δραστηριοτήτων που πραγματοποιούνται σε ένα υπολογιστικό σύστημα, σε μια εφαρμογή ή σε μια ιστοσελίδα.

Οι επιμέρους καταγραφές ονομάζονται **log messages** και αποτελούν μηνύματα τα οποία δημιουργούνται από το ίδιο το σύστημα με σκοπό την περιγραφή των γεγονότων που συμβαίνουν κατά τη λειτουργία του. Οι πληροφορίες που περιέχονται σε αυτά τα μηνύματα αναφέρονται ως **log data** [4, p. 2].

Στη βιβλιογραφία έχουν προταθεί διάφορες κατηγοριοποιήσεις των log μηνυμάτων. Μία συνηθισμένη προσέγγιση βασίζεται στην πηγή προέλευσης των μηνυμάτων, σύμφωνα με την οποία τα logs διακρίνονται σε μηνύματα συστήματος, εφαρμογής, ασφάλειας και συμπεριφοράς χρήστη [35].

Μια εναλλακτική κατηγοριοποίηση βασίζεται στον τύπο της πληροφορίας που μεταφέρει το μήνυμα. Σύμφωνα με το [4, p. 3], τα log μηνύματα μπορούν να διακριθούν σε ενημερωτικά (informational), αποσφαλμάτωσης (debug), προειδοποίησης (warning), σφάλματος (error) και συναγερμού (alert).

Τα πληροφοριακά μηνύματα (informational logs) και τα μηνύματα αποσφαλμάτωσης (debug logs) έχουν κυρίως ενημερωτικό χαρακτήρα και καταγράφουν την κανονική λειτουργία ενός συστήματος ή παρέχουν λεπτομερείς πληροφορίες χρήσιμες κατά τη διαδικασία εντοπισμού σφαλμάτων. Τα προειδοποιητικά μηνύματα (warning logs) υποδεικνύουν ότι μια λειτουργία δεν εκτελέστηκε με τον αναμενόμενο τρόπο, χωρίς όμως να εμποδίζεται απαραίτητα η συνέχιση της λειτουργίας του συστήματος. Αντίθετα, τα μηνύματα σφαλμάτων (error logs) καταγράφουν αποτυχίες λειτουργιών ή προβλήματα που επηρεάζουν άμεσα τη λειτουργία του συστήματος. Τέλος, τα μηνύματα συναγερμού (alert logs) σχετίζονται συνήθως με κρίσιμα γεγονότα, όπως ζητήματα ασφάλειας ή άλλες μη αναμενόμενες καταστάσεις που απαιτούν άμεση προσοχή.

Υπάρχουν διάφοροι τύποι αρχείων καταγραφής, όπως δυαδικά αρχεία (binary logs), αρχεία που αποθηκεύονται σε σχεσιακές βάσεις δεδομένων και αρχεία καταγραφής χαρακτήρων ASCII. Οι δύο πρώτες κατηγορίες δεν είναι άμεσα αναγνώσιμες από τον άνθρωπο και συνήθως ακολουθούν μια πιο αυστηρά καθορισμένη δομή σε σύγκριση με τα αρχεία

χαρακτήρων ASCII [4, p. 38–40]. Η παρούσα διπλωματική εργασία εστιάζει στην τελευταία κατηγορία, δηλαδή στα αρχεία καταγραφής χαρακτήρων ASCII.

Τα log μηνύματα αποτελούνται συνήθως από μια δομημένη ακολουθία πεδίων που περιγράφουν ένα γεγονός το οποίο συνέβη σε ένα σύστημα. Κάθε καταγραφή οργανώνεται σε επιμέρους πληροφοριακά στοιχεία, τα οποία αποτυπώνουν διαφορετικές πτυχές του συμβάντος που καταγράφεται. Μέσω αυτής της δομής καθίσταται δυνατή η συστηματική αποθήκευση και επεξεργασία των δεδομένων καταγραφής, διευκολύνοντας διαδικασίες όπως η παρακολούθηση της λειτουργίας του συστήματος, η ανάλυση συμβάντων και ο εντοπισμός σφαλμάτων. Η ακριβής μορφή και το πλήθος των πεδίων μπορεί να διαφέρει ανάλογα με το σύστημα ή την εφαρμογή που παράγει τα logs.

Ένα παράδειγμα μηνύματος log από καταγεγραμμένα logs του λειτουργικού συστήματος Android, όπως εμφανίζονται στα συγκεντρωμένα σύνολα δεδομένων που έχουν δημοσιευθεί στη βιβλιογραφία [16], [38], παρουσιάζεται παρακάτω:

```
03-17 16:16:07.635 1702 1820 I DisplayPowerController:  HBM
brightnessIn =38
```

2.1.2 Δομημένα και Αδόμητα Δεδομένα

Με βάση τον τρόπο οργάνωσης και αναπαράστασης της πληροφορίας τους, τα δεδομένα μπορούν να κατηγοριοποιηθούν σε δομημένα, ημι-δομημένα και αδόμητα. Η διάκριση αυτή είναι ιδιαίτερα σημαντική, καθώς επηρεάζει τον τρόπο με τον οποίο τα δεδομένα μπορούν να αποθηκευτούν, να επεξεργαστούν και να αναλυθούν. Παράλληλα, με την εξέλιξη της τεχνολογίας παρατηρείται συνεχής αύξηση του όγκου των μη δομημένων δεδομένων, γεγονός που δημιουργεί πρόσθετες προκλήσεις ως προς τη διαχείριση και την επεξεργασία τους.

Όπως παρουσιάζεται στο [24], τα δομημένα δεδομένα αφορούν πληροφορίες οργανωμένες με σαφή και προκαθορισμένο τρόπο, ο οποίος επιτρέπει την εύκολη αποθήκευση και επεξεργασία τους σε παραδοσιακές σχεσιακές βάσεις δεδομένων. Συνήθως οργανώνονται σε αρχεία κειμένου που μπορούν να χωριστούν σε γραμμές και στήλες, γεγονός που διευκολύνει τη χρήση τους σε εφαρμογές ανάλυσης και εξόρυξης δεδομένων.

Στην επόμενη κατηγορία ανήκουν τα ημι-δομημένα δεδομένα, τα οποία διαθέτουν μια εσωτερική δομή χωρίς όμως να ακολουθούν ένα αυστηρά προκαθορισμένο σχήμα. Χαρακτηριστικό παράδειγμα αποτελούν τα αρχεία XML και όσον αφορά τα logs, τα αρχεία μορφής `.xes`, τα οποία χρησιμοποιούνται συχνά στον χώρο της εξόρυξης διαδικασιών. Τα αρχεία αυτά οργανώνουν τα δεδομένα καταγραφής σε γεγονότα και ίχνη εκτέλεσης, επιτρέποντας έτσι την εφαρμογή εργαλείων ανάλυσης και εξόρυξης διαδικασιών.

Αντίθετα, τα περισσότερα αρχεία καταγραφής ανήκουν στην κατηγορία των αδόμητων δεδομένων. Αυτό σημαίνει ότι δεν ακολουθούν κάποιο αυστηρό πρότυπο δομής, γεγονός που καθιστά πιο δύσκολη την άμεση επεξεργασία και ανάλυσή τους. Για τον λόγο αυτό συχνά απαιτείται μια επιπλέον διαδικασία μετασχηματισμού, κατά την οποία εντοπίζονται μοτίβα και πρότυπα μέσα στα μηνύματα καταγραφής, ώστε τα δεδομένα να μετατραπούν σε πιο δομημένη μορφή που να μπορεί να αξιοποιηθεί από εργαλεία ανάλυσης και εξόρυξης

δεδομένων [24].

2.1.3 Προκλήσεις Ανάλυσης και Ετερογένεια

Παρότι πολλά `.log` αρχεία παρουσιάζουν ορισμένα κοινά χαρακτηριστικά, η ακριβής μορφή και δομή τους μπορεί να διαφέρει σημαντικά μεταξύ διαφορετικών συστημάτων και εφαρμογών.

Όλα τα log μηνύματα περιλαμβάνουν κάποιο πεδίο χρονικής πληροφορίας που δηλώνει τη στιγμή κατά την οποία συνέβη το καταγεγραμμένο γεγονός. Ωστόσο, ακόμη και τα χρονικά αυτά πεδία δεν εμφανίζονται πάντα με την ίδια μορφή. Συνήθεις εκφράσεις της χρονικής πληροφορίας είναι η χρονική σήμανση (timestamp), η ώρα (time) και η ημερομηνία (date).

Πέρα από ορισμένα συχνά χρησιμοποιούμενα στοιχεία, δεν υπάρχει ένα καθολικό πρότυπο για τη δομή των log μηνυμάτων. Ως αποτέλεσμα, η μορφή τους μπορεί να διαφέρει σημαντικά μεταξύ διαφορετικών συστημάτων. Παρόλα αυτά, αρκετά log αρχεία περιλαμβάνουν πεδία όπως το επίπεδο σοβαρότητας (level), το υποσύστημα ή στοιχείο του συστήματος (component), τον χρήστη (user ή userID), το περιεχόμενο του μηνύματος (content ή message) και την ταυτότητα της διεργασίας (process ID ή PID). Η ακριβής σύνθεση των πεδίων αυτών εξαρτάται από το σύστημα που παράγει τα logs.

Η ετερογένεια αυτή μπορεί να παρατηρηθεί και σε πραγματικά σύνολα δεδομένων καταγραφής. Για παράδειγμα, για δύο από τα σύνολα δεδομένων των [16], [38], μία ενδεικτική μορφή του περιεχομένου μιας καταγραφής και της αντίστοιχης μορφοποίησης των πεδίων (log format) παρουσιάζεται παρακάτω.

- Για το σύνολο δεδομένων του HealthApp: Μία καταγραφή είναι:

  ```
  20171223-22:15:29:959|Step_ExtSDM|30002312|calculateCaloriesWithCache
  totalCalories=126797
  ```

  Ενώ η μορφοποίηση των πεδίων της είναι:

  ```
  <Time>|<Component>|<Pid>|<Content>
  ```

- Για το σύνολο δεδομένων του BGL: Μία καταγραφή είναι:

  ```
  APPREAD 1117869876 2005.06.04 R27-M1-N4-I:J18-U01
  2005-06-04-00.24.36.222560 R27-M1-N4-I:J18-U01 RAS APP
  FATAL ciod:  failed to read message prefix on control stream
  (CioStream socket to 172.16.96.116:33370
  ```

  Ενώ η μορφοποίηση των πεδίων της είναι:

  ```
  <Label> <Timestamp> <Date> <Node> <Time> <NodeRepeat> <Type>
  <Component> <Level> <Content>
  ```

Η διαφοροποίηση αυτή στη μορφή των καταγραφών αποτελεί μία από τις βασικές προκλήσεις στην ανάλυση αρχείων καταγραφής. Στην πράξη, τα πληροφοριακά συστήματα λειτουργούν συνεχώς και παράγουν πολύ μεγάλο όγκο log δεδομένων, τα οποία προέρχονται από διαφορετικές εφαρμογές, υποσυστήματα και υπολογιστικά περιβάλλοντα. Ως αποτέλεσμα, τα δεδομένα καταγραφής χαρακτηρίζονται τόσο από μεγάλο μέγεθος όσο και από σημαντική ετερογένεια στη δομή και το περιεχόμενό τους, γεγονός που δυσχεραίνει την ανάπτυξη γενικευμένων μεθόδων επεξεργασίας και ανάλυσης [35].

2.1.4 Event Logs και το Πρότυπο XES

Τα `.log` αρχεία αποτελούν τη μη δομημένη μορφή των καταγραφών που παράγονται κατά τη λειτουργία ενός πληροφοριακού συστήματος. Για την υποστήριξη πιο συστηματικής ανάλυσης των δεδομένων αυτών έχουν προταθεί πρότυπα αναπαράστασης που οργανώνουν τις καταγραφές σε πιο δομημένη μορφή. Ένα από τα πιο διαδεδομένα πρότυπα στον χώρο της εξόρυξης διαδικασιών είναι το πρότυπο `.xes`, το οποίο χρησιμοποιείται για την αναπαράσταση event logs.

Τα αρχεία `.xes` (eXtensible Event Stream), σύμφωνα με το πρότυπο του IEEE [14], αποτελούν μια μορφή αρχείων βασισμένη στη γλώσσα XML και έχουν ως σκοπό τη δημιουργία ενός κοινού προτύπου αναπαράστασης δεδομένων καταγραφής. Η μορφή αυτή επιτρέπει την οργάνωση των δεδομένων καταγραφής με τρόπο που διευκολύνει την εφαρμογή μεθόδων ανάλυσης και εξόρυξης διαδικασιών, συμβάλλοντας έτσι στη βελτίωση της γενίκευσης και της διαλειτουργικότητας των εργαλείων επεξεργασίας.

Ένα αρχείο `.xes` οργανώνεται ιεραρχικά. Στο ανώτερο επίπεδο βρίσκεται το log, το οποίο περιλαμβάνει ένα σύνολο από ίχνη (traces). Κάθε trace αντιστοιχεί σε μία εκτέλεση μιας διαδικασίας και αποτελείται από μια ακολουθία γεγονότων (events). Τα events περιγράφουν επιμέρους γεγονότα που συνέβησαν κατά την εκτέλεση της διαδικασίας και συνοδεύονται από χαρακτηριστικά όπως η χρονική σήμανση, ο πόρος ή ο τύπος δραστηριότητας.

Όπως φαίνεται στο παράδειγμα 2.1 από το [13], η ιεραρχική δομή ενός αρχείου `.xes` αποτυπώνεται μέσω των στοιχείων XML. Το στοιχείο `<log>` περιλαμβάνει μεταδεδομένα για το σύνολο των καταγραφών, ενώ στο εσωτερικό του βρίσκονται ένα ή περισσότερα στοιχεία `<trace>`, τα οποία αντιστοιχούν σε επιμέρους εκτελέσεις της διαδικασίας. Κάθε trace περιέχει διαδοχικά στοιχεία `<event>`, τα οποία περιγράφουν συγκεκριμένα γεγονότα και συνοδεύονται από επιμέρους χαρακτηριστικά που αποτυπώνονται ως ζεύγη `key-value`.

```
<log xes.version="1.3" xes.features="nested-attributes" openxes.version="1.8" xmlns=
    "http://www.xes-standard.org/">
  <string key="Creator" value="Fluxicon Nitro"/>
  <trace>
    <string key="Name" value="Case1280"/>
    <string key="Creator" value="Fluxicon Nitro"/>
    <event>
      <string key="Resource" value="FL"/>
      <date key="Timestamp" value="2010-03-15T07:59:00.000+02:00"/>
      <string key="Operation" value="Handle Email"/>
```

```
      <string key="Agent Position" value="FL"/>
      <string key="Customer ID" value="Customer 1074"/>
      <string key="Product" value="iPhone"/>
      <string key="Service Type" value="Product Assistance"/>
      <string key="Agent" value="Susi"/>
    </event>
    <event>
    . . .
    </ event>
    . . .
  </trace>
  <trace>
  . . .
  </ trace>
  . . .
</log>
```

2.1: Παράδειγμα δομής XES αρχείου

2.2 Επεξεργασία Αρχείων Καταγραφής

Στα πλαίσια της παρούσας εργασίας εξετάζονται βασικές λειτουργίες επεξεργασίας αρχείων καταγραφής. Αρχικά εφαρμόζεται η διαδικασία log parsing, μέσω της οποίας εξάγονται δομημένες πληροφορίες από τα μη δομημένα log μηνύματα. Στη συνέχεια τα αποτελέσματα μετατρέπονται σε αρχεία `.xes`, τα οποία χρησιμοποιούνται ως είσοδος για την εφαρμογή τεχνικών εξόρυξης διαδικασιών. Στην ενότητα αυτή παρουσιάζεται η βασική λογική που διέπει τις παραπάνω διαδικασίες.

2.2.1 Ανάλυση log μηνυμάτων (Log Parsing)

Το log parsing είναι μια διαδικασία που μετασχηματίζει τα αδόμητα αρχεία καταγραφής σε πιο δομημένη μορφή, διαχωρίζοντας τις καταγραφές ή τα log μηνύματα στα επιμέρους δομικά τους στοιχεία. Μέσω της διαδικασίας αυτής μπορούν να εξαχθούν πεδία μιας καταγραφής, όπως η χρονική πληροφορία, η ταυτότητα διεργασίας το μήνυμα ή ό,τι άλλο πεδίο υπάρχει, επιτρέποντας την αποθήκευση των δεδομένων σε μορφές όπως .json ή .csv ή τη διατήρησή τους σε δομημένη μορφή μέσα στον κώδικα για περαιτέρω επεξεργασία [4, p. 122].

Παράλληλα, σημαντικό μέρος του log parsing αποτελεί η ανάλυση του ίδιου του περιεχομένου των log μηνυμάτων. Τα μηνύματα που παράγονται από ένα πληροφοριακό σύστημα περιγράφουν γεγονότα που συμβαίνουν κατά τη λειτουργία του και συνήθως αποτελούνται από ένα σταθερό και ένα μεταβλητό μέρος. Το σταθερό μέρος αντιστοιχεί στον τύπο του γεγονότος, για παράδειγμα `"user login"`, ενώ το μεταβλητό μέρος περιλαμβάνει ειδικότερες πληροφορίες σχετικά με το συμβάν, όπως το όνομα χρήστη, π.χ. `username_example1234`. Μέσω της διαδικασίας parsing εντοπίζονται τα επαναλαμβανόμενα πρότυπα των μηνυμάτων, τα οποία αντιστοιχούν σε διαφορετικούς τύπους γεγονότων,

καθώς και οι μεταβλητές παράμετροι που διαφοροποιούνται μεταξύ των καταγραφών. Τα μηνύματα αυτά εντοπίζονται συνήθως σε πεδία που αναφέρονται ως περιεχόμενο ή μήνυμα.

Ένα απλό παράδειγμα της διαδικασίας parsing φαίνεται παρακάτω για μια καταγραφή από το σύνολο δεδομένων του HealthApp:

```
20171223-22:15:29:959|Step_ExtSDM|30002312|calculateCaloriesWithCache
totalCalories=126797
```

- Το parsing ολόκληρης της καταγραφής θα έχει ώς αποτέλεσμα:

  ```
  "Time":20171223-22:15:29:959
  "Component":  Step_ExtSDM
  "Pid":  30002312
  "Content":calculateCaloriesWithCache totalCalories=126797
  ```

- Ενώ το parsing του μηνύματος θα είναι:

  ```
  "event template":calculateCaloriesWithCache totalCalories=<*>
  ```

Η εξαγωγή των προτύπων αυτών μπορεί να πραγματοποιηθεί με διάφορες προσεγγίσεις, όπως μεθόδους βασισμένες σε μοτίβα, σε συντακτική ανάλυση, σε σημασιολογική επεξεργασία ή σε μεγάλα γλωσσικά μοντέλα.

Συστήματα Επεξεργασίας Βασισμένα σε Μοτίβα

Μια συχνή προσέγγιση στην επεξεργασία αρχείων καταγραφής βασίζεται σε τεχνικές αντιστοίχισης προτύπων (pattern matching). Στα συστήματα αυτά απαιτείται ο εκ των προτέρων καθορισμός κανονικών εκφράσεων (regular expressions) ή προτύπων grok (grok patterns), είτε μέσω χειροκίνητης είτε μέσω αλγοριθμικής διαδικασίας, με σκοπό την εξαγωγή δομημένων πεδίων από τα αρχεία log.

Η λειτουργία τους βασίζεται στην ύπαρξη προκαθορισμένων κανόνων ανάλυσης (parsing rules), η δημιουργία και συντήρηση των οποίων απαιτεί εξειδικευμένη γνώση της δομής των δεδομένων και των ιδιαιτεροτήτων του συστήματος που παράγει τις καταγραφές [39].

Ωστόσο, λόγω της ετερογένειας και της συχνής μεταβολής των μορφών των καταγραφών, η διαχείριση και συντήρηση ενός μεγάλου συνόλου κανόνων μπορεί να καταστεί ιδιαίτερα πολύπλοκη, παρόλο που η προσέγγιση αυτή παραμένει αποτελεσματική όταν οι κανόνες ανταποκρίνονται με ακρίβεια στα δεδομένα εισόδου.

Στις επόμενες υποενότητες παρουσιάζονται συστήματα που επικεντρώνονται κυρίως στην ανάλυση του περιεχομένου των log μηνυμάτων και στον εντοπισμό των αντίστοιχων προτύπων γεγονότων.

Συστήματα Επεξεργασίας Βασισμένα στη Σύνταξη

Ως εναλλακτική προσέγγιση των συστημάτων βασισμένων σε κανόνες, έχουν αναπτυχθεί αυτοματοποιημένοι parsers των οποίων η λειτουργία βασίζεται σε συντακτικούς κανόνες

και ευρετικές μεθόδους (heuristics). Οι αλγόριθμοι αυτοί ομαδοποιούν τα μηνύματα καταγραφής μέσω της αναγνώρισης κοινών δομικών χαρακτηριστικών, όπως διαχωριστικών συμβόλων (delimiters) και της συχνότητας εμφάνισης λεκτικών μονάδων (tokens), επιτρέποντας την εξαγωγή προτύπων μηνυμάτων (message templates) χωρίς ανθρώπινη παρέμβαση.

Ως αποτέλεσμα της διαδικασίας αυτής, κάθε εγγραφή καταγραφής αντιστοιχίζεται σε μία ομάδα (cluster) της οποίας τα μέλη μοιράζονται το ίδιο πρότυπο μηνύματος και συνεπώς αντιπροσωπεύουν τον ίδιο τύπο γεγονότος (event).

Παρά το γεγονός ότι οι μέθοδοι αυτές επιτρέπουν την αυτοματοποιημένη αναγνώριση προτύπων μηνυμάτων, βασίζονται κυρίως σε επιφανειακά δομικά χαρακτηριστικά των καταγραφών. Για τον λόγο αυτό έχουν αναπτυχθεί προσεγγίσεις που επιχειρούν να αξιοποιήσουν και τη σημασιολογική πληροφορία των μηνυμάτων.

Σημασιολογικά Συστήματα Επεξεργασίας

Μια πιο πρόσφατη κατηγορία προσεγγίσεων στο log parsing βασίζεται στη σημασιολογική ανάλυση των μηνυμάτων καταγραφής.Η ανάλυση αυτή αξιοποιεί τεχνικές μηχανικής μάθησης και επεξεργασίας φυσικής γλώσσας (natural language processing). Σε αντίθεση με τις μεθόδους που βασίζονται κυρίως στη συντακτική δομή των μηνυμάτων, οι προσεγγίσεις αυτές επιχειρούν να αναλύσουν τη συμφραστική σημασία (contextual meaning) και τους εννοιολογικούς ρόλους (semantic roles) των στοιχείων που περιλαμβάνονται στα log μηνύματα.

Με τον τρόπο αυτό καθίσταται δυνατή η αναγνώριση προτύπων γεγονότων ακόμη και σε περιπτώσεις όπου τα μηνύματα παρουσιάζουν μεγαλύτερη ποικιλία στη μορφή τους, επιτυγχάνοντας συχνά υψηλότερη ακρίβεια σε σύνθετα, ετερογενή και διαρκώς εξελισσόμενα περιβάλλοντα καταγραφής. Η εξέλιξη των τεχνικών επεξεργασίας φυσικής γλώσσας οδήγησε περαιτέρω στην αξιοποίηση μεγάλων γλωσσικών μοντέλων για την ανάλυση των αρχείων καταγραφής.

Συστήματα Επεξεργασίας Βασισμένα σε Μεγάλα Γλωσσικά Μοντέλα

Τα συστήματα επεξεργασίας που βασίζονται σε Μεγάλα Γλωσσικά Μοντέλα (Large Language Models – LLMs) αποτελούν μια πρόσφατη προσέγγιση στο log parsing. Οι μέθοδοι αυτές αξιοποιούν τη βαθιά κατανόηση φυσικής γλώσσας και τις δυνατότητες μάθησης εντός συμφραζομένων (in-context learning) των LLMs για την ερμηνεία των μηνυμάτων καταγραφής.

Σε αντίθεση με προσεγγίσεις που βασίζονται σε προκαθορισμένους κανόνες ή σε εξειδικευμένη εκπαίδευση μοντέλων (fine-tuning), οι αναλυτές που βασίζονται σε LLMs αντιμετωπίζουν την ανάλυση των logs ως μια παραγωγική διαδικασία (generative task), η οποία καθοδηγείται από οδηγίες και παραδείγματα σε φυσική γλώσσα. Η προσέγγιση αυτή επιτρέπει καλύτερη γενίκευση, διαχείριση σύνθετων σημασιολογικών αμφισημιών και προσαρμογή σε νέες μορφές καταγραφής ακόμη και με περιορισμένο αριθμό παραδειγμάτων.

Οι διαφορετικές αυτές προσεγγίσεις αντικατοπτρίζουν την εξέλιξη των τεχνικών επεξ-

εργασίας αρχείων καταγραφής, από μεθόδους βασισμένες σε κανόνες έως σύγχρονες προσεγγίσεις που αξιοποιούν τεχνικές μηχανικής μάθησης και μεγάλα γλωσσικά μοντέλα.

2.2.2 Μετατροπή σε Δομημένα Γεγονότα (Event Logs)

Η διαδικασία αυτή αφορά τη μετατροπή των αποτελεσμάτων του log parsing σε αρχεία `.xes`, τα οποία χρησιμοποιούνται για την αναπαράσταση event logs και την εφαρμογή τεχνικών εξόρυξης διαδικασιών.

Αρχικά, εντοπίζονται τα χρονικά πεδία που περιλαμβάνονται στις δομημένες πλέον καταγραφές. Τα πεδία αυτά μετατρέπονται σε μορφή timestamp ώστε να διασφαλίζεται η συμβατότητα με το πρότυπο `.xes` και να είναι δυνατή η σωστή χρονική διάταξη των γεγονότων.

Ιδιαίτερα σημαντικό στάδιο αποτελεί η δημιουργία των ιχνών (traces), δηλαδή η ομαδοποίηση των γεγονότων σε ακολουθίες. Η ομαδο-ποίηση αυτή μπορεί να πραγματοποιηθεί με διαφορετικά κριτήρια, ανάλογα με τα διαθέσιμα πεδία των καταγραφών. Για παράδειγμα, μπορεί να χρησιμοποιηθεί η ταυτότητα διεργασίας (process ID), η ταυτότητα χρήστη ή κάποιο άλλο κοινό χαρακτηριστικό που επιτρέπει τη συσχέτιση των σχετικών γεγονότων.

Στην παρούσα εργασία, η ομαδοποίηση των γεγονότων πραγματοποιείται με βάση χρονικά παράθυρα. Κάθε ίχνος αποτελείται από γεγονότα που συμβαίνουν μέσα σε ένα ορισμένο χρονικό διάστημα και διατάσσονται με βάση τη χρονική τους σειρά. Η προσέγγιση αυτή αποτελεί ουσιαστικό στοιχείο της προτεινόμενης μεθοδολογίας, καθώς ενοποιεί λειτουργικά το στάδιο του log parsing με τη δημιουργία event logs για να εφαρμοστεί process mining, δύο στάδια που στις υπάρχουσες προσεγγίσεις αντιμετωπίζονται ξεχωριστά. Παράλληλα, συμβάλλει στην αυτοματοποίηση της συνολικής διαδικασίας και ενισχύει τη συγκρότηση μιας πιο ολοκληρωμένης ροής επεξεργασίας, περιορίζοντας την ανάγκη για εκτεταμένες χειροκίνητες παρεμβάσεις μεταξύ των επιμέρους σταδίων.

Επιπλέον, δεδομένου ότι τα γεγονότα προκύπτουν από τα πρότυπα μηνυμάτων που εντοπίζονται κατά τη διαδικασία του log parsing, σημαντικό ρόλο στη δημιουργία των event logs έχει το αναγνωριστικό γεγονότος (eventID), το οποίο αντιστοιχεί στο πρότυπο του μηνύματος. Για τον λόγο αυτό, η ποιότητα των αποτελεσμάτων της διαδικασίας log parsing επηρεάζει άμεσα και την ποιότητα των παραγόμενων event logs.

Τέλος, τα υπόλοιπα πεδία των καταγραφών αποθηκεύονται ως χαρακτηριστικά (attributes) των γεγονότων, διατηρώντας το όνομα και την τιμή τους.

2.2.3 Εξόρυξη Δεδομένων

Στο πλαίσιο της σύγχρονης τεχνολογικής εξέλιξης, ο όγκος των παραγόμενων δεδομένων αυξάνεται συνεχώς. Τα δεδομένα αυτά έχουν τη δυνατότητα να παρέχουν πολύ χρήσιμες πληροφορίες, εφόσον εφαρμοστούν κατάλληλες τεχνικές επεξεργασίας και ανάλυσης. Για την περιγραφή του φαινομένου αυτού, στη βιβλιογραφία χρησιμοποιείται ο όρος Μεγάλα Δεδομένα (Big Data), ο οποίος αναφέρεται σε σύνολα δεδομένων τόσο μεγάλου όγκου και πολυπλοκότητας ώστε η διαχείρισή τους να καθίσταται δύσκολη με παραδοσιακές μεθόδους.

Στη βιβλιογραφία αναφέρονται τέσσερα βασικά χαρακτηριστικά που διέπουν τα δεδομένα αυτά. Συγκεκριμένα, πρόκειται για τον όγκο (volume), την ταχύτητα δημιουργίας και επεξεργασίας τους (velocity), την ποικιλία των μορφών με τις οποίες εμφανίζονται (variety) και την αξιοπιστία ή εμπιστοσύνη των δεδομένων (veracity).

Η ύπαρξη αυτών των χαρακτηριστικών δημιουργεί σημαντικές προκλήσεις σε διάφορα στάδια της επεξεργασίας δεδομένων, όπως η αποθήκευση, η προεπεξεργασία, η οπτικοποίηση και η ανακάλυψη προτύπων, καθώς οι κλασικές τεχνικές ανάλυσης συχνά δεν επαρκούν για την αποτελεσματική αξιοποίησή τους [40, p. 768–772].

Η εξόρυξη δεδομένων (data mining) αφορά την αυτόματη απόκτηση γνώσης από μεγάλα σύνολα δεδομένων μέσω μη τετριμμένων μεθόδων ανάλυσης. Δηλαδή, δεν περιορίζεται στην απλή ανάκτηση πληροφοριών από τα δεδομένα, αλλά περιλαμβάνει την εφαρμογή εξειδικευμένων αλγορίθμων με σκοπό την ανακάλυψη προτύπων και σχέσεων που δεν είναι άμεσα εμφανείς.

Η εξόρυξη δεδομένων αποτελεί μέρος της διαδικασίας Ανακάλυψης Γνώσης σε Βάσεις Δεδομένων (Knowledge Discovery in Databases - KDD). Η διαδικασία αυτή περιλαμβάνει μια σειρά σταδίων, όπως τη συλλογή των δεδομένων, την προεπεξεργασία τους, την εφαρμογή τεχνικών εξόρυξης δεδομένων και τη μεταγενέστερη επεξεργασία των αποτελεσμάτων. Στόχος της εξόρυξης δεδομένων είναι είτε η πρόβλεψη είτε η περιγραφή της συμπεριφοράς των δεδομένων, μέσω τεχνικών όπως η κατηγοριοποίηση (classification), η παλινδρόμηση (regression), η ανάλυση συσχετίσεων, η ομαδοποίηση (cluster analysis) και ο εντοπισμός ανωμαλιών [32, p. 1–10].

Οι τεχνικές εξόρυξης δεδομένων αποτελούν σημαντικό εργαλείο για την αξιοποίηση μεγάλων συνόλων δεδομένων, επιτρέποντας την ανακάλυψη προτύπων και χρήσιμων γνώσεων. Στην επόμενη ενότητα παρουσιάζεται πιο συγκεκριμένα η εξόρυξη διαδικασιών, η οποία αποτελεί εξειδικευμένη εφαρμογή των τεχνικών αυτών σε δεδομένα καταγραφής γεγονότων.

Εξόρυξη Διαδικασιών (Process Mining)

Η εξόρυξη διαδικασιών αποτελεί εξειδικευμένη εφαρμογή τεχνικών εξόρυξης δεδομένων σε δεδομένα καταγραφής γεγονότων (event logs). Στο πλαίσιο της παρούσας εργασίας, τα event logs προκύπτουν από τη μετατροπή των αρχείων καταγραφής του συστήματος σε δομημένα γεγονότα, όπως περιγράφηκε στις προηγούμενες ενότητες. Μέσω των τεχνικών αυτών καθίσταται δυνατή η ανακάλυψη, η ανάλυση και η κατανόηση της πραγματικής συμπεριφοράς των διαδικασιών που εκτελούνται σε ένα πληροφοριακό σύστημα με βάση τα καταγεγραμμένα γεγονότα.

Σύμφωνα με το βιβλίο [34, p. 3–7], η εξόρυξη διαδικασιών σχετίζεται με την ανάλυση των διεργασιών ενός συστήματος. Στόχος της είναι η παροχή μιας συνολικής εικόνας της εκτέλεσης μιας διαδικασίας, επιτρέποντας τον εντοπισμό αιτιακών σχέσεων, πιθανών προβλημάτων και ευκαιριών βελτίωσης, συμβάλλοντας παράλληλα στην ενίσχυση της αυτοματοποίησης των διαδικασιών.

Η εξόρυξη διαδικασιών χωρίζεται σε διαφορετικούς τύπους [34, p. 23–28]. Οι βασικότεροι από αυτούς είναι:

- **Ανακάλυψη διαδικασιών (process discovery)**, όπου ένα μοντέλο διαδικασίας εξάγεται αυτόματα από τα event logs.
- **Έλεγχος συμμόρφωσης (conformance checking)**, όπου συγκρίνεται ένα υπάρχον μοντέλο διαδικασίας με τα πραγματικά δεδομένα εκτέλεσης για τον εντοπισμό αποκλίσεων.
- **Ανάλυση απόδοσης (performance analysis)**, η οποία επιτρέπει τον εντοπισμό προβλημάτων απόδοσης και σημείων συμφόρησης σε μια διαδικασία.
- **Συγκριτική εξόρυξη διαδικασιών**, όπου συγκρίνονται διαφορετικά event logs για τον εντοπισμό διαφορών και ομοιοτήτων μεταξύ διαφορετικών συνθηκών εκτέλεσης.
- **Προβλεπτική εξόρυξη διαδικασιών**, η οποία αξιοποιεί τα δεδομένα για την πρόβλεψη μελλοντικών συμπεριφορών μιας διαδικασίας.
- **Εξόρυξη Διαδικασιών με εστίαση στις δράσεις (Action-oriented process mining)**, όπου τα αποτελέσματα της ανάλυσης χρησιμοποιούνται για την πρόταση ενεργειών βελτίωσης της διαδικασίας.

Στο πλαίσιο της ανακάλυψης διαδικασίας, ο στόχος είναι η παραγωγή ενός μοντέλου που να είναι ταυτόχρονα κατανοητό και επαρκώς ακριβές ως προς τα παρατηρούμενα δεδομένα. Ωστόσο, στην πράξη τα event logs συχνά περιέχουν θόρυβο, ελλιπείς ή σπάνιες συμπεριφορές και γεγονότα διαφορετικού επιπέδου λεπτομέρειας (granularity), με αποτέλεσμα να απαιτείται προσεκτική επιλογή της μεθόδου εξόρυξης και κατάλληλη προεπεξεργασία. Αντίστοιχα, στον έλεγχο συμμόρφωσης, η σύγκριση μοντέλου και καταγραφών επιτρέπει τον εντοπισμό παραβιάσεων κανόνων, παρακάμψεων βημάτων ή μη αναμενόμενων ακολουθιών, κάτι ιδιαίτερα χρήσιμο για σκοπούς ελέγχου, κανονιστικής συμμόρφωσης και βελτιστο-ποίησης.

Για την καλύτερη κατανόηση των βασικών λειτουργιών της εξόρυξης διαδικασιών, στο Σχήμα 2.1 από το [34] παρουσιάζεται μια συνοπτική απεικόνιση της τυπικής ροής του process mining, από την απόκτηση των δεδομένων έως τη λήψη αποφάσεων και τη βελτίωση της διαδικασίας. Αρχικά, τα πληροφοριακά συστήματα παράγουν αρχεία καταγραφής από τα οποία εξάγονται δεδομένα γεγονότων. Τα δεδομένα αυτά εξερευνώνται και προεπεξεργάζονται μέσω επιλογής, φιλτραρίσματος και καθαρισμού. Στη συνέχεια τα δεδομένα χρησιμοποιούνται για την εύρεση ενός μοντέλου και για την αντιστοίχιση των εκτελέσεων πάνω σε αυτό, ώστε να προκύψουν έλεγχος συμμόρφωσης και ανάλυση απόδοσης, καθώς και διαγνωστικά που βοηθούν στην ερμηνεία αποκλίσεων και προβλημάτων. Με βάση αυτά, μπορούν να γίνουν συγκρίσεις μεταξύ διαφορετικών συνθηκών εκτέλεσης και να παραχθούν προβλέψεις και προτάσεις βελτίωσης. Τέλος, τα αποτελέσματα οδηγούν σε ενέργειες και μετασχηματισμούς της διαδικασίας, δημιουργώντας έναν κύκλο συνεχούς βελτίωσης.

Η αποτελεσματικότητα της εξόρυξης διαδικασιών εξαρτάται σε σημαντικό βαθμό από την ποιότητα των προηγούμενων σταδίων επεξεργασίας, όπως το log parsing και η μετατροπή των καταγραφών σε event logs. Καθώς δεν πρόκειται για ένα απομονωμένο βήμα,

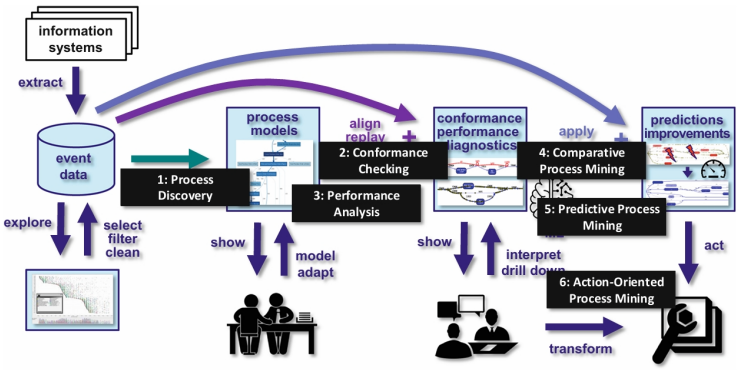


Figure 2.1: Τυπική ροή της εξόρυξης διαδικασιών (Process Mining)

αλλά επηρεάζεται άμεσα από τον τρόπο παραγωγής των event logs, η σωστή αναγνώριση των γεγονότων και η κατάλληλη οργάνωσή τους σε ίχνη είναι κρίσιμες για την παραγωγή αξιόπιστων αποτελεσμάτων. Ειδικότερα, παράγοντες όπως η ορθή ομαδοποίηση σε cases, οι αξιόπιστες χρονικές σημάνσεις και οι συνεπείς ονομασίες δραστηριοτήτων καθορίζουν σε μεγάλο βαθμό την ποιότητα των μοντέλων που θα εξαχθούν και, κατ' επέκταση, την αξιοπιστία των συμπερασμάτων που προκύπτουν από τις τεχνικές process mining.

Διαδικαστικά και Δηλωτικά Μοντέλα

Στο πλαίσιο της εξόρυξης διαδικασιών, τα μοντέλα που χρησιμοποιούνται για την αναπαράσταση της συμπεριφοράς μιας διαδικασίας μπορούν να διακριθούν σε δύο βασικές κατηγορίες: τα διαδικαστικά (procedural) και τα δηλωτικά (declarative) μοντέλα [33, p. 266–267].

Τα διαδικαστικά μοντέλα περιγράφουν ρητά τη ροή εκτέλεσης μιας διαδικασίας, καθορίζοντας συγκεκριμένες ακολουθίες δραστηριοτήτων και σημεία απόφασης. Παραδείγματα τέτοιων μοντέλων αποτελούν τα Petri nets, τα BPMN μοντέλα, τα process trees, τα Directly-Follows Graphs (DFG) graphs, τα οποία χρησιμοποιούνται ευρέως για την περιγραφή καλά δομημένων και σχετικά προβλέψιμων διαδικασιών [34, p. 31].

Αντίθετα, τα δηλωτικά μοντέλα δεν καθορίζουν μια συγκεκριμένη ροή εκτέλεσης, αλλά περιγράφουν ένα σύνολο περιορισμών (constraints) που πρέπει να ικανοποιούνται κατά την εκτέλεση της διαδικασίας. Με τον τρόπο αυτό επιτρέπεται μεγαλύτερη ευελιξία στη συμπεριφορά των διεργασιών, γεγονός που καθιστά τα δηλωτικά μοντέλα ιδιαίτερα κατάλληλα για δυναμικά και λιγότερο προβλέψιμα περιβάλλοντα. Στο πεδίο της εξόρυξης διαδικασιών, η διάκριση αυτή είναι ιδιαίτερα σημαντική, καθώς επηρεάζει τόσο τις τεχνικές ανακάλυψης μοντέλων όσο και την ερμηνεία των αποτελεσμάτων που προκύπτουν από τα event logs.

Αντίθετα, τα δηλωτικά μοντέλα δεν καθορίζουν μία συγκεκριμένη ροή εκτέλεσης, αλλά περιγράφουν ένα σύνολο περιορισμών που ορίζουν τι επιτρέπεται και τι απαγορεύεται κατά την εκτέλεση της διαδικασίας. Μία από τις πιο γνωστές δηλωτικές γλώσσες μοντελοποίησης είναι η Declare, η οποία εκφράζει τη συμπεριφορά μιας διαδικασίας ως σύνολο

λογικών περιορισμών μεταξύ δραστηριοτήτων και βασίζεται στη Γραμμική Χρονική Λογική (Linear Temporal Logic, LTL). Με αυτόν τον τρόπο, η Declare υιοθετεί τη φιλοσοφία ότι «όλα είναι δυνατά, εκτός αν απαγορεύονται ρητά», παρέχοντας μεγαλύτερη ευελιξία σε περιβάλλοντα με υψηλή μεταβλητότητα και πολλές εξαιρέσεις. Επιπλέον, οι περιορισμοί της μπορούν να αξιοποιηθούν άμεσα στον έλεγχο συμμόρφωσης, καθώς επιτρέπουν τον υπολογισμό, για κάθε κανόνα, του ποσοστού των περιπτώσεων που τον ικανοποιούν, ενώ μπορούν να ενσωματώσουν και χρονικές απαιτήσεις, όπως η υποχρέωση να ακολουθήσει μία δραστηριότητα μέσα σε συγκεκριμένο χρονικό διάστημα μετά από ένα γεγονός.

DECLARE και Εξόρυξη Περιορισμών

Σε πολλές πραγματικές επιχειρησιακές διαδικασίες η ακριβής σειρά εκτέλεσης των δραστηριοτήτων δεν είναι πάντα πλήρως καθορισμένη. Για τον λόγο αυτό έχουν αναπτυχθεί δηλωτικές προσεγγίσεις μοντελοποίησης διαδικασιών, στις οποίες η συμπεριφορά μιας διαδικασίας δεν περιγράφεται ως αυστηρή ακολουθία βημάτων αλλά μέσω ενός συνόλου περιορισμών που πρέπει να ικανοποιούνται κατά την εκτέλεση της διαδικασίας. Μία από τις πιο γνωστές γλώσσες δηλωτικής μοντελοποίησης είναι η DECLARE, η οποία χρησιμοποιεί μια προσέγγιση βασισμένη σε περιορισμούς (constraint-based modeling) για την περιγραφή διαδικασιών.

Σε αντίθεση με τα διαδικαστικά μοντέλα, όπου ορίζεται ρητά η ακριβής ροή εκτέλεσης μιας διαδικασίας, τα δηλωτικά μοντέλα καθορίζουν ένα σύνολο κανόνων που περιορίζουν τις επιτρεπτές συμπεριφορές του συστήματος. Με άλλα λόγια, η εκτέλεση μιας διαδικασίας θεωρείται έγκυρη εφόσον δεν παραβιάζει κανέναν από τους καθορισμένους περιορισμούς. Με αυτόν τον τρόπο το μοντέλο δεν καθορίζει όλες τις δυνατές ακολουθίες δραστηριοτήτων, αλλά επιτρέπει οποιαδήποτε συμπεριφορά που συμμορφώνεται με τους περιορισμούς που έχουν τεθεί [27].

Για να γίνει πιο σαφές πως εκφράζονται οι περιορισμοί στα δηλωτικά μοντέλα, είναι χρήσιμο να εξεταστούν ορισμένα χαρακτηριστικά παραδείγματα. Ένας συχνά χρησιμοποιούμενος περιορισμός είναι ο response(A,B), σύμφωνα με τον οποίο κάθε φορά που εκτελείται μια δραστηριότητα A, πρέπει σε κάποιο μεταγενέστερο σημείο να εκτελεστεί και μια δραστηριότητα B. Με τον τρόπο αυτό εκφράζεται μια εξάρτηση μεταξύ των δύο δραστηριοτήτων χωρίς να καθορίζεται η ακριβής θέση τους στη συνολική ροή της διαδικασίας.

Ένα ακόμη παράδειγμα αποτελεί ο περιορισμός precedence(A,B), σύμφωνα με τον οποίο μια δραστηριότητα B μπορεί να εκτελεστεί μόνο εφόσον έχει προηγηθεί τουλάχιστον μία φορά η δραστηριότητα A. Παράλληλα, υπάρχουν και περιορισμοί που αφορούν τη συχνότητα εμφάνισης μιας δραστηριότητας, όπως ο περιορισμός existence(A,n), ο οποίος ορίζει ότι μια δραστηριότητα A πρέπει να εμφανιστεί τουλάχιστον n φορές σε ένα ίχνος εκτέλεσης. Μέσω τέτοιων περιορισμών καθίσταται δυνατή η περιγραφή της επιτρεπτής συμπεριφοράς μιας διαδικασίας με ευέλικτο τρόπο, επιτρέποντας πολλαπλές εναλλακτικές ακολουθίες εκτέλεσης που συμμορφώνονται με τους καθορισμένους κανόνες.

Η εξόρυξη περιορισμών (constraint mining) αποτελεί εφαρμογή της εξόρυξης διαδικασιών σε δηλωτικά μοντέλα. Στην περίπτωση αυτή, οι περιορισμοί δεν ορίζονται χειροκίνητα

αλλά ανακαλύπτονται αυτόματα από τα event logs που καταγράφουν την εκτέλεση των διαδικασιών. Μέσω της ανάλυσης των καταγεγραμμένων γεγονότων, μπορούν να εντοπιστούν επαναλαμβανόμενα μοτίβα συμπεριφοράς και να εξαχθούν οι περιορισμοί που περιγράφουν τις εξαρτήσεις μεταξύ των δραστηριοτήτων.

Η προσέγγιση αυτή είναι ιδιαίτερα χρήσιμη σε περιβάλλοντα όπου οι διαδικασίες είναι ευέλικτες ή λιγότερο δομημένες, καθώς επιτρέπει την κατανόηση των πραγματικών κανόνων που διέπουν την εκτέλεση των δραστηριοτήτων χωρίς να απαιτείται πλήρης προκαθορισμός της ροής της διαδικασίας. Επιπλέον, οι περιορισμοί που ανακαλύπτονται μπορούν να χρησιμοποιηθούν για την επαλήθευση συμμόρφωσης, την ανάλυση αποκλίσεων και τη βελτίωση των επιχειρησιακών διαδικασιών.

3 Σχετικές Προσεγγίσεις - Εργαλεία

Στην παρούσα ενότητα παρουσιάζονται συστήματα τα οποία εκτελούν επιμέρους στάδια της συνολικής διαδικασίας, καθώς και συστήματα που προσεγγίζουν την επεξεργασία αρχείων καταγραφής με πιο ολιστικό τρόπο. Ωστόσο, τα τελευταία διαφοροποιούνται ως προς τη μεθοδολογία και τα βήματα επεξεργασίας σε σχέση με την προσέγγιση που ακολουθείται στην παρούσα διπλωματική εργασία.

3.1 Εργαλεία Log Parsing

Όπως αναφέρθηκε και παραπάνω, τα εργαλεία που μετασχηματίζουν μη δομημένα αρχεία καταγραφής σε δομημένη μορφή βασίζονται σε τεχνικές log parsing και κατηγοριοποιούνται σε διαφορετικές προσεγγίσεις. Παράλληλα, υπάρχουν και υλοποιήσεις που δεν εντάσσονται πλήρως σε κάποια γενική κατηγορία, αλλά ακολουθούν πιο εξειδικευμένες μεθόδους.

Στη συνέχεια παρουσιάζονται αντιπροσωπευτικά παραδείγματα τέτοιων συστημάτων, με έμφαση στις βασικές διαφορές ως προς τη λειτουργία και την προσέγγισή τους. Για ορισμένα από αυτά παρατίθενται ενδεικτικά παραδείγματα χρήσης στο Παράρτημα Α.

3.1.1 Logstash

Το συγκεκριμένο σύστημα ανήκει σε μία κατηγορία όπου εντάσσονται συστήματα που βασίζονται σε μοτίβα και κανονικές εκφράσεις για την ανάλυση και τον μετασχηματισμό μη δομημένων αρχείων καταγραφής. Παρακάτω παρουσιάζεται η βασική λειτουργία του Logstash [9], το οποίο αποτελεί ένα από τα πιο διαδεδομένα εργαλεία της κατηγορίας αυτής.

Το Logstash αποτελεί ένα βοηθητικό εργαλείο στην ροή επεξεργασίας δεδομένων (data-processing pipeline), το οποίο βασίζεται σε μια λογική τριών σταδίων. Τα στάδια αυτά είναι τα Input, Filter και Output.

- Στο στάδιο Input συλλέγει ακατέργαστα δεδομένα από διάφορες πηγές, όπως αρχεία ή υπηρεσίες cloud.
- Στο στάδιο Filter χρησιμοποιεί εξειδικευμένα φίλτρα για την ανάλυση μη δομημένων αρχείων καταγραφής, κυρίως μέσω κανονικών εκφράσεων, μετατρέποντάς τα σε δομημένα και ονομασμένα πεδία.
- Τέλος, στο στάδιο Output αποστέλλει τα μετασχηματισμένα δεδομένα σε έναν προορισμό, όπως το Elasticsearch, μια βάση δεδομένων, ένα αρχείο ή σε κάποιον αποθηκευτικό χώρο στο cloud.

Η αρχιτεκτονική αυτή καθιστά το Logstash ιδιαίτερα ισχυρό εργαλείο για τη μετατροπή ακατέργαστων αρχείων καταγραφής σε δομημένα δεδομένα έτοιμα για ανάλυση. Η αποτελεσματικότητά του ενισχύεται όταν τα μηνύματα καταγραφής ακολουθούν κάποια επαναλαμβανόμενη δομή, η οποία μπορεί να αξιοποιηθεί μέσω του φίλτρου grok. Επιπλέον, κατά τη φάση φιλτραρίσματος, υποστηρίζονται λειτουργίες μετασχηματισμού όπως

η προσθήκη, αφαίρεση ή μετονομασία πεδίων και μετασχηματισμός της μορφής των δεδομένων, επιτρέποντας την ευέλικτη διαχείριση των ροών logs.

Τα επόμενα κεφάλαια παρουσιάζουν αυτοματοποιημένους parsers οι οποίοι εστιάζουν στον εντοπισμό templates από μηνύματα καταγραφής. Τα πρότυπα αυτά αντιπροσωπεύουν τη σταθερή δομή των μηνυμάτων, διαχωρίζοντας τα μεταβλητά τμήματα από τα σταθερά.

Στο πλαίσιο της παρούσας εργασίας, τα εξαγόμενα πρότυπα χρησιμοποιούνται στη συνέχεια ως κατηγορίες γεγονότων, οι οποίες είναι απαραίτητες τα επόμενα στάδια.

3.1.2 Drain

Τα συστήματα επεξεργασίας βασισμένα στη σύνταξη στοχεύουν στην αυτόματη αναγνώριση της δομής των μηνυμάτων καταγραφής, αξιοποιώντας συντακτικούς κανόνες εντοπισμό προτύπων. Παρακάτω, παρουσιάζεται ο αλγόριθμος Drain [11], ένας online parser με αποδοτική εξαγωγή προτύπων από ροές δεδομένων καταγραφής.

Το Drain είναι ένας αυτοματοποιημένος log parser που διαχωρίζει τα σταθερά μέρη ενός μηνύματος καταγραφής από τα μεταβλητά, με στόχο τον εντοπισμό του event template. Η λειτουργία του βασίζεται σε ένα δέντρο ανάλυσης προκαθορισμένου βάθους (fixed-depth parse tree), όπου οι εσωτερικοί κόμβοι υλοποιούν κανόνες, ενώ οι κόμβοι φύλλων αντιστοιχούν τα νέα μηνύματα σε ομάδες (clusters) παρόμοιων καταγραφών.

Πριν από την εκτέλεσή του, το Drain απαιτεί τον καθορισμό ορισμένων παραμέτρων, όπως το βάθος του δέντρου, τον μέγιστο αριθμό παιδιών ανά κόμβο και κανόνες προεπεξεργασίας όπως κανονικές εκφράσεις για μεταβλητά στοιχεία όπως διευθύνσεις IP ή αναγνωριστικά. Η διαδικασία ανάλυσης ενός νέου μηνύματος log ακολουθεί τα παρακάτω βήματα:

- Ο κόμβος στη ρίζα κατευθύνει το μήνυμα σε έναν κλάδο με βάση τον αριθμό των tokens που περιέχει.
- Στα επόμενα επίπεδα, η δρομολόγηση βασίζεται στα αρχικά tokens του μηνύματος, θεωρώντας συνήθως ως σταθερά εκείνα που δεν περιέχουν αριθμητικά ψηφία.
- Το μήνυμα καταλήγει σε ένα φύλλο, το οποίο περιέχει τα υποψήφια clusters. Εκεί υπολογίζεται ένας δείκτης ομοιότητας μεταξύ του μηνύματος και των υπαρχόντων templates. Αν η ομοιότητα υπερβαίνει ένα προκαθορισμένο κατώφλι, το μήνυμα αντιστοιχίζεται στο πιο κατάλληλο cluster και το template ενημερώνεται, διαφορετικά, δημιουργείται νέο cluster.

Η ιεραρχική αυτή προσέγγιση επιτρέπει στο Drain να επιτυγχάνει υψηλή αποδοτικότητα και καλή ακρίβεια, καθιστώντας το κατάλληλο για περιβάλλοντα συνεχούς ροής δεδομένων (streaming). Στην πράξη, η ανάλυση υλοποιείται μέσω μιας συνάρτησης parse(), η οποία ενσωματώνει όλα τα επιμέρους βήματα, από την προεπεξεργασία έως την αντιστοίχιση σε clusters.

3.1.3 LogPPT

Τα σημασιολογικά συστήματα επεξεργασίας αποτελούν μια πιο πρόσφατη κατηγορία log parsers, τα οποία αξιοποιούν τεχνικές μηχανικής μάθησης και προεκπαιδευμένα γλωσσικά μοντέλα για την κατανόηση της σημασίας των μηνυμάτων καταγραφής. Σε αντίθεση με τις προηγούμενες προσεγγίσεις, τα συστήματα αυτά αξιοποιούν τη σημασιολογική πληροφορία των λέξεων για τον εντοπισμό προτύπων. Ένα τέτοιο σύστημα είναι το logPPT [19] που αναλύεται παρακάτω.

Το LogPPT είναι ένας parser που αξιοποιεί ένα προεκπαιδευμένο γλωσσικό μοντέλο (pre-trained language model) για να διακρίνει τα σταθερά από τα μεταβλητά τμήματα ενός μηνύματος log, βασισμένο στην σημασία των μηνυμάτων αντί για τους συντακτικούς κανόνες. Η λειτουργία του βασίζεται σε μια προσέγγιση «prompt-based fine-tuning», όπου το μοντέλο προσαρμόζεται στο έργο της ανάλυσης logs χρησιμοποιώντας ένα μικρό σύνολο επισημασμένων παραδειγμάτων.

Η διαδικασία ανάλυσης ενός νέου μηνύματος log πραγματοποιείται ως εξής:

- Ένα προεκπαιδευμένο μοντέλο, το οποίο έχει προσαρμοστεί μέσω των prompt και στο οποίο έχει γίνει fine-tuning, αναλύει το νέο μήνυμα log.
- Το μοντέλο αξιολογεί κάθε token, αξιοποιώντας τη σημασιολογική του γνώση ώστε να διακρίνει λέξεις που είναι πιθανό να αποτελούν σταθερά στοιχεία, όπως για παράδειγμα η λέξη error, connected κλπ., από στοιχεία που μεταβάλλονται μεταξύ διαφορετικών μηνυμάτων όπως αναγνωριστικές τιμές, δεκαεξαδικά δεδομένα κ.ά.
- Στη συνέχεια παράγεται το αντίστοιχο template, στο οποίο τα μεταβλητά tokens αντικαθίστανται με σύμβολα, επιτρέποντας την ομαδοποίηση μηνυμάτων που αντιστοιχούν στον ίδιο τύπο γεγονότος.

Η προσέγγιση αυτή επιτρέπει στο LogPPT να γενικεύει αποτελεσματικά σε διαφορετικές πηγές logs, γεγονός που το καθιστά ιδιαίτερα κατάλληλο για δυναμικά και ετερογενή περιβάλλοντα καταγραφής. Στην πράξη, η εκτέλεση του parser πραγματοποιείται μέσω του φακέλου github του εργαλείου [22], όπου παρέχονται προεκπαιδευμένα μοντέλα βασισμένα σε transformers καθώς και τα απαραίτητα scripts για την επεξεργασία των δεδομένων.

3.1.4 DivLog

Αυτή η ενότητα εξετάζει την επεξεργασία logs με χρήση Μεγάλων Γλωσσικών Μοντέλων (LLMs). Οι προσεγγίσεις αυτές αξιοποιούν την κατανόηση φυσικής γλώσσας και τις δυνατότητες εκμάθησης με συμφραζόμενα (in-context learning) των LLMs για την ερμηνεία των log μηνυμάτων. Σε αντίθεση με τα προηγούμενα εργαλεία που βασίζονται σε σταθερούς κανόνες ή σε εξειδικευμένη εκπαίδευση μοντέλου, οι parsers που βασίζονται σε LLM αντιμετωπίζουν την επεξεργασία των logs ως μια διαδικασία καθοδηγούμενη από οδηγίες και παραδείγματα σε φυσική γλώσσα. Αυτό τους επιτρέπει να επιτυγχάνουν υψηλή γενίκευση, να διαχειρίζονται σύνθετες σημασιολογικές ασάφειες και να προσαρμόζονται

σε διαφορετικές μορφές log με ελάχιστα παραδείγματα. Ένα τέτοιο εργαλείο είναι το DivLog [36] που παρουσιάζεται παρακάτω. Αξίζει να σημειωθεί ότι σε αυτήν την κατηγορία ανήκει και το εργαλείο LILAC [37] που αναπτύσσεται στην προτεινόμενη μεθοδολογία, λεπτομέρειες για το οποίο δίνονται στην αντίστοιχη ενότητα.

Το DivLog προσεγγίζει το log parsing ως πρόβλημα κατανόησης της σημασιολογίας των log μηνυμάτων. Σε αντίθεση με προηγούμενες προσεγγίσεις που εστιάζουν είτε σε συντακτικά χαρακτηριστικά είτε σε token-level ταξινόμηση, το DivLog αξιοποιεί προεκπαιδευμένα γλωσσικά μοντέλα για την εξαγωγή log templates με βάση τον τρόπο με τον οποίο ερμηνεύει τα δοσμένα παραδείγματα.

Η βασική ιδέα του DivLog είναι η εκμάθηση διακριτών αναπαραστάσεων (representations) για τα log μηνύματα, ώστε να είναι δυνατός ο διαχωρισμός διαφορετικών τύπων γεγονότων ακόμη και σε περιβάλλοντα με υψηλή ετερογένεια και θόρυβο.

Η διαδικασία λειτουργίας του DivLog είναι η εξής:

- Αρχικά, συλλέγονται log δεδομένα τα οποία παρουσιάζουν μεγάλη ποικιλία στη μορφή τους.
- Έπειτα, επιλέγεται ένα μικρό υποσύνολο αυτών, το οποίο χρησιμοποιείται ως παραδείγματα για τη δημιουργία του prompt.
- Το μοντέλο, με βάση το prompt και τα δοσμένα παραδείγματα, εντοπίζει τα αντίστοιχα templates για τα νέα log μηνύματα.

Η προσέγγιση αυτή επιτρέπει στο DivLog να επιτυγχάνει υψηλή ακρίβεια σε σύνθετα και δυναμικά περιβάλλοντα, όπου οι παραδοσιακές μέθοδοι parsing δυσκολεύονται να γενικεύσουν. Επιπλέον, η αξιοποίηση LLMs καθιστά δυνατή την κατανόηση βαθύτερων σημασιολογικών σχέσεων μεταξύ των μηνυμάτων.

3.1.5 ByteBrain

Άλλοι parsers δεν εντάσσονται σε κάποια από τις βασικές προσεγγίσεις που παρουσιάστηκαν προηγουμένως, καθώς ακολουθούν διαφορετικές και συχνά πιο εξειδικευμένες μεθοδολογίες για την επεξεργασία των log δεδομένων. Ενδεικτικά, ένας τέτοιος parser είναι το ByteBrain [20], ο οποίος παρουσιάζεται παρακάτω.

Το ByteBrain αποτελεί μια προσέγγιση log parsing που εστιάζει στην αποδοτική αναγνώριση προτύπων μέσα από την ανάλυση της δομής και της συχνότητας εμφάνισης των tokens στα log μηνύματα. Αντί να βασίζεται αποκλειστικά σε προκαθορισμένους κανόνες ή σε προεκπαιδευμένα μοντέλα, αξιοποιεί συντακτικά χαρακτηριστικά όπως ο αριθμός των token και στη συνέχεια, μηχανισμούς συσταδοποίησης για την εξαγωγή των templates.

Η διαδικασία λειτουργίας του ByteBrain συνοψίζεται ως εξής:

- Αρχικά, τα μη δομημένα log μηνύματα μετατρέπονται σε αριθμητικές αναπαραστάσεις μέσω διαδικασιών όπως η κατάτμηση σε tokens (tokenization), η αντικατάσταση κοινών μεταβλητών στοιχείων και η απομάκρυνση διπλότυπων εγγραφών, ώστε να καταστούν κατάλληλα για αλγοριθμική επεξεργασία.

- Στη συνέχεια, τα μηνύματα οργανώνονται σε αρχικές ομάδες με βάση απλά χαρακτηριστικά, όπως το μήκος τους ή κοινά προθέματα, επιτρέποντας την αποδοτικότερη και παράλληλη επεξεργασία τους.

- Για κάθε ομάδα εφαρμόζεται ιεραρχική ομαδοποίηση (hierarchical clustering), κατά την οποία δημιουργείται μια δενδρική δομή όπου κάθε επίπεδο αντιστοιχεί σε πιο εξειδικευμένα πρότυπα μηνυμάτων.

- Κατά τη διάρκεια της διαδικασίας, χρησιμοποιείται ένας δείκτης αξιολόγησης για να εκτιμηθεί η ποιότητα των ομάδων και να αποφασιστεί πότε πρέπει να σταματήσει η περαιτέρω εξειδίκευση.

- Τέλος, τα παραγόμενα clusters αντιστοιχίζονται σε templates, τα οποία μπορούν να χρησιμοποιηθούν και σε επόμενα στάδια μέσω διαδικασιών online αντιστοίχισης νέων log μηνυμάτων.

Με τον τρόπο αυτό, το ByteBrain μπορεί να αναγνωρίζει αποτελεσματικά επαναλαμβανόμενες δομές μέσα σε μεγάλα και ετερογενή σύνολα δεδομένων, χωρίς να εξαρτάται από αυστηρούς κανόνες ή εκτενή εκπαίδευση μοντέλων.

3.2 Εργαλεία Process Mining

Τα εργαλεία για mining αποτελούν το τελικό στάδιο επεξεργασίας των log δεδομένων, όπου παράγονται τα τελικά αποτελέσματα και εξάγονται χρήσιμα συμπεράσματα για τη λειτουργία ενός συστήματος.

Στα επόμενα υποκεφάλαια παρουσιάζονται ενδεικτικά εργαλεία που εκτελούν mining με έμφαση στο declare mining, το οποίο αποτελεί και στάδιο της παρούσας εργασίας, δηλαδή, ανάλυει event logs με βάση δηλωτικά μοντέλα, όπου μία διαδικασία περιγράφεται μέσω περιορισμών αντί για αυστηρές ροές εκτέλεσης.

3.2.1 Declare4py

Το Declare4py [7] είναι μια βιβλιοθήκη python που υλοποιεί βασικές λειτουργίες declarative process mining, βασισμένες στη γλώσσα Declare και την επέκτασή της MP-Declare. Η συγκεκριμένη επέκταση επιτρέπει τον ορισμό περιορισμών που λαμβάνουν υπόψη όχι μόνο τη σειρά των ενεργειών, αλλά και δεδομένα σχετικά με τα event και χρονικές σχέσεις στους περιορισμους.

Η υλοποίηση του σε python το καθιστά ιδιαίτερα ευέλικτο, καθώς μπορεί να ενσωματωθεί εύκολα σε μεγαλύτερα συστήματα και να συνδυαστεί με δημοφιλή frameworks μηχανικής μάθησης. Επιπλέον, διατίθεται μέσω github στο [8] και δέχεται αρχεία σε μορφή `.xes` και `.decl`.

Το Declare4py έχει δυνατότητα να κάνει τα παρακάτω:

- Έλεγχο συμμόρφωσης, κατά τον οποίο εξετάζει αν τα traces ενός log ικανοποιούν τους περιορισμούς ενός δοσμένου μοντέλου, ενώ για κάθε περιορισμό υπολογίζονται μετρικές όπως ενεργοποιήσεις, ικανοποιήσεις και παραβιάσεις.

- Ανακάλυψη διαδικασίας, όπου εξάγεται ένα σύνολο περιορισμών που ισχύουν για ένα σημαντικό ποσοστό των traces. Η διαδικασία βασίζεται στον εντοπισμό συχνών δραστηριοτήτων και και στη δημιουργία υποψήφιων constraints, από τα οποία διατηρούνται μόνο όσα ξεπερνούν ένα κατώφλι υποστήριξης (support threshold).

- Έλεγχο ερωτημάτων (query checking), όπου δίνονται μερικώς καθορισμένοι περιορισμοί, δηλαδή οι δραστηριότητες του constraint μπορεί να μην είναι συγκεκριμένες, και το εργαλείο εντοπίζει ποιοι συνδυασμοί δραστηριοτήτων τους ικανοποιούν και πάλι για κάποιο support threshold.

Η διαδικασία αυτή επιτρέπει στο Declare4py να εξάγει χρήσιμη πληροφορία από τα event logs με ευέλικτο τρόπο, προσφέροντας μια ολοκληρωμένη εικόνα της συμπεριφοράς των διαδικασιών και διευκολύνοντας την περαιτέρω ανάλυση τους.

3.2.2 RuM

Το RuM [1] είναι μια εφαρμογή που εστιάζει στην εξόρυξη κανόνων με βάση τη γλώσσα Declare και την επέκτασή της MP-Declare. Επιπλέον, υποστηρίζει τη διατύπωση μοντέλων και σε φυσική γλώσσα, επιτρέποντας ακόμη και σε μη εξειδικευμένους χρήστες να αλληλεπιδρούν με το σύστημα πιο εύκολα.

Η εφαρμογή υποστηρίζει αρχεία τύπου `.xes` και `.decl`, παρέχοντας λειτουργικότητες αντίστοιχες με άλλες πλατφόρμες declarative process mining.

Οι βασικές λειτουργικότητες του RuM συνοψίζονται ως εξής:

- Ανακάλυψη διαδικασίας, όπου αξιοποιούνται τέσσερις διαφορετικές μέθοδοι: DeclareMiner, MINERful, MP-Declare Miner και MP-MINERful. Στην παρούσα εργασία χρησιμοποιείται η μέθοδος MINERful, η οποία αναλύεται περισσότερο παρακάτω. Οι μέθοδοι που βασίζονται στο MP-Declare επιτρέπουν την ανακάλυψη πιο σύνθετων σχέσεων, ενσωματώνοντας πληροφορία δεδομένων και χρόνου. Τα αποτελέσματα μπορούν να παρουσιαστούν με διάφορους τρόπους, όπως μέσω της γραφικής αναπαράστασης των περιορισμών, σε μορφή κειμένου ή ως αυτόματο πεπερασμένων καταστάσεων (finite-state machine).

- Έλεγχος συμμόρφωσης, όπου το σύστημα συγκρίνει ένα event log με ένα δοσμένο Declare μοντέλο. Για κάθε περιορισμό υπολογίζονται οι ενεργοποιήσεις, οι ικανοποιήσεις και οι παραβιάσεις, παρέχοντας λεπτομερή εικόνα της συμπεριφοράς κάθε trace ως προς το μοντέλο.

- Επεξεργαστής MP-Declare (MP-Declare Editor), ο οποίος επιτρέπει μόνο αρχεία τύπου `.decl`, συμπεριλαμβάνοντας ορισμούς δραστηριοτήτων, χαρακτηριστικών και περιορισμούς με συνθήκες που αφορούν το χρόνο και τα δεδομένα και έχει ως

στόχο την επεξεργασία τους. Οι τρόποι οπτικοποίησης παραμένουν αντίστοιχοι όπως στην ανακάλυψη διαδικασιών και οι χρήστες μπορούν να ορίζουν περιορισμούς μέσω φυσικής γλώσσας.

- Παραγωγή log, η οποία υλοποιείται μέσω των μεθόδων AlloyLogGenerator και MINERful Log Generator. Οι μέθοδοι αυτές επιτρέπουν τη δημιουργία συνθετικών event logs με βάση ένα δοσμένο μοντέλο, δίνοντας τη δυνατότητα ρύθμισης παραμέτρων όπως το ποσοστό traces που ικανοποιούν τετριμμένα τους περιορισμούς ή το ποσοστό traces που περιέχουν παραβιάσεις. Τα παραγόμενα logs μπορούν να εξαχθούν σε μορφή `.xes`.

Οι παραπάνω δυνατότητες καθιστούν το RuM ένα ολοκληρωμένο εργαλείο για declarative process mining, συνδυάζοντας πολλαπλές τεχνικές ανάλυσης με φιλικές προς τον χρήστη διεπαφές. Συνολικά, το RuM απευθύνεται τόσο σε ερευνητές όσο και σε ειδικούς με περιορισμένη εμπειρία στο Declare, προσφέροντας διαφορετικά επίπεδα αλληλεπίδρασης και ευελιξία στη χρήση. Παράλληλα, διατηρεί μια συνεπή φιλοσοφία ως προς τις παραμέτρους και τη λειτουργία των επιμέρους μεθόδων για τις ίδιες λειτουργικότητες, διευκολύνοντας τη σύγκριση και την αξιοποίηση των αποτελεσμάτων.

3.2.3 ASP-Based Declarative Process Mining

Η παρακάτω προσέγγιση βασίζεται στο Answer Set Programming (ASP) και αντιμετωπίζει το declarative process mining ως πρόβλημα λογικής μοντελοποίησης και επίλυσης περιορισμών [3]. Αντί να χρησιμοποιούνται εξειδικευμένοι αλγόριθμοι για κάθε επιμέρους εργασία, το πρόβλημα περιγράφεται μέσω κανόνων και σχέσεων, και η λύση προκύπτει αυτόματα από έναν solver που αναζητά ένα σύνολο απαντήσεων (answer sets) που να ικανοποιεί όλους τους περιορισμούς. Οι κανόνες αυτοί εκφράζουν σχέσεις μεταξύ γεγονότων, ενεργειών και δεδομένων.

Το ASP αποτελεί μια μορφή δηλωτικού προγραμματισμού (declarative programming), όπου ο χρήστης ορίζει τους κανόνες που ισχύουν και όχι το πώς θα υπολογιστεί η λύση. Έτσι, η λογική των Declare μοντέλων μπορεί να αποτυπωθεί εύκολα μέσω του ASP.

Ένα σημαντικό χαρακτηριστικό του είναι η δυνατότητα χειρισμού δεδομένων (data-aware), επιτρέποντας την ενσωμάτωση όχι μόνο της σειράς των ενεργειών αλλά και συνθηκών που αφορούν τιμές δεδομένων και χρονικές σχέσεις. Αυτό το καθιστά ιδιαίτερα κατάλληλο για declarative process mining, όπου οι περιορισμοί συχνά εξαρτώνται από πολλαπλές διαστάσεις.

Η βασική ιδέα είναι ότι τα προβλήματα του process mining μετατρέπονται σε σύνολα λογικών κανόνων, ενώ οι λύσεις αντιστοιχούν σε ερμηνείες που ικανοποιούν αυτούς τους κανόνες.

Οι κύριες λειτουργικότητες της προσέγγισης είναι:

- Παραγωγή event logs, όπου δημιουργούνται συνθετικά traces που ικανοποιούν ένα σύνολο περιορισμών. Στο πλαίσιο του ASP, η διαδικασία αυτή διατυπώνεται ως πρόβλημα εύρεσης λύσεων, δηλαδή κάθε παραγόμενο trace αντιστοιχεί σε ένα έγκυρο

σύνολο γεγονότων που ικανοποιεί όλους τους κανόνες, χωρίς να απαιτείται ρητή προσομοίωση της διαδικασίας.

- Έλεγχος συμμόρφωσης, κατά τον οποίο εξετάζεται αν ένα trace ή ένα σύνολο traces ευθυγραμμίζεται με τους δοσμένους περιορισμούς. Μέσω του ASP, ο έλεγχος αυτός μετατρέπεται σε πρόβλημα όπου αναζητείται αν υπάρχει κάποιο ίχνος που να καλύπτει τους κανόνες του μοντέλου.

- Έλεγχος ερωτημάτων, όπου δίνονται μερικώς καθορισμένοι περιορισμοί και το σύστημα αναζητά τις αναθέσεις που τους καθιστούν έγκυρους. Η δηλωτική φύση του ASP επιτρέπει την άμεση μοντελοποίηση αυτής της αναζήτησης ως πρόβλημα επίλυσης περιορισμών, ενώ υποστηρίζεται και η ενσωμάτωση συνθηκών πάνω σε δεδομένα (data-aware), ενισχύοντας τη δυνατότητα έκφρασης πιο σύνθετων ερωτημάτων.

Οι παραπάνω λειτουργίες βασίζονται σε ένα κοινό πλαίσιο λογικών κανόνων, γεγονός που επιτρέπει την επαναχρησιμοποίηση της ίδιας μοντελοποίησης με μικρές προσαρμογές ανά περίπτωση.

Συνολικά, η χρήση του ASP προσφέρει μια ενιαία και εκφραστική προσέγγιση στο declarative process mining, όπου διαφορετικά προβλήματα αντιμετωπίζονται ως παραλλαγές του ίδιου λογικού σχήματος, με ιδιαίτερο πλεονέκτημα τη διαχείριση δεδομένων και σύνθετων περιορισμών.

3.2.4 SIESTA Declare Mining

Το SIESTA [23] αποτελεί ένα εργαλείο για declare mining που στοχεύει στην αποδοτική εξαγωγή δηλωτικών μοντέλων από event logs. Η βασική του ιδέα είναι η αξιοποίηση τεχνικών indexing, όπου για κάθε ζευγάρι δραστηριοτήτων που εμφανίζεται στα ίχνη κατασκευάζονται βοηθητικές δομές δεδομένων. Οι δομές αυτές αποθηκεύουν στατιστικά στοιχεία και πληροφορίες εμφάνισης των γεγονότων ή ζευγαριών γεγονότων (event-type pair), επιτρέποντας την γρήγορη εκτέλεση των επόμενων βημάτων ανάλυσης.

Οι λειτουργίες που μπορεί να επιτελέσει ο declare miner του SIESTA είναι:

- Αναγνώριση μοτίβων, όπως για παράδειγμα, το σύστημα μπορεί να εντοπίζει τα traces που περιέχουν μία ακολουθία δραστηριοτήτων. Μέσω αυτής της διαδικασίας είναι δυνατή η εξαγωγή προτύπων που αντιστοιχούν σε Declare constraints και κατ'επέκταση ενός declare μοντέλου.

- Στατιστικά δεδομένα παρέχονται για κάθε ζεύγος δραστηριοτήτων σε ένα trace, όπως συχνότητα εμφάνισης ή συν-εμφάνισης. Τα στατιστικά αυτά μπορούν να αξιοποιηθούν για την αξιολόγηση της σημαντικότητας ενός υποψήφιου περιορισμού.

- Συνέχιση μοτίβου όπου για κάποιο μοτίβο εντοπίζονται τα event που είναι πιθανότερο να ακολουθήσουν.

Η αξιοποίηση των indices που περιγράφηκε επιτρέπει στο SIESTA να εκτελεί τις παραπάνω λειτουργίες με υψηλή αποδοτικότητα και δυνατότητα παραλληλοποίησης. Το πλεονέκτημα αυτό είναι ιδιαίτερα σημαντικό ειδικά όταν η εξαγωγή ενός μοντέλου γίνεται για διάφορες τιμές παραμέτρων και είναι αναγκαίες οι πολλές επαναλήψεις. Επομένως, το εργαλείο είναι κατάλληλο για μεγάλα και δυναμικά σύνολα δεδομένων.

3.3 Εργαλεία Πολλαπλών Διαδικασιών

Σε αυτό το κεφάλαιο παρουσιάζονται εργαλεία τα οποία υποστηρίζουν πολλαπλά στάδια επεξεργασίας log δεδομένων, προσφέροντας ένα ενοποιημένο περιβάλλον για συλλογή, ανάλυση, αποθήκευση και οπτικοποίηση.

3.3.1 Graylog

Σύμφωνα με το [31], το Graylog αποτελεί ένα εργαλείο διαχείρισης και ανάλυσης log δεδομένων, με κύριο στόχο την παρακολούθηση συστημάτων και την έγκαιρη ανίχνευση προβλημάτων σε πραγματικό χρόνο. Παρέχει δυνατότητες αναζήτησης, ειδοποιήσεων και οπτικοποίησης, καλύπτοντας πολλαπλές ανάγκες σε ένα ενιαίο σύστημα.

Η συνολική ροή λειτουργίας του Graylog περιγράφεται παρακάτω: [10]

- Αρχικά, συλλέγονται log δεδομένα από πολλαπλές πηγές μέσω των inputs, ενώ εξασφαλίζεται η ασφαλής μεταφορά τους μέσω κρυπτογράφησης.
- Στη συνέχεια, τα δεδομένα οργανώνονται σε streams με βάση προκαθορισμένους κανόνες, οι οποίοι είναι κανόνες φιλτραρίσματος και οργάνωσης.
- Κατά τη ροή τους, τα logs μπορούν να υποστούν επεξεργασία μέσω pipelines, όπως parsing, εμπλουτισμό με επιπλέον πληροφορία ή αντιστοίχιση μέσω lookup tables ανάλογα με τις προτιμήσεις του χρήστη.
- Τα δεδομένα αποθηκεύονται indexing που ορίζονται από τον χρήστη.
- Τέλος, παρέχονται μηχανισμοί ανάλυσης, οπτικοποίησης και ειδοποίησης, όπως dashboards και alerts, για την υποστήριξη λειτουργιών παρακολούθησης και ασφάλειας.

Συνολικά, το Graylog αποτελεί ένα ολοκληρωμένο σύστημα διαχείρισης δεδομένων καταγραφής, το οποίο καλύπτει πολλαπλά στάδια επεξεργασίας, από τη συλλογή έως την ανάλυση και το alerting. Ωστόσο, δεν εστιάζει στην εξόρυξη διαδικασιών ούτε στην εκτέλεση αναλύσεων αυτού του τύπου, όπως άλλα εργαλεία. Επιπλέον, η αποτελεσματική χρήση του απαιτεί σημαντική εμπειρία από τον χρήστη, καθώς προϋποθέτει τον καθορισμό πολλών παραμέτρων και ρυθμίσεων στα ενδιάμεσα στάδια επεξεργασίας. Η πολυπλοκότητα αυτή καθιστά τη χρήση του συχνά απαιτητική, ιδίως όταν εμπλέκονται πολλαπλά επίπεδα επεξεργασίας ή και όταν ο χρήστης δεν ασχολείται αποκλειστικά με το συγκεκριμένο σύστημα αλλά και με άλλα πληροφοριακά περιβάλλοντα.

3.3.2 SigNoz

Το SigNoz [30] είναι μία πλατφόρμα ανοικτού κώδικα για ενοποιημένη παρακολούθηση εφαρμογών, η οποία συγκεντρώνει logs, metrics και traces στο ίδιο περιβάλλον, με στόχο την ταχύτερη ανίχνευση σφαλμάτων και τη διερεύνηση προβλημάτων απόδοσης. Παρουσιάζεται ως οικονομικότερη open-source εναλλακτική λύση απέναντι σε αντίστοιχα εμπορικά εργαλεία, όπως αναφέρεται στο repository του στο github.

Η βασική ροή λειτουργίας του SigNoz μπορεί να συνοψιστεί ως εξής: [28] [29]

- Αρχικά, το χρησιμοποιούμενο σύστημα διασυνδέεται με το SigNoz μέσω του OpenTelemetry, ώστε να αποστείλει δεδομένα αρχείων καταγραφής ενώ παράλληλα υποστηρίζει πολλά διαφορετικά περιβάλλοντα όπως python, java, wordpress και άλλες πλατφόρμες.

- Το SigNoz αναγνωρίζει τις υπηρεσίες του συστήματος με βάση τα δεδομένα που λαμβάνει από την εφαρμογή μέσω OpenTelemetry instrumentation. Πιο συγκεκριμένα, χρησιμοποιεί το πεδίο `service.name`, το οποίο δηλώνεται από την ίδια την εφαρμογή και αντιμετωπίζεται ως ξεχωριστή υπηρεσία της.

- Στη συνέχεια, το SigNoz εμφανίζει τις εφαρμογές και τις υπηρεσίες που στέλνουν δεδομένα στο σύστημα, προσφέροντας μία συγκεντρωτική εικόνα της κατάστασής τους.

- Για κάθε υπηρεσία, το SigNoz υπολογίζει και προβάλλει βασικές μετρικές παρακολούθησης, όπως latency, error rate και requests per second, ώστε να εντοπίζονται προβλήματα απόδοσης ή αποτυχίες.

- Επιπλέον, παρέχει δυνατότητες πιο αναλυτικής διερεύνησης μέσω traces, καθώς και ξεχωριστή παρακολούθηση για database calls και external calls, ώστε ο χρήστης να μπορεί να εξετάσει λεπτομερέστερα τη συμπεριφορά του συστήματος.

Και αυτό το εργαλείο συνιστά ένα ολοκληρωμένο εργαλείο για monitoring, το οποίο καλύπτει βασικά στάδια συλλογής, οργάνωσης και ανάλυσης λειτουργικών δεδομένων σε ένα ενιαίο περιβάλλον. Παρ' όλα αυτά, η σχεδίασή του εστιάζει κυρίως στην παρακολούθηση της απόδοσης εφαρμογών και στην ανίχνευση λειτουργικών προβλημάτων. Παράλληλα, η χρήση του προϋποθέτει ορισμένη αρχική παραμετροποίηση, όπως το κατάλληλο instrumentation και ο καθορισμός των υπηρεσιών που αποστέλλονται από τα τηλεμετρικά δεδομένα. Κατά συνέπεια, παρά την ευρεία υποστήριξη observability λειτουργιών, δεν στοχεύει άμεσα στη ροή ανάλυσης που απαιτείται σε σενάρια process mining ούτε μπορεί να επεξεργαστεί αυτόματα μη δομημένα αρχεία.

3.3.3 Elasticsearch

Το Elasticsearch [17] αποτελεί ένα κατανεμημένο σύστημα αναζήτησης και ανάλυσης δεδομένων, σχεδιασμένο για τη διαχείριση μεγάλου όγκου πληροφορίας και για την ταχεία

ανάκτηση αποτελεσμάτων. Υποστηρίζει διαφορετικούς τύπους δεδομένων και παρέχει δυνατότητες πραγματικού χρόνου για αναζήτηση και επεξεργασία.

Η βασική λογική λειτουργίας του Elasticsearch περιγράφεται ως εξής:

- Αρχικά, τα δεδομένα εισάγονται στο σύστημα σε μορφή `.json`.
- Στη συνέχεια, τα έγγραφα οργανώνονται σε συλλογές που περιλαμβάνουν δεδομένα παρόμοιου τύπου ή κοινής χρήσης.
- Κατά την εισαγωγή τους, το περιεχόμενο των εγγράφων μπορεί να υποστεί επεξεργασία, ιδιαίτερα όταν περιλαμβάνει κείμενο, ώστε οι όροι που περιέχονται σε αυτό να αναγνωρίζονται και να είναι διαθέσιμοι για αναζήτηση.
- Έπειτα, το Elasticsearch δημιουργεί indexing δομές, όπως ανεστραμμένα ευρετήρια, οι οποίες συνδέουν τους όρους με τα έγγραφα όπου εμφανίζονται, επιτρέποντας έτσι τη γρήγορη ανάκτηση πληροφορίας.
- Τέλος, το σύστημα παρέχει δυνατότητες αναζήτησης, φιλτραρίσματος και ανάλυσης δεδομένων με υψηλή απόδοση, ακόμη και σε περιβάλλοντα μεγάλης κλίμακας.

Συνολικά, το Elasticsearch αποτελεί ένα ισχυρό και ευέλικτο εργαλείο για αποθήκευση, οργάνωση, αναζήτηση και ανάλυση δεδομένων. Ωστόσο, ο κύριος προσανατολισμός του αφορά την αποδοτική διαχείριση και ανάκτηση πληροφορίας και όχι την εξόρυξη διαδικασιών. Επιπλέον, η αξιοποίησή του προϋποθέτει κατάλληλη παραμετροποίηση ως προς τη δομή και το indexing των δεδομένων. Επομένως, παρότι υποστηρίζει ουσιαστικά στάδια επεξεργασίας και ανάλυσης, δεν εστιάζει άμεσα στη μετατροπή logs σε event logs ούτε στην εκτέλεση process mining αναλύσεων.

3.4 Περιορισμοί

Παρά την ποικιλία και την τεχνολογική ωριμότητα των εργαλείων που παρουσιάστηκαν στις προηγούμενες ενότητες, προκύπτει ότι τα περισσότερα καλύπτουν μόνο επιμέρους στάδια της συνολικής επεξεργασίας των log δεδομένων. Οι log parsers επικεντρώνονται κυρίως στην εξαγωγή δομής ή templates από τα ακατέργαστα μηνύματα, χωρίς να προχωρούν συστηματικά στη μετατροπή των αποτελεσμάτων σε event logs ή στην εφαρμογή τεχνικών process mining. Αντίστοιχα, τα εργαλεία declarative ή process mining προϋποθέτουν συνήθως ως είσοδο ήδη δομημένα δεδομένα, συχνά σε μορφή `.xes`, χωρίς να αντιμετωπίζουν το πρόβλημα της αρχικής ανάλυσης και κανονικοποίησης των μη δομημένων αρχείων καταγραφής.

Τα εργαλεία που προσεγγίζουν την ανάλυση με πιο ολιστικό τρόπο, όπως το Graylog, το SigNoz και το Elasticsearch, εστιάζουν κυρίως στη συλλογή, αποθήκευση, αναζήτηση, παρακολούθηση και οπτικοποίηση δεδομένων. Η σχεδίασή τους εξυπηρετεί κυρίως ανάγκες monitoring και operational analytics και όχι μια ολοκληρωμένη προσέγγιση για εξόρυξη διαδικασιών. Επιπλέον, αρκετά από αυτά τα συστήματα απαιτούν σημαντική αρχική παραμετροποίηση, όπως ορισμό κανόνων, σχημάτων, υπηρεσιών, πεδίων ή pipelines, γεγονός που

περιορίζει τη δυνατότητα άμεσης εφαρμογής τους σε ετερογενή και άγνωστα σύνολα δεδομένων και την ευκολότερη αυτοματοποίηση της διαδικασίας.

Ένας ακόμη σημαντικός περιορισμός αφορά την έλλειψη μιας ενιαίας ροής που να συνδέει με συστηματικό τρόπο όλα τα απαραίτητα στάδια: από το raw log, στο structured log, από εκεί στη δημιουργία event logs και τελικά στην εξαγωγή γνώσης μέσω process mining. Στη βιβλιογραφία, τα στάδια αυτά αντιμετωπίζονται συχνά ως ανεξάρτητα προβλήματα, με αποτέλεσμα να απαιτούνται πολλαπλές ετερογενείς τεχνολογίες και χειροκίνητες παρεμβάσεις για τη σύνδεσή τους. Η αποσπασματική αυτή αντιμετώπιση αυξάνει την πολυπλοκότητα της διαδικασίας και ενδέχεται να εισαγάγει απώλεια πληροφορίας ή ασυνέπειες στα ενδιάμεσα βήματα.

Κατά συνέπεια, αναδεικνύεται η ανάγκη για μία πιο ενιαία και γενικεύσιμη προσέγγιση, η οποία να ξεκινά από μη δομημένα αρχεία καταγραφής διαφορετικής προέλευσης και να καταλήγει σε αναπαραστάσεις κατάλληλες για εξόρυξη διαδικασιών. Η ανάγκη αυτή αποτελεί και το βασικό κίνητρο της παρούσας εργασίας, η οποία επιχειρεί να γεφυρώσει τα κενά μεταξύ parsing, μετασχηματισμού των δεδομένων και declarative process mining μέσα από μία συνεκτική ροή επεξεργασίας.

4 Εφαρμογή Δηλωτικής Εξόρυξης Διαδικασιών σε Τυχαία Αρχεία Log: Μια Ολοκληρωμένη Προσέγγιση

Σε αυτήν την ενότητα παρουσιάζονται αναλυτικά οι αλγόριθμοι και τα επιμέρους στάδια που συγκροτούν το προτεινόμενο συνολικό μοντέλο. Ειδικότερα, περιγράφεται η υλοποίηση της πλήρους ροής επεξεργασίας, από τα ακατέργαστα αρχεία καταγραφής έως την παραγωγή των τελικών αποτελεσμάτων της εξόρυξης διαδικασιών.

4.1 Επισκόπηση Υλοποίησης

Αναλυτικότερα, η ροή του συστήματος, όπως απεικονίζεται στο Σχήμα 4.1, ξεκινά από την ανάλυση ενός μη δομημένου αρχείου log με το εργαλείο LILAC, με στόχο τον εντοπισμό των τύπων γεγονότων, δηλαδή των event templates και eventIDs. Με βάση τα αποτελέσματα του parsing και τη δοσμένη μορφοποίηση του log (format), παράγεται ένα αρχείο `.csv`, στο οποίο κάθε καταγραφή μετατρέπεται σε δομημένη μορφή και κάθε πεδίο της αντιστοιχίζεται σε μία ξεχωριστή στήλη. Στη συνέχεια, εφαρμόζεται ο προτεινόμενος αλγόριθμος δημιουργίας ιχνών, ώστε οι καταγραφές να οργανωθούν σε event logs, τα οποία μετατρέπονται σε μορφή `.xes` για να καταστούν κατάλληλα για εξόρυξη διαδικασιών. Στο τελικό στάδιο, αξιοποιείται το MINERful για την εξαγωγή δηλωτικών περιορισμών, οι οποίοι παρέχουν χρήσιμη πληροφορία για τις σχέσεις και τις εξαρτήσεις μεταξύ των γεγονότων.

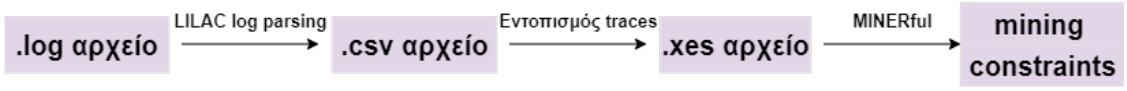


Figure 4.1: Συνολική ροή της προσέγγισης

Από άποψη υλοποίησης, η συνολική ροή της μεθοδολογίας περιγράφεται στον Αλγόριθμο 1. Αρχικά, γίνεται log parsing στο ακατέργαστο αρχείο καταγραφής ώστε να εξαχθούν τα βασικά δομικά στοιχεία των μηνυμάτων. Στη συνέχεια, τα δεδομένα αυτά μετασχηματίζονται σε event log, μέσω της αναγνώρισης των πεδίων και της δημιουργίας traces. Τέλος, το event log σε μορφή `.xes` δίνεται ως είσοδος στο MINERful, το οποίο εξάγει τους δηλωτικούς περιορισμούς που περιγράφουν τη συμπεριφορά της διαδικασίας.

Algorithm 1 Συνολική ροή της προτεινόμενης μεθοδολογίας

1: **structured_log.csv, log_templates.csv** ← run_lilac_parsing(**raw_log.log** shot, example_size, model)
2: **output_log.xes** ← convert_csv_to_xes(**raw_log.log, structured_log.csv, log_templates.csv**, num_of_avg_intervals)
3: **minerful_output.csv** ← run_minerful(**output_log.xes**, support_threshold)

4.2 Παρουσίαση και Αξιοποίηση του LILAC στην Προτεινόμενη Μεθοδολογία

Σε αυτό το κεφάλαιο παρουσιάζονται η βασική λειτουργία του LILAC, οι παρεμβάσεις που πραγματοποιήθηκαν με στόχο τη γενίκευση της χρήσης του, καθώς και ο τρόπος με τον οποίο ενσωματώνεται στην προτεινόμενη μεθοδολογία της παρούσας διπλωματικής εργασίας.

4.2.1 Βασική Αρχιτεκτονική του LILAC

Στο πλαίσιο της παρούσας εργασίας το LILAC [37] είναι το εργαλείο που χρησιμοποιείται για την εξαγωγή templates και eventIDs από τα ακατέργαστα log μηνύματα παρέχοντας τη δομημένη αναπαράσταση πάνω στην οποία βασίζονται τα επόμενα στάδια του μετασχηματισμού και της εξόρυξης διαδικασιών.

Το LILAC αποτελεί μία προσέγγιση log parsing βασισμένη σε Μεγάλα Γλωσσικά Μοντέλα, στην οποία η ανάλυση των log μηνυμάτων πραγματοποιείται μέσω ενός parser που αξιοποιεί το in-context learning. Το βασικό στοιχείο που διαφοροποιεί το LILAC από άλλες αντίστοιχες LLM-based προσεγγίσεις είναι η adaptive parsing cache, η οποία αποθηκεύει τα templates που έχουν ήδη εξαχθεί, αποφεύγει επαναλαμβανόμενα ερωτήματα προς το γλωσσικό μοντέλο και υποστηρίζει την προσαρμοστική αναθεώρηση των αποτελεσμάτων.

Η parsing cache, η οποία αποθηκεύει τα templates που έχουν ήδη εξαχθεί, στηρίζεται στην παρατήρηση ότι στα πραγματικά συστήματα ο αριθμός των διαφορετικών log templates είναι σημαντικά μικρότερος από τον συνολικό αριθμό των log μηνυμάτων. Έτσι, μέσω της αποθήκευσης και της αντιστοίχισης με ήδη γνωστά templates, αποφεύγονται επαναλαμβανόμενα ερωτήματα προς το γλωσσικό μοντέλο και βελτιώνεται σημαντικά η αποδοτικότητα της διαδικασίας parsing.

Για κάθε νέο log μήνυμα, το LILAC ελέγχει αρχικά αν υπάρχει ήδη αντίστοιχο template στην parsing cache. Αν βρεθεί αντιστοίχιση, το αποθηκευμένο template χρησιμοποιείται άμεσα ως αποτέλεσμα parsing. Διαφορετικά, ανακτώνται από τη μνήμη σχετικά templates με υψηλή ομοιότητα προς το νέο μήνυμα και στη συνέχεια ο ICL-enhanced parser δημιουργεί το κατάλληλο prompt και ζητά από το γλωσσικό μοντέλο την εξαγωγή νέου template. Με βάση τόσο το νέο template όσο και τα σχετικά αποθηκευμένα templates, το σύστημα αποφασίζει είτε να εισαγάγει νέο template στην cache είτε να αναθεωρήσει κάποιο ήδη υπάρχον, ώστε τα αποτελέσματα να παραμένουν πιο συνεπή και ακριβή.

Ο παραπάνω μηχανισμός αυτοδιόρθωσης είναι ιδιαίτερα σημαντικός σε περιπτώσεις όπου άλλα εργαλεία δυσκολεύονται να γενικεύσουν, όπως σε μηνύματα χαμηλής συχνότητας ή σε tokens που μοιάζουν γλωσσικά με σταθερούς όρους, παρότι στην πραγματικότητα αποτελούν μεταβλητές τιμές. Χαρακτηριστικά παραδείγματα είναι λέξεις όπως `admin` ή `root`, οι οποίες σε αρκετά περιβάλλοντα μπορούν να εμφανίζονται είτε ως σταθερά στοιχεία είτε ως ειδικές τιμές πεδίων. Χάρη στην αξιοποίηση σημασιολογικής πληροφορίας και

στη συνεχή αναθεώρηση της cache, το LILAC επιτυγχάνει υψηλή ακρίβεια και αυξημένη προσαρμοστικότητα σε ετερογενή σύνολα δεδομένων, υπερέχοντας συχνά έναντι αμιγώς συντακτικών ή παλαιότερων σημασιολογικών προσεγγίσεων.

Για την παρούσα μεθοδολογία, ο μηχανισμός αυτός είναι κρίσιμος επειδή επηρεάζει άμεσα την ποιότητα της ενδιάμεσης δομημένης αναπαράστασης. Όσο πιο συνεπής είναι η γενίκευση των μεταβλητών τμημάτων και η αντιστοίχιση των μηνυμάτων σε σωστά templates, τόσο πιο αξιόπιστα είναι και τα δεδομένα που τροφοδοτούν τα επόμενα στάδια δημιουργίας event logs και εξόρυξης περιορισμών.

Ενδεικτική Εκτέλεση του LILAC

Το LILAC εκτελείται μέσω του αντίστοιχου github repository [15] και η χρήση του προϋποθέτει τον καθορισμό βασικών παραμέτρων που επηρεάζουν τη διαδικασία του parsing. Ειδικότερα, ορίζεται ο αριθμός των shots, δηλαδή το σύνολο των διαθέσιμων υποψήφιων παραδειγμάτων, ο αριθμός των in-context examples που θα αντληθούν από το αντίστοιχο αρχείο shots για τη διαμόρφωση του prompt, καθώς και το γλωσσικό μοντέλο που θα χρησιμοποιηθεί. Στο παράδειγμα που ακολουθεί, για `shot=8` και `example_size=2`, το σύστημα επιλέγει κάθε φορά τα δύο πιο σχετικά παραδείγματα από το διαθέσιμο σύνολο των οκτώ, με βάση την ομοιότητά τους προς το εκάστοτε νέο log μήνυμα και χρησιμοποιεί το μοντέλο GPT-3.5, όπως φαίνεται στον Αλγόριθμο 2.

Algorithm 2 Εκτέλεση του LILAC για log parsing

```
cd ...\LILAC-main\benchmark\evaluation
python LILAC_eval.py –shot 8 –example_size 2 –model gpt-3.5
```

Μετά την ολοκλήρωση της επεξεργασίας, το LILAC παράγει για κάθε dataset δύο αρχεία `.csv`. Το αρχείο `dataset_name_structured.csv` περιλαμβάνει τα αρχικά log μηνύματα μαζί με το αναγνωριστικό γεγονότος (eventID) και το template που αποδόθηκε σε κάθε ένα από αυτά, ενώ το αρχείο `dataset_name_templates.csv` περιέχει το σύνολο των διαφορετικών templates που αναγνωρίστηκαν και τα αντίστοιχα αναγνωριστικά τους. Τα αρχεία αυτά αποτελούν την ενδιάμεση δομημένη αναπαράσταση πάνω στην οποία βασίζονται τα επόμενα στάδια της μεθοδολογίας.

Περιορισμοί και Πρακτικές Δυσκολίες του LILAC

Παρά τα πλεονεκτήματά του, το LILAC παρουσιάζει και ορισμένους πρακτικούς περιορισμούς όταν επιχειρείται η αξιοποίησή του πέρα από τα αρχικά benchmark datasets για τα οποία έχει σχεδιαστεί. Η ενσωμάτωση ενός νέου συνόλου δεδομένων δεν είναι πλήρως αυτοματοποιημένη, καθώς απαιτεί την προσθήκη ή τροποποίηση πολλών αρχείων όπως στα αρχεία settings.py, LILAC_eval.py και το LILAC_sampling. Αυτό δείχνει ότι, παρότι το εργαλείο μπορεί να προσαρμοστεί, η επέκτασή του σε νέα περιβάλλοντα εξακολουθεί να προϋποθέτει χειροκίνητες παρεμβάσεις και καλή γνώση της εσωτερικής δομής του κώδικα.

Ένας επιπλέον περιορισμός αφορά την εξάρτηση από sampled examples. Στο αρχικό LILAC, τα παραδείγματα αυτά δεν εισάγονται χειροκίνητα από τον χρήστη ένα προς ένα,

αλλά παράγονται μέσω της διαδικασίας sampling από τα ήδη καθορισμένα benchmark datasets. Κατά συνέπεια, όταν ένα νέο dataset δεν έχει ενσωματωθεί κατάλληλα στη διαδικασία αυτή ή όταν δεν μπορούν να παραχθούν τα αντίστοιχα sampled examples, το εργαλείο δεν μπορεί να αξιοποιήσει κανονικά τον μηχανισμό in-context learning. Επομένως, η απόδοση του LILAC δεν εξαρτάται μόνο από το ίδιο το γλωσσικό μοντέλο, αλλά και από τη διαθεσιμότητα της απαραίτητης υποδομής δειγματοληψίας και παραδειγμάτων για κάθε dataset.

Ακόμη, στην αρχική του μορφή το LILAC βασίζεται στη χρήση μοντέλων GPT μέσω της υπηρεσίας της OpenAI, γεγονός που προϋποθέτει κατάλληλη πρόσβαση στην αντίστοιχη υποδομή και συνδέει τη λειτουργία του με μία εξωτερική υπηρεσία για την εκτέλεση του γλωσσικού μοντέλου. Κατά συνέπεια, η αξιοποίησή του δεν είναι πλήρως αυτόνομη στο αρχικό του περιβάλλον.

4.2.2 Τροποποιήσεις του LILAC

Με γνώμονα τους παραπάνω περιορισμούς, στην παρούσα εργασία πραγματοποιήθηκαν στοχευμένες τροποποιήσεις σε επιμέρους αρχεία του LILAC, με στόχο τη διευκόλυνση της εφαρμογής του σε νέα ή διαφορετικά σύνολα δεδομένων και σε πιο πρακτικά σενάρια εκτέλεσης. Οι παρεμβάσεις αυτές δεν μεταβάλλουν τον βασικό πυρήνα του αλγορίθμου log parsing ή της διαδικασίας sampling, αλλά αφορούν κυρίως την οργάνωση του κώδικα, την παραμετροποίηση της εκτέλεσής του και τη συνολική προσαρμογή του εργαλείου στις ανάγκες της παρούσας εργασίας.

Αρχικά, τροποποιήθηκε το αρχείο LILAC_sampling.py, το οποίο υλοποιεί τη διαδικασία δειγματοληψίας των παραδειγμάτων που αξιοποιούνται στη συνέχεια από το σύστημα για τη διαμόρφωση των in-context examples. Οι τροποποιήσεις στο αρχείο αυτό αφορούσαν κυρίως την οργάνωση και τη γενίκευση της ροής εκτέλεσης του script και όχι τον ίδιο τον πυρήνα της δειγματοληπτικής μεθόδου. Ειδικότερα, η αρχική υλοποίηση, στην οποία οι βασικές παράμετροι ήταν σταθερά ορισμένες μέσα στο κύριο σώμα του προγράμματος, επεκτάθηκε ώστε να υποστηρίζει παραμετροποίηση μέσω γραμμής εντολών, δυναμική επιλογή των shot values, πιο συστηματική διαχείριση διαδρομών αρχείων, καθώς και ρητούς ελέγχους ύπαρξης των απαιτούμενων αρχείων εισόδου και των ήδη παραγόμενων αποτελεσμάτων. Παράλληλα, η κύρια λογική επεξεργασίας μεταφέρθηκε από το κύριο σώμα του προγράμματος σε μία ξεχωριστή συνάρτηση, γεγονός που καθιστά τον κώδικα πιο καθαρό και οργανωμένο.

Στη συνέχεια, τροποποιήθηκε το αρχείο gpt_query.py, το οποίο είναι υπεύθυνο για τη διασύνδεση του LILAC με το γλωσσικό μοντέλο κατά το στάδιο εξαγωγής των templates, και αντικαταστάθηκε από το ollama_query.py. Η βασική μεταβολή σε αυτό το σημείο δεν αφορά τον τρόπο με τον οποίο το σύστημα ερμηνεύει, ελέγχει και μετα-επεξεργάζεται τα παραγόμενα templates, αλλά τον μηχανισμό μέσω του οποίου πραγματοποιείται η κλήση του μοντέλου. Ειδικότερα, στην αρχική υλοποίηση η επικοινωνία γινόταν μέσω της υπηρεσίας OpenAI, με χρήση κατάλληλων διαπιστευτηρίων πρόσβασης και απομακρυσμένων κλήσεων προς τα αντίστοιχα μοντέλα, ενώ στη νέα εκδοχή η ίδια γενική ροή προσαρμόστηκε ώστε να αξιοποιεί τοπικό μοντέλο μέσω της πλατφόρμας Ol-

lama. Παράλληλα, διατηρήθηκαν οι βασικές συναρτήσεις και η κύρια λογική του εργαλείου, ώστε να αλλάξει μόνο ο τρόπος επικοινωνίας με το LLM και όχι ο θεμελιώδης τρόπος λειτουργίας του LILAC. Με τον τρόπο αυτό, το σύστημα καθίσταται περισσότερο αυτόνομο και λιγότερο εξαρτημένο από εξωτερικές υπηρεσίες, χωρίς να μεταβάλλεται ουσιαστικά ο μηχανισμός παραγωγής και επαλήθευσης των log templates.

Τροποποιήθηκε επίσης το αρχείο LILAC_eval.py, το οποίο χρησιμοποιείται για την εκτέλεση του parsing, με παρεμβάσεις που αφορούσαν κυρίως την οργάνωση της ροής εκτέλεσης και τη διαχείριση των αρχείων εισόδου και εξόδου. Οι αλλαγές αυτές δεν μεταβάλλουν τον βασικό τρόπο λειτουργίας του script, αλλά διευκολύνουν την αξιοποίησή του στο πλαίσιο της παρούσας εργασίας.

4.2.3 Περιγραφή της Ενσωμάτωσης του LILAC στον κώδικα

Για την ενσωμάτωση του LILAC στην προτεινόμενη υλοποίηση, καθώς και για την αξιοποίηση των τροποποιήσεων που περιγράφηκαν προηγουμένως, ορίστηκαν ορισμένες βοηθητικές συναρτήσεις που αναλαμβάνουν την προετοιμασία και την προσαρμογή του, ώστε να μπορεί να χρησιμοποιηθεί μέσα στον συνολικό κώδικα. Αρχικά, γίνεται η προετοιμασία του περιβάλλοντος εκτέλεσης και ελέγχεται ότι υπάρχουν τα απαραίτητα αρχεία παραδειγμάτων που απαιτούνται για τη διαδικασία parsing. Σε περίπτωση που αυτά δεν είναι διαθέσιμα, ο αριθμός των παραδειγμάτων ορίζεται σε 0, ώστε το πρόγραμμα να μπορεί να εκτελεστεί και χωρίς τη χρήση in-context examples.

Στη συνέχεια, άλλες βοηθητικές συναρτήσεις αναλαμβάνουν την προσωρινή προσαρμογή των ρυθμίσεων του LILAC, ώστε να μπορεί να εκτελεστεί για ένα νέο dataset χωρίς να απαιτούνται χειροκίνητες αλλαγές στα αρχεία του. Οι αλλαγές αυτές εφαρμόζονται μόνο για τη συγκεκριμένη εκτέλεση και, μετά την ολοκλήρωσή της, τα αρχεία επανέρχονται στην αρχική τους κατάσταση. Έτσι, το LILAC εντάσσεται στη συνολική ροή του κώδικα με πιο συστηματικό τρόπο και αυτοματοποιούνται βασικά βήματα που διαφορετικά θα έπρεπε να εκτελούνται χειροκίνητα πριν από κάθε χρήση του LILAC.

Στο κύριο πρόγραμμα, καλούνται οι παραπάνω βοηθητικές συναρτήσεις ώστε το LILAC να λειτουργεί ως επιμέρους στάδιο της συνολικής μεθοδολογίας. Πιο συγκεκριμένα, το πρόγραμμα προετοιμάζει το περιβάλλον, εφαρμόζει τις αναγκαίες προσωρινές προσαρμογές, εκτελεί το parsing καλώντας το LILAC και παραλαμβάνει τα δομημένα αρχεία που παράγονται. Τα αρχεία αυτά χρησιμοποιούνται στη συνέχεια στα επόμενα στάδια της μετατροπής σε event logs και της εξόρυξης διαδικασιών. Με αυτόν τον τρόπο, το LILAC δεν χρησιμοποιείται ως απομονωμένο εξωτερικό εργαλείο, αλλά ως ενσωματωμένο τμήμα της συνολικής ροής επεξεργασίας.

Ο Αλγόριθμος 3 συνοψίζει τη λογική με την οποία το LILAC εντάσσεται στη συνολική ροή εκτέλεσης. Αρχικά προετοιμάζεται το περιβάλλον και ελέγχεται η διαθεσιμότητα των απαραίτητων αρχείων παραδειγμάτων, ενώ όταν αυτά δεν υπάρχουν ο αριθμός των examples μηδενίζεται ώστε η εκτέλεση να συνεχίζεται κανονικά. Στη συνέχεια εφαρμόζονται οι αναγκαίες προσωρινές αλλαγές, εκτελείται το parsing και τα παραγόμενα αρχεία επιστρέφονται στο κύριο πρόγραμμα και τέλος γίνεται η αποκατάσταση των αρχικών αρχείων.

Algorithm 3 Ενσωμάτωση του LILAC στον συνολικό κώδικα

```
prepare_lilac_environment()
sample_files_exist ← ensure_sample_files(dataset, shot)
if sample files do not exist then
    example_size ← 0
settings_backup ← backup_settings()
eval_backup ← backup_lilac_eval()
update_settings_for_dataset(dataset)
run_lilac_parsing(dataset, shot, example_size, model)
structured_logs.csv, log_templates.csv ← load_parsing_results()
restore_settings(settings_backup)
restore_lilac_eval(eval_backup)
```

4.3 Μετατροπή των αποτελεσμάτων parsing σε event logs και αρχεία XES

Μετά την ολοκλήρωση του parsing από το LILAC, το επόμενο στάδιο επεξεργασίας είναι η διαμόρφωση των παραγόμενων δομημένων αρχείων για να είναι έτοιμα για την εξόρυξη διαδικασιών. Για τον σκοπό αυτό υλοποιήθηκε ένα σύνολο συναρτήσεων που αναλαμβάνει τη μετάβαση από τα αποτελέσματα του parsing σε ίχνη και τελικά σε αρχεία μορφής `.xes` . Οι συναρτήσεις αυτές δεν περιορίζονται σε μία απλή μετατροπή μορφής αρχείων, αλλά καλύπτουν βήματα όπως την ερμηνεία της μορφής του log, την εξαγωγή και αποκατάσταση χρονικής πληροφορίας, την εκτίμηση ενός κατάλληλου χρονικού παραθύρου για τη δημιουργία traces και την τελική οργάνωση των γεγονότων σε event logs.

Αρχικά αξιοποιείται το δοσμένο log format του συνόλου δεδομένων ώστε να αντιστοιχηθούν τα κομμάτια των καταγραφών στα πεδία που περιγράφουν. Έτσι, καθίσταται δυνατή η εξαγωγή στοιχείων όπως τα χρονικά δεδομένα, τα events με τα αντίστοιχα αναγνωριστικά τους και οι υπόλοιπες πληροφορίες που περιέχει κάθε καταγραφή. Στη συνέχεια εκτελούνται συναρτήσεις για την μετατροπή των χρονικών πληροφοριών σε timestamps έτσι ώστε να είναι συμβατές με την μορφή που χρειάζεται να έχουν στο αρχείο `.xes` που θα δημιουργηθεί.

Για την δημιουργία των traces υπολογίζεται ένα κατάλληλο χρονικό παράθυρο έτσι ώστε να ομαδοποιηθούν τα γεγονότα. Το παράθυρο αυτό υπολογίζεται με βάση τη μέση χρονική απόσταση μεταξύ των log και μιας παραμέτρου που καθορίζει πόσες φορές επί αυτήν την διάρκεια θα είναι το χρονικό παράθυρο αυτό. Τέλος, κάθε γραμμή log αντιστοιχίζεται σε ένα συγκεκριμένο trace και παράγεται ένα ενδιάμεσο αρχείο `.csv` όπου πέρα από τα στοιχεία κάθε καταγραφής σημειώνεται και το αναγνωριστικό του ίχνους (traceID) και έπειτα μετατρέπεται σε μορφή `.xes`, ώστε να μπορεί να χρησιμοποιηθεί στο στάδιο της εξόρυξης διαδικασιών. Η λογική αυτής της διαδικασίας παρουσιάζεται παρακάτω στον Αλγόριθμο 4.

Algorithm 4 Μετατροπή των αποτελεσμάτων σε αρχείο XES

```
normalize_log_format(log_format)
extract_fields_from_log_lines(raw_log, log_format)
infer_timestamps(raw_log, log_format)
time_window ← estimate_trace_window(raw_log, log_format)
event_log.csv ← assign_logs_to_traces(raw_log, structured_log, time_window)
output_log.xes ← convert_csv_to_xes(event_log.csv)
```

4.3.1 Διαχωρισμός πεδίων και δημιουργία CSV

Το πρώτο στάδιο του μετασχηματισμού αφορά την αποδόμηση κάθε καταγραφής σε πεδία και την δημιουργία ενός ενδιάμεσου αρχείου `.csv`. Αρχικά, το δοσμένο log format κανονικοποιείται, ώστε χαρακτήρες που εμφανίζονται κυριολεκτικά στη συμβολοσειρά του log format, όπως αγκύλες, παρενθέσεις ή κατακόρυφες γραμμές, να αποδίδονται σωστά πριν από την ανάλυση των πεδίων. Στη συνέχεια, κάθε πεδίο του format αντιστοιχίζεται σε ένα συγκεκριμένο μέρος της καταγραφής και εντοπίζονται τα στοιχεία του log όπως η ημερομηνία, η ώρα, το μήνυμα και οποιαδήποτε άλλα γνωρίσματα περιλαμβάνει το εκάστοτε log.

Πέρα από τον απλό διαχωρισμό κάθε γραμμής στα επιμέρους πεδία της, η διαδικασία αυτή περιλαμβάνει και τον χειρισμό της χρονικής πληροφορίας. Η απόδοση της χρονικής σήμανσης δεν γίνεται με έναν μόνο τρόπο, αλλά ακολουθεί μία σειρά ελέγχων που προβλέπει η υλοποίηση για να καλύψει όσο το δυνατόν περισσότερα formats. Αρχικά αναζητούνται στο log format πεδία με ονομασίες date, month, day, time και timestamp. Η αντιστοίχιση των ονομασιών αυτών πραγματοποιείται χωρίς διάκριση πεζών και κεφαλαίων χαρακτήρων, ώστε να μπορούν να αναγνωρίζονται ισοδύναμες γραφές των ίδιων πεδίων. Για κάθε καταγραφή, εξετάζεται πρώτα αν η πληροφορία time μπορεί να συνδυαστεί με πεδία όπως month και day, ώστε να προκύψει το ζητούμενο timestamp. Αν αυτό δεν είναι δυνατό, ελέγχεται ο συνδυασμός date και time. Στη συνέχεια, αν υπάρχει πεδίο timestamp, αυτό εξετάζεται ως έχει και επιχειρείται να αναγνωριστεί πρώτα ως πλήρες datetime, δηλαδή ως τιμή που ήδη περιέχει ημερομηνία και ώρα. Σε περίπτωση που δεν περιέχει και τις δύο πληροφορίες, το πεδίο αντιμετωπίζεται ως απλή χρονική πληροφορία και οι μέρες ανατίθονται αυτόματα με time offset. Αντίστοιχα, όταν δεν υπάρχει αξιοποιήσιμο timestamp, γίνεται προσπάθεια ανάκτησης της χρονικής τιμής μόνο από το time, εάν αυτό υπάρχει. Η παραπάνω διαδικασία φαίνεται καλύτερα στον Αλγόριθμο 5

Η διαδικασία αυτή είναι αναγκαία, επειδή τα σύνολα δεδομένων δεν ακολουθούν κοινή μορφή ως προς τα χρονικά πεδία. Βέβαια, επειδή η αναγνώριση των πεδίων βασίζεται στις ονομασίες που δηλώνονται στο format, ο μηχανισμός δεν μπορεί να εντοπίσει σωστά τη χρονική πληροφορία όταν τα πεδία αυτά έχουν ονομαστεί με διαφορετικές μορφές.

Αφού εξαχθούν τα επιμέρους πεδία και αποδοθεί timestamp σε κάθε εγγραφή, δημιουργείται ένα νέο ενδιάμεσο αρχείο `.csv`, το οποίο δεν περιέχει μόνο τα αποτελέσματα του parsing αλλά και τη δομή που απαιτείται για τα επόμενα στάδια. Στο αρχείο αυτό αποθηκεύονται για κάθε γραμμή το αναγνωριστικό ίχνους, το αναγνωριστικό γεγονότος, η

Algorithm 5 Απόδοση χρονικής σήμανσης σε μία καταγραφή από τα πεδία της

```
1: temporal_fields ← get_temporal_fields(log_format)
2: for each log_record do
3:     if time + month/day or date are available then
4:         timestamp ← get_combined_timestamp()
5:     else if timestamp is available then
6:         if timestamp already contains date and time then
7:             timestamp ← get_full_timestamp()
8:         else
9:             timestamp ← get_time_based_timestamp()
10:            update_day_offset()
11:    else if time is available then
12:        timestamp ← get_time_based_timestamp()
13:        update_day_offset()
14:    else
15:        timestamp ← null
16:    return timestamp
```

χρονική σήμανση, το template του γεγονότος, το αρχικό μήνυμα και ολόκληρη η γραμμή log. Παράλληλα, διατηρούνται και τα υπόλοιπα διαθέσιμα πεδία ως επιπλέον γνωρίσματα. Έτσι, το `.csv` λειτουργεί ως ενδιάμεση αναπαράσταση ανάμεσα στο raw log και στο τελικό event log, διατηρώντας τόσο τη δομή όσο και τη σημασιολογική πληροφορία της αρχικής καταγραφής.

Μόλις ολοκληρωθεί η εξαγωγή των πεδίων και έχει προσδιοριστεί το timestamp κάθε εγγραφής, παράγεται ένα ενδιάμεσο αρχείο `.csv` για τα επόμενα στάδια της επεξεργασίας. Σε αυτό καταγράφονται, για κάθε γραμμή log, το αναγνωριστικό του ίχνους (traceID), το αναγνωριστικό του γεγονότος (eventID), το timestamp, το πρότυπο (event template), το αρχικό μήνυμα και ολόκληρη η γραμμή της καταγραφής. Διατηρούνται επίσης και τα υπόλοιπα πεδία που έχουν εξαχθεί, ώστε να μην χάνεται πρόσθετη πληροφορία από το αρχικό log. Το αρχείο `.csv` που δημιουργείται αποτελεί το ενδιάμεσο βήμα ανάμεσα στο ακατέργαστο αρχείο καταγραφής και το τελικό event log, διατηρώντας τόσο τη δομή όσο και το περιεχόμενο της αρχικής εγγραφής.

Η δημιουργία των traces γίνεται με γνώμονα τον χρόνο, επομένως τα events κάθε trace αντιστοιχούν σε μια ακολουθία γεγονότων που έχουν συμβεί στο σύστημα σε ένα ορισμένο χρονικό διάστημα. Με βάση το χρονικό παράθυρο που έχει υπολογιστεί, κάθε νέα καταγραφή εντάσσεται στο ίδιο ίχνος ή οδηγεί στη δημιουργία νέου. Αυτό συμβαίνει είτε όταν η διάρκεια του τρέχοντος ίχνους ξεπεράσει το καθορισμένο όριο του χρονικού παραθύρου, είτε όταν η απόσταση από την προηγούμενη καταγραφή είναι αρκετά μεγάλη ώστε να θεωρηθεί ότι ξεκινά νέα ακολουθία γεγονότων. Το αρχείο `.csv` που δημιουργείται δεν αποτελεί απλώς ένα αρχείο με διαχωρισμένα πεδία, αλλά την ολοκληρωμένη μορφή event log πάνω στην οποία θα βασιστεί η τελική εξαγωγή σε `.xes`.

4.3.2 Μετατροπή CSV αρχείου σε XES

Το αρχείο `.csv` που έχει δημιουργηθεί από την παραπάνω υλοποίηση μετατρέπεται σε αρχείο τύπου `.xes`. Σε αυτό το στάδιο, το `.csv` φορτώνεται ως πίνακας δεδομένων και γίνεται αρχικά έλεγχος στις χρονικές τιμές που έχουν αποδοθεί στα γεγονότα. Η στήλη του χρόνου μετατρέπεται σε κατάλληλη μορφή timestamp, ενώ οι εγγραφές για τις οποίες δεν προκύπτει έγκυρη χρονική πληροφορία απομακρύνονται. Το βήμα αυτό είναι απαραίτητο, ώστε τα γεγονότα να μπορούν να τοποθετηθούν σωστά στον χρόνο και το παραγόμενο event log να είναι συνεπές για τα επόμενα στάδια της ανάλυσης.

Στη συνέχεια, οι βασικές στήλες του `.csv` αντιστοιχίζονται στα πεδία που χρησιμοποιούνται στο πρότυπο `.xes`. Πιο συγκεκριμένα, το αναγνωριστικό του ίχνους χρησιμοποιείται ως case identifier, το όνομα του γεγονότος αντιστοιχίζεται στο activity name και η χρονική σήμανση στο αντίστοιχο πεδίο του χρόνου. Με αυτή τη μετατροπή, το αρχείο αποκτά τη δομή που απαιτείται για να ερμηνευθεί ως event log.

Τα υπόλοιπα πεδία του αρχείου `.csv` δεν παραλείπονται, αλλά μεταφέρονται ως γνωρίσματα των γεγονότων και έτσι διατηρούνται πληροφορίες όπως το επίπεδο σοβαρότητας, το αρχικό περιεχόμενο της καταγραφής ή άλλα πεδία που έχουν εξαχθεί στα προηγούμενα βήματα. Αυτό έχει σημασία, επειδή επιτρέπει το τελικό αρχείο να διατηρεί περισσότερη πληροφορία από την αρχική καταγραφή και όχι μόνο τη βασική ακολουθία των γεγονότων.

Στο τελευταίο βήμα, ο πίνακας δεδομένων μετατρέπεται από τη βιβλιοθήκη process mining σε εσωτερική αναπαράσταση event log και στη συνέχεια εξάγεται σε μορφή `.xes`. Το αρχείο που προκύπτει αποτελεί πλέον την τελική είσοδο για την εξόρυξη διαδικασιών, αφού περιέχει traces, events και τα αντίστοιχα γνωρίσματά τους σε τυποποιημένη μορφή. Με αυτόν τον τρόπο ολοκληρώνεται η μετάβαση από το ενδιάμεσο δομημένο αρχείο `.csv` σε ένα event log που μπορεί να χρησιμοποιηθεί άμεσα στα επόμενα στάδια της μεθοδολογίας

4.4 Δηλωτική Εξόρυξη Διαδικασιών με χρήση του εργαλείου MINERful

Στο τελικό στάδιο της μεθοδολογίας το MINERful [6] χρησιμοποιείται μετά τη δημιουργία των event logs σε μορφή `.xes` για την εξόρυξη περιορισμών. Σε αντίθεση με το LILAC, εδώ δεν απαιτήθηκαν τροποποιήσεις στον πυρήνα του εργαλείου. Το MINERful δέχεται ως είσοδο τα αρχεία που παράχθηκαν στα προηγούμενα στάδια και επιστρέφει τα αποτελέσματα του declarative process mining σε ένα νέο αρχείο `.csv`.

4.4.1 Βασική Λειτουργία του MINERful

Όπως περιγράφεται και στο σχετικό paper, το MINERful είναι μία μέθοδος ανακάλυψης δηλωτικών περιορισμών από event logs, με έμφαση σε διαδικασίες όπου δεν υπάρχει μία αυστηρά προκαθορισμένη ροή εκτέλεσης. Η βασική του λογική είναι ότι το μοντέλο δεν εκφράζεται ως ένα πλήρες διάγραμμα, αλλά ως ένα σύνολο constraints μεταξύ δραστηριοτήτων, το οποίο αποτυπώνει τι επιτρέπεται, τι αναμένεται να συμβεί και ποιες σχέσεις

εμφανίζονται συστηματικά μέσα στα traces.

Το MINERful λειτουργεί σε 2 φάσεις. Αρχικά οργανώνει τη διαθέσιμη πληροφορία του log σε μία ενδιάμεση βάση γνώσης με στατιστικά στοιχεία για τις εμφανίσεις και τις συσχετίσεις των δραστηριοτήτων. Στη συνέχεια, πάνω σε αυτή τη βάση, υπολογίζει τους περιορισμούς που υποστηρίζονται από τα δεδομένα. Με αυτόν τον τρόπο αποφεύγεται ένας πιο ακριβός και επαναληπτικός έλεγχος όλων των πιθανών constraints απευθείας πάνω στα traces.

Το εργαλείο εξάγει περιορισμούς τύπου Declare και τους συνοδεύει με μετρικές όπως το support, το confidence level και το interest factor. Επιπλέον, απομακρύνονται constraints που είναι πλεονάζοντα ή ασθενέστερα από άλλα ισχυρότερα της ίδιας ιεραρχίας έτσι ώστε το τελικό μοντέλο να παραμένει πιο συμπαγές και πιο εύχρηστο στην ερμηνεία του.

Ενδεικτική Εκτέλεση του MINERful

Το MINERful εκτελείται μέσω του αντίστοιχου github repository [5] και, στο πλαίσιο της παρούσας εργασίας, αξιοποιείται ως εξωτερικό εργαλείο java μετά την παραγωγή του αρχείου `.xes`. Για να είναι δυνατή η εκτέλεσή του, απαιτείται να είναι διαθέσιμα στο τοπικό repository το MINERful.jar, ο φάκελος bin και οι βιβλιοθήκες του φακέλου lib, από τα οποία συντίθεται το απαραίτητο classpath. Παράλληλα, απαιτείται και η ύπαρξη εγκατεστημένου περιβάλλοντος java.

Το MINERful μπορεί να κληθεί με διάφορες παραμέτρους. Η βασικότερη από είναι το support threshold, το οποίο καθορίζει πόσο συχνά πρέπει να ικανοποιείται ένας περιορισμός μέσα στα traces ώστε να διατηρηθεί στο τελικό αποτέλεσμα. Πέρα από το support, στο paper περιγράφονται και οι συμπληρωματικές μετρικές confidence level και interest factor. Το confidence level δεν εξετάζει μόνο αν ένας περιορισμός ικανοποιείται συχνά, αλλά και πόσο συχνά εμφανίζονται συνολικά οι σχετικές δραστηριότητες στο log. Αντίστοιχα, το interest factor αποτιμά πόσο ουσιαστική είναι η σχέση των εμπλεκόμενων δραστηριοτήτων, συνυπολογίζοντας όχι μόνο την υποστήριξη του περιορισμού αλλά και τη γενικότερη παρουσία τους στο σύνολο των traces. Με τον τρόπο αυτό, η αξιολόγηση των constraints δεν βασίζεται μόνο στο αν εμφανίζονται συχνά, αλλά και στο κατά πόσο είναι ουσιαστικά για την περιγραφή της διαδικασίας.

Σύμφωνα με το paper, οι προεπιλεγμένες τιμές των αντίστοιχων thresholds είναι `support=1`, `confidence=0` και `interest factor=0`. Ωστόσο, στην παρούσα υλοποίηση, κατά την κλήση του εργαλείου μέσω κώδικα ορίζεται ρητά μόνο η τιμή του support threshold και επομένως, η πειραματική μεταβολή των αποτελεσμάτων ελέγχεται μέσω του support.

Μετά την ολοκλήρωση της εκτέλεσης, το MINERful παράγει αρχείο `.csv` με τους περιορισμούς που εξορύχθηκαν από το συγκεκριμένο event log. Στην υλοποίηση της παρούσας εργασίας, για κάθε τιμή support δημιουργείται ξεχωριστό αρχείο εξόδου, ενώ παράλληλα καταγράφονται και στοιχεία όπως ο χρόνος και η κατάσταση της εκτέλεσης. Τα αρχεία αυτά χρησιμοποιούνται στη συνέχεια για την αξιολόγηση της ποιότητας του mining και για τη σύγκριση των αποτελεσμάτων μεταξύ διαφορετικών ρυθμίσεων.

Algorithm 6 Εκτέλεση του MINERful για declarative process mining

```
1: classpath ← MINERful.jar + bin + lib/*.jar
2: input_xes ← dataset_log.xes
3: output_csv ← dataset_minerful_output.csv
4: java -cp classpath minerful.MinerFulMinerStarter -iLF input_xes -s 0.9 -oCSV
   output_csv
```

Περιορισμοί και Πρακτικές Παρατηρήσεις για το MINERful

Παρότι το MINERful είναι αποδοτικό και κατάλληλο για declarative process mining, η ποιότητα του αποτελέσματός του εξαρτάται άμεσα από την ποιότητα του event log που του δίνεται ως είσοδος. Αν τα traces δεν έχουν διαμορφωθεί σωστά ή αν οι δραστηριότητες που προέκυψαν από το parsing δεν αποτυπώνουν με συνέπεια τους πραγματικούς τύπους γεγονότων, τότε και οι εξαγόμενοι περιορισμοί είναι πιθανό να είναι θορυβώδεις ή λιγότερο αξιόπιστοι.

Επιπλέον, το support threshold επηρεάζει αισθητά το τελικό σύνολο constraints. Χαμηλότερες τιμές επιτρέπουν την εμφάνιση περισσότερων περιορισμών, αλλά αυξάνουν τον κίνδυνο να περιληφθούν και ασθενέστερα ή λιγότερο ουσιαστικά μοτίβα. Αντίθετα, υψηλότερες τιμές οδηγούν σε πιο αυστηρό αποτέλεσμα, το οποίο είναι συχνά καθαρότερο αλλά μπορεί να αποκλείσει χρήσιμες συμπεριφορές που εμφανίζονται λιγότερο συχνά. Για τον λόγο αυτό, στην παρούσα εργασία το MINERful εκτελείται για διαφορετικές τιμές support, ώστε να μελετάται και η επίδραση της παραμέτρου αυτής.

Τέλος, σε πρακτικό επίπεδο, το MINERful παραμένει ένα εξωτερικό εργαλείο που χρησιμοποιείται στο τελικό στάδιο της υλοποίησης και η αποτελεσματική αξιοποίησή του προϋποθέτει την ύπαρξη ενός σωστά προετοιμασμένου event log από τα προηγούμενα βήματα.

4.4.2 Περιγραφή της Ενσωμάτωσης του MINERful στον κώδικα

Η ενσωμάτωση του MINERful στην υλοποίηση της εργασίας πραγματοποιείται μέσω βοηθητικής συνάρτησης, η οποία παραλαμβάνει το παραγόμενο αρχείο `.xes`, συνθέτει το απαιτούμενο Java classpath και εκτελεί το εργαλείο διάφορες τιμές του support threshold. Για κάθε εκτέλεση δημιουργείται ξεχωριστό αρχείο εξόδου `.csv`, ενώ παράλληλα καταγράφεται και ο χρόνος εκτέλεσης.

Στο κύριο πρόγραμμα, το MINERful καλείται όταν έχει ολοκληρωθεί επιτυχώς η μετατροπή των δομημένων logs σε traces και στη συνέχεια σε `.xes`. Με αυτόν τον τρόπο, το εργαλείο εντάσσεται ως διακριτό αλλά πλήρως συνδεδεμένο στάδιο της συνολικής μεθοδολογίας το οποίο παραλαμβάνει το αποτέλεσμα των προηγούμενων βημάτων και επιστρέφει το τελικό επίπεδο γνώσης, δηλαδή το σύνολο των δηλωτικών περιορισμών που περιγράφουν τη συμπεριφορά του συστήματος.

Για την καλύτερη κατανόηση της ροής του κώδικα υπάρχει ένα παράδειγμα εκτέλεσης

στο παράρτημα Β.

5 Πειραματική Αξιολόγηση

5.1 Πειραματική Διάταξη και Παράμετροι

Η πειραματική αξιολόγηση της προτεινόμενης μεθοδολογίας οργανώνεται βάσει ενός συνόλου παραμέτρων που επηρεάζουν τα επιμέρους στάδια της επεξεργασίας, δηλαδή το log parsing, τη μετατροπή σε αρχεία `.xes` και την εξόρυξη περιορισμών. Η συστηματική μεταβολή των παραμέτρων αυτών επιτρέπει τη διερεύνηση της επίδρασής τους τόσο στα ενδιάμεσα αποτελέσματα όσο και στην τελική ποιότητα της συνολικής ροής επεξεργασίας.

Στο στάδιο του log parsing, οι βασικές παράμετροι που εξετάζονται είναι ο αριθμός των shots, ο αριθμός των in-context παραδειγμάτων που αξιοποιούνται τελικά κατά την εξαγωγή των templates, καθώς και το γλωσσικό μοντέλο που χρησιμοποιείται. Οι τιμές του αριθμού shots που εξετάζονται είναι 8, 16, 32, 64 και 128. Η παράμετρος αυτή αντιστοιχεί στο πλήθος των διαθέσιμων υποψήφιων παραδειγμάτων templates, από τα οποία επιλέγονται τα πιο σχετικά για κάθε νέο log μήνυμα όπως συζητήθηκαν στην ενότητα 4.2, όπως συζητήθηκε στην ενότητα. Ως προς τον αριθμό των παραδειγμάτων που τελικά χρησιμοποιούνται ως in-context examples, εξετάζονται τρεις διαφορετικές περιπτώσεις: η χρήση όλων των διαθέσιμων σχετικών παραδειγμάτων, η μη χρήση παραδειγμάτων, καθώς και η χρήση σταθερού αριθμού παραδειγμάτων. Στην παρούσα εργασία ο σταθερός αυτός αριθμός ορίστηκε σε 8, ώστε να καταστεί δυνατή η σύγκριση ανάμεσα στην πλήρη αξιοποίηση των διαθέσιμων παραδειγμάτων, στην πλήρη απουσία τους και σε μία ενδιάμεση συνθήκη περιορισμένης αξιοποίησής τους. Επιπλέον, η διαδικασία parsing εκτελείται με δύο διαφορετικά γλωσσικά μοντέλα της πλατφόρμας Ollama, τα llama3 και qwen2.5:7b, με στόχο να διερευνηθεί κατά πόσο η επιλογή μοντέλου επηρεάζει την ποιότητα των αποτελεσμάτων.

Στο στάδιο της μετατροπής σε αρχεία `.xes`, βασική παράμετρος αποτελεί το χρονικό παράθυρο εντός του οποίου τα γεγονότα ομαδοποιούνται σε traces. Το μέγεθος αυτού του χρονικού παραθύρου εξαρτάται από την παράμετρο num_of_avg_intervals, η οποία εκφράζει πόσες φορές η επί τη μέση χρονική απόσταση μεταξύ δύο διαδοχικών log μηνυμάτων θα είναι το παράθυρο που χρησιμοποιείται ως βάση για τον σχηματισμό ενός ίχνους. Παράλληλα, εφαρμόζεται ένας πρόσθετος κανόνας σύμφωνα με τον οποίο, όταν η χρονική απόσταση δύο διαδοχικών καταγραφών υπερβαίνει το δεκαπλάσιο της μέσης χρονικής απόστασης, θεωρείται ότι ξεκινά νέο trace. Δεδομένου ότι η δομή των traces καθορίζει άμεσα τη μορφή των παραγόμενων event logs, η συγκεκριμένη παράμετρος επηρεάζει ουσιαστικά και τα επόμενα στάδια της εξόρυξης διαδικασιών.

Τέλος, στο στάδιο του mining εξετάζεται η παράμετρος του κατωφλίου υποστήριξης (support threshold), η οποία καθορίζει το ελάχιστο επίπεδο υποστήριξης που πρέπει να παρουσιάζει ένας υποψήφιος περιορισμός προκειμένου να διατηρηθεί στο τελικό σύνολο των εξαγόμενων constraints. Στην παρούσα εργασία εξετάζονται οι τιμές 0.3, 0.5, 0.7 και 0.9, ώστε να αξιολογηθεί η επίδραση διαφορετικών βαθμών αυστηρότητας στην τελική εξόρυξη περιορισμών. Με τον τρόπο αυτό καθίσταται δυνατή η συστηματική μελέτη της ευαισθησίας των αποτελεσμάτων ως προς τις βασικές παραμέτρους της προτεινόμενης μεθοδολογίας.

5.1.1 Σύνολα Δεδομένων

Για την πειραματική αξιολόγηση της προτεινόμενης μεθοδολογίας αξιοποιήθηκαν σύνολα δεδομένων που χρησιμοποιούνται στη σχετική βιβλιογραφία ως benchmark για την ανάλυση αρχείων καταγραφής [16], [38]. Τα σύνολα αυτά έχουν συλλεχθεί από ετερογενή υπολογιστικά περιβάλλοντα και καλύπτουν διαφορετικούς τύπους συστημάτων και μορφών καταγραφής. Ειδικότερα, εξετάζονται τα Android, Apache, BGL, Hadoop, HDFS, HealthApp, HPC, Linux, Mac, OpenSSH, OpenStack, Proxifier, Spark, Thunderbird, Windows και Zookeeper. Για καθένα από αυτά διατίθενται τόσο εκδόσεις περιορισμένου μεγέθους, της τάξης των 2 χιλιάδων εγγραφών, όσο και μεγαλύτερες εκδόσεις με αρκετά περισσότερα log μηνύματα.

Στο πλαίσιο της παρούσας εργασίας επιλέχθηκαν οι μικρότερες εκδοχές των συνόλων δεδομένων, δηλαδή τα αρχεία των 2 χιλιάδων εγγραφών. Η επιλογή αυτή υπαγορεύθηκε αφενός από τους διαθέσιμους υπολογιστικούς πόρους και αφετέρου από την ανάγκη συστηματικής διερεύνησης πολλαπλών συνδυασμών παραμέτρων και επαναλαμβανόμενων πειραματικών εκτελέσεων υπό συγκρίσιμες συνθήκες.

Συνολικά εξετάστηκαν 16 σύνολα δεδομένων, από τα οποία τα 11 συμπεριλήφθηκαν στην τελική ποσοτική αξιολόγηση. Η προτεινόμενη μεθοδολογία εφαρμόστηκε επιτυχώς στη μεγάλη πλειονότητα των περιπτώσεων, γεγονός που υποδεικνύει ότι μπορεί να υποστηρίξει μια ευρεία κατηγορία ετερογενών μορφών log και να λειτουργήσει ικανοποιητικά σε διαφορετικά περιβάλλοντα καταγραφής. Ωστόσο, 5 σύνολα δεδομένων, τα BGL, HPC, Mac, OpenSSH και Thunderbird, εξαιρέθηκαν από την τελική ανάλυση, του συνόλου της πειραματικής διαδικασίας. Οι δυσκολίες αυτές σχετίζονταν με τη μορφή των χρονικών πεδίων, με αμφίσημη ή μη άμεσα αξιοποιήσιμη χρονική πληροφορία, που οδηγούσε σε μία μη ρεαλιστική εκτίμηση του χρονικού παραθύρου για τη δημιουργία traces. Για τον λόγο αυτό, τα datasets αυτά δεν συμπεριλήφθηκαν στην τελική σύγκριση, ώστε τα αποτελέσματα να βασίζονται αποκλειστικά σε περιπτώσεις όπου η μετατροπή από raw logs σε event logs και η επακόλουθη εξόρυξη διαδικασιών να πραγματοποιείται με τεχνική συνέπεια και επαρκή αξιοπιστία. Επομένως, τα αποτελέσματα της εργασίας υποστηρίζουν ότι η προτεινόμενη προσέγγιση παρουσιάζει ουσιαστική δυνατότητα γενίκευσης σε ένα ευρύ αλλά όχι απεριόριστο φάσμα συνόλων δεδομένων, ενώ παράλληλα αναδεικνύουν τις προκλήσεις που εξακολουθούν να εμφανίζονται σε ιδιαίτερα απαιτητικές μορφές καταγραφών.

5.2 Αξιολόγηση LILAC Parsing

Η αξιολόγηση της απόδοσης του LILAC βασίζεται αρχικά σε τέσσερεις μετρικές, τις F1 Template Accuracy, Parsing Accuracy, F1 Grouping Accuracy και Grouping Accuracy, σύμφωνα με το [37], οι οποίες αποσκοπούν στην αποτύπωση της συμπεριφοράς του parsing των αρχείων καταγραφής σχετικά με την δημιουργία προτύπων και την ικανότητα του συστήματος να ανακτά τη δομή αναφοράς των δεδομένων.

5.2.1 F1 Template Accuracy και Parsing Accuracy

Σε πρώτη φάση εξετάζεται η δημιουργία προτύπων με δύο μετρικές, τις F1 Template Accuracy και Parsing Accuracy.

Η F1 Template Accuracy (FTA) αξιολογεί τα templates που ανακτήθηκαν σωστά αξιοποιώντας το f1 score των Precision Template Accuracy (PTA) και το Recall Template Accuracy (RTA) έτσι ώστε να μην εξετάζει αποκλειστικά πόσα πρότυπα αναφοράς ανακαλύφθηκαν, αλλά να λαμβάνει υπόψη και την ύπαρξη πλεοναζόντων ή λανθασμένων inferred templates.

Πιο συγκεκριμένα, έστω ότι με G συμβολίζεται το σύνολο των ground-truth templates και με I το σύνολο των inferred templates που παράγονται από το μοντέλο. Επίσης, με $|G \cap I|$ συμβολίζεται το πλήθος των προτύπων που ανιχνεύονται ορθά, δηλαδή των κοινών στοιχείων των δύο συνόλων.

Η μετρική Precision Template Accuracy (PTA) ορίζεται ως:

$$\text{Precision Template Accuracy} = \frac{|G \cap I|)}{|I|} \cdot 100 \tag{5.1}$$

και η μετρική Recall Template Accuracy (RTA) ορίζεται ως:

$$\text{Recall Template Accuracy} = \frac{|G \cap I|)}{|G|} \cdot 100 \tag{5.2}$$

Τέλος, η μετρική F1 Template Accuracy (FTA) ορίζεται ως:

$$\text{F1 Template Accuracy} = \frac{2 \cdot RTA \cdot PTA}{RTA + PTA} \cdot 100 \tag{5.3}$$

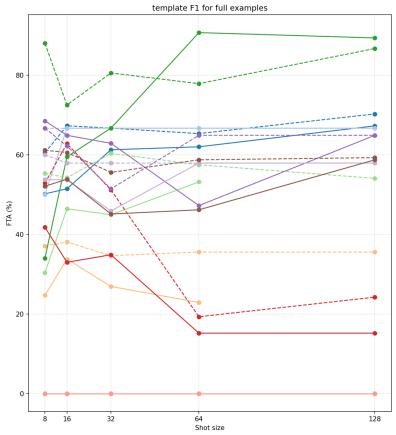
Με τον τρόπο αυτό, η μετρική F1 Template Accuracy αποτιμά την ικανότητα εντοπισμού της πραγματικής δομής των logs, αλλά και τον βαθμό συνοχής του τελικού συνόλου προτύπων.

Όταν οι τιμές της παραπάνω μετρικής είναι υψηλές υποδηλώνουν ότι το παραγόμενο σύνολο templates προσεγγίζει ικανοποιητικά το ground truth, χωρίς σημαντική εισαγωγή περιττών ή λάθος προτύπων. Όταν οι τιμές είναι κοντά στο μηδέν υποδεικνύουν ότι το πλήθος των σωστά αντιστοιχισμένων inferred templates είναι αρκετά μικρό σε σχέση με το πλήθος των πραγματικών template και όλου του συνόλου των infered templates.

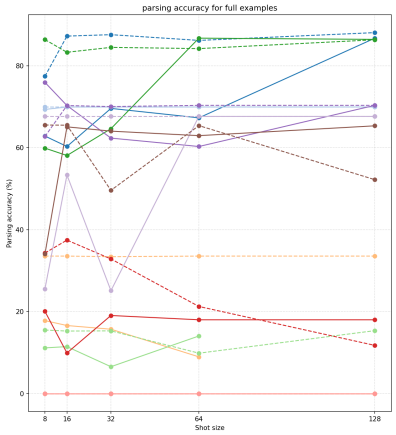
Στην αξιολόγηση συμπεριλαμβάνεται και η μετρική Parsing Accuracy (PA) η οποία είναι μια διαδεδομένη μετρική η οποία χρησιμοποιείται για την αξιολόγηση πολλών αντίστοιχων εργαλείων [19], [21], [36], [20]. Έστω C_m το σύνολο των μηνυμάτων για τα οποία εντοπίζεται σωστά το πρότυπο τους και M το σύνολο των καταγραφών και άρα και των μηνυμάτων. Η δεύτερη μετρική αποτυπώνει το ποσοστό των μηνυμάτων όπου εντοπίζονται επιτυχώς τα πρότυπά τους και ορίζεται ως:

$$\text{Parsing Accuracy} = \frac{|C_m|}{|M|} \cdot 100 \tag{5.4}$$

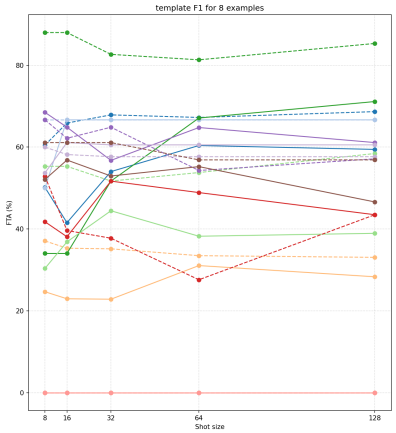
Σε αντίθεση με την προηγούμενη μετρική, αυτή εστιάζει αποκλειστικά στην ικανότητα ανάκτησης των πραγματικών template των μηνυμάτων στο σύνολο τους.



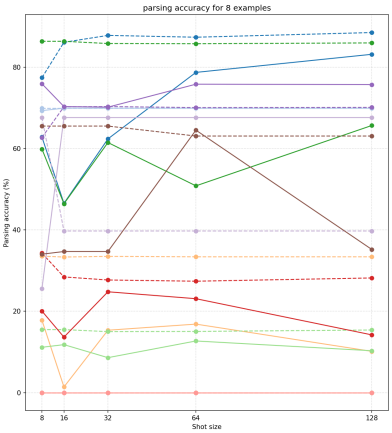
(a) F1 Template Accuracy, full examples



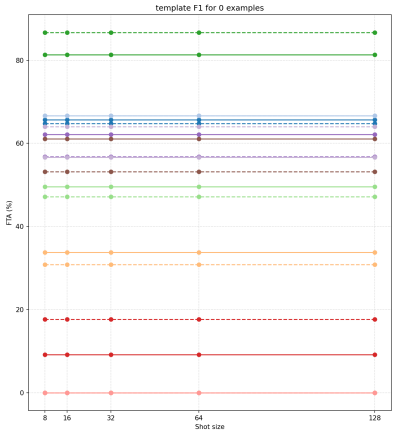
(b) Parsing Accuracy, full examples



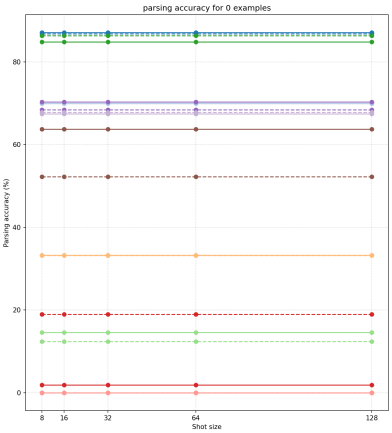
(c) F1 Template Accuracy, 8 examples



(d) Parsing Accuracy, 8 examples



(e) F1 Template Accuracy, 0 examples



(f) Parsing Accuracy, 0 examples



Figure 5.1: Αποτελέσματα αξιολόγησης του LILAC parsing για διαφορετικό αριθμό παρεχόμενων παραδειγμάτων μαζί με το υπόμνημα χρωμάτων και τύπων γραμμής(FTA & PA).

Στο Σχήμα 5.1 παρουσιάζονται τα αποτελέσματα για αυτές τις 2 μετρικές, όπου κάθε γραμμή αποτελείται από τις 2 αυτές μετρικές για κάθε αριθμό παραδειγμάτων. Φαίνεται αρχικά ότι η εναλλαγή του μοντέλου δεν οδηγεί σταθερά σε καλύτερη ή χειρότερη επίδοση και ότι εξαρτάται και από το επιλεγμένο γλωσσικό μοντέλο, τον αριθμό των shots και των παραδειγμάτων.

Η επίδραση του shot size εμφανίζει συνήθως μεγαλύτερες διακυμάνσεις στις μικρές τιμές, ενώ για μεγαλύτερα shots η απόδοση τείνει να σταθεροποιείται ή να βελτιώνεται ελαφρά, με ελάχιστες μόνο εξαιρέσεις για τα σύνολα δεδομένων OpenStack και Zookeeper για κάποιους συνδυασμούς παραμέτρων. Αυτό υποδηλώνει ότι, μετά από ένα σημείο, η περαιτέρω αύξηση των shots δεν αλλάζει ριζικά τη συμπεριφορά του parsing, αλλά λειτουργεί περισσότερο ως μηχανισμός σταδιακής εξομάλυνσης.

Το parsing εμφανίζει σχετικά αντίστοιχη επίδοση όταν δεν παρέχονται παραδείγματα για τα περισσότερα datasets κάτι που δείχνει ότι η μέθοδος είναι λειτουργική ακόμα και όταν δεν υπάρχουν διαθέσιμα παραδείγματα, αφού το μοντέλο καλείται να αναγνωρίσει τα templates χωρίς επαρκές παραδειγματικό πλαίσιο.

Τα σύνολα δεδομένων για τα οποία το parsing φαίνεται να λειτουργεί καλύτερα έχουν μεγαλύτερη απόδοση όταν χρησιμοποιούνται όλα τα παραδείγματα, όμως αυτό φαίνεται ότι επηρεάζεται και από τις υπόλοιπες παραμέτρους.

Αξίζει επίσης να σημειωθεί ότι σε ορισμένες περιπτώσεις δεν υπολογίζεται το σημείο 128 examples και 128 shots, συγκεκριμένα για τα Linux και Hadoop, επειδή το LLM δεν καταφέρνει να ολοκληρώσει σωστά το parsing και δεν παράγεται έγκυρο αποτέλεσμα προς αξιολόγηση. Ένα επιπλέον στοιχείο που προκύπτει από το σχήμα είναι ότι οι αποκλίσεις ανάμεσα στις καμπύλες της FTA και του Parsing Accuracy δείχνουν πως η σωστή ανάκτηση των templates ως συνόλο δεν ταυτίζεται πάντοτε με την ακριβή αντιστοίχιση κάθε μεμονωμένου μηνύματος, άρα η δυσκολία του parsing αποτυπώνεται με διαφορετικό τρόπο στις δύο μετρικές.

Τέλος, το σύνολο δεδομένων HDFS δεν εμφανίζεται στο διάγραμμα του Parsing Accuracy διότι ενώ καταφέρνει να εντοπίσει σωστά κάποια templates αυτά δεν ταυτίζονται πλήρως με τα δεδομένα, όπως για παράδειγμα εντοπίζεται το template "PacketResponder <*> for block <*> terminating" για το δοσμένο "<PacketResponder <*> for block blk_<*> terminating>".

Για την καλύτερη λοιπόν αξιολόγηση των αποτελεσμάτων στην επόμενη ενότητα προτείνεται μία συμπληρωματική μετρική που σχετίζεται με την ακρίβεια χαλαρώνοντας λίγο την προϋπόθεση για ακριβή ταύτιση των προτύπων των μηνυμάτων.

Soft Parsing Accuracy

Αρχικά ορίζεται ένα νέο σύνολο, το C_{ms} το οποίο αφορά τα μηνύματα που έχουν αναλυθεί και η ομοιότητα των προτύπων τους με τα πραγματικά είναι 0.95 χρησιμοποιώντας τον αλγόριθμο SequenceMatcher. Η νέα μετρική θα είναι:

$$\text{Soft Parsing Accuracy} = \frac{|C_{ms}|}{|M|} \cdot 100 \tag{5.5}$$

Η συγκεκριμένη μετρική δεν απαιτεί ακριβή ταύτιση μεταξύ ground-truth και inferred templates, αλλά αποτιμά τον βαθμό εγγύτητας μεταξύ τους. Για τον λόγο αυτό λειτουργεί συμπληρωματικά προς τις αυστηρότερες μετρικές FTA και Parsing Accuracy. Ειδικότερα, μπορεί να αναδείξει περιπτώσεις όπου το σύστημα αποτυγχάνει να αναπαράγει ακριβώς ένα template, αλλά εξακολουθεί να εντοπίζει σε μεγάλο βαθμό τη δομή του και την ουσία του. Έτσι, η ανάλυση της τιμής της Soft Parsing Accuracy επιτρέπει μια πιο λεπτομερή αξιολόγηση της ποιότητας του parsing, ιδίως όταν υπάρχουν μικρές λεκτικές ή μορφολογικές αποκλίσεις μεταξύ των παραγόμενων και των πραγματικών templates. Επομένως, οι υψηλότερες τιμές της μετρικής υποδηλώνουν μεγαλύτερη συνολική ομοιότητα μεταξύ των inferred και των ground-truth templates.

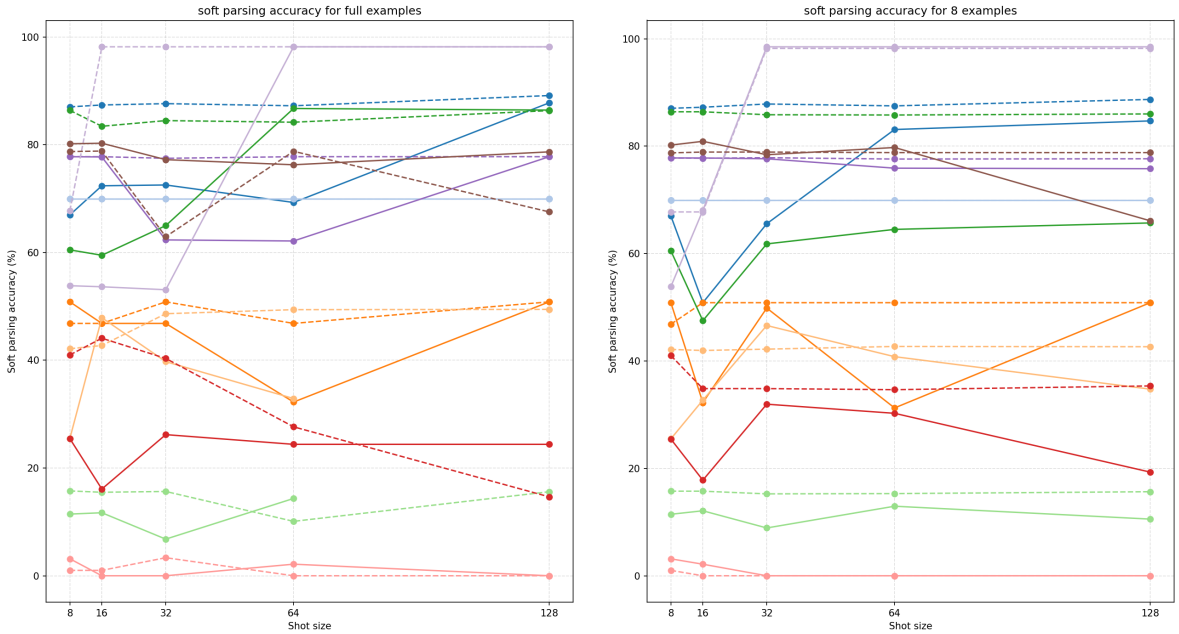
Στο Σχήμα 5.2 παρουσιάζονται οι τιμές της μετρικής για τον διάφορο αριθμό παραδειγμάτων, shot και μοντέλων. Εμφανίζονται γενικά υψηλότερες σε σχέση με τις αυστηρότερες μετρικές του προηγούμενου Σχήματος 5.1, κάτι που είναι αναμενόμενο επειδή δεν απαιτείται ακριβής αναπαραγωγή ολόκληρου του template, αλλά αξιολογείται αν το παραγόμενο template παραμένει λεκτικά και δομικά κοντά στο ground truth. Με άλλα λόγια, ακόμη και όταν το μοντέλο δεν εντοπίζει πλήρως το αμετάβλητο μέρος ενός template, μπορεί να αναπαράγει σωστά το μεταβλητό ή πιο εμφανές τμήμα του και έτσι να διατηρείται σχετικά υψηλή τιμή ομοιότητας. Αυτό αναδεικνύει και μία ουσιαστική αδυναμία του parsing: σε αρκετές περιπτώσεις το LLM φαίνεται να προσεγγίζει σωστά το δυναμικό μέρος του μηνύματος, αλλά να μην απομονώνει με την ίδια επιτυχία το σταθερό, μη μεταβλητό μέρος που τελικά καθορίζει το template με αυστηρό τρόπο. Παρ' όλα αυτά, το γεγονός ότι η μετρική παραμένει υψηλή είναι επίσης θετικό, διότι δείχνει ότι ακόμη και όταν δεν επιτυγχάνεται ακριβής ταύτιση, το σύστημα συχνά συλλαμβάνει τη γενική μορφή και τη σημασιολογική δομή των μηνυμάτων. Έτσι, η Soft Parsing Accuracy δείχνει ότι η αποτυχία του parsing δεν είναι πάντοτε πλήρης, αλλά σε αρκετές περιπτώσεις περιορίζεται σε μερική αστοχία ως προς την ακριβή οριοθέτηση του τελικού template.

5.2.2 F1 Grouping Accuracy και Grouping Accuracy

Σε δεύτερη φάση εξετάζεται η ομαδοποίηση των προτύπων σε σχέση με τα μηνύματα. Οι μετρικές αυτές λαμβάνουν υπόψη ποια μηνύματα ανήκουν στο ίδιο πρότυπο και το θεωρούν ως μία εννίαια ομάδα. Στη συνέχεια, αξιολογούν σε ποιο βαθμό κάθε πρότυπο αποδίδεται πραγματικά σε όλα τα μηνύματα, χωρίς να περιλαμβάνει λιγότερα ή περισσότερα από όσα θα έπρεπε.

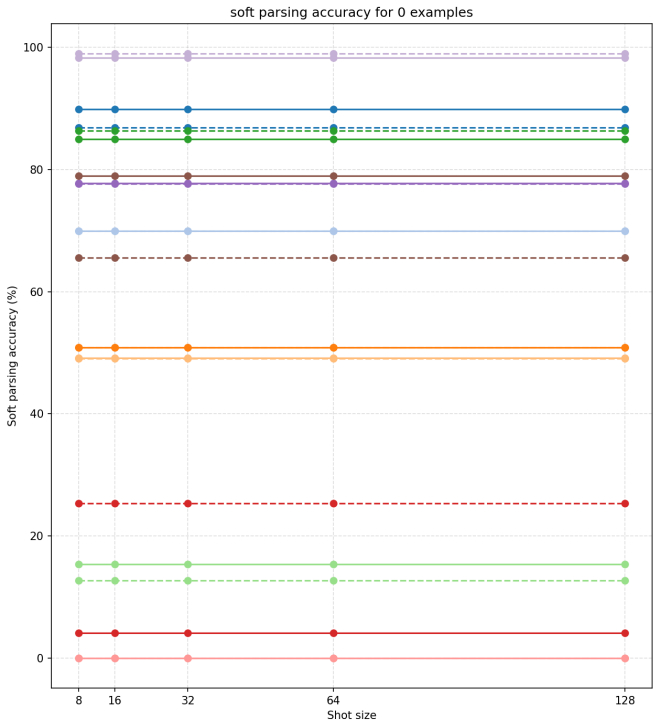
Η F1 Grouping Accuracy (FGA) αξιολογεί τις ομαδοποιήσεις προτύπων που ανακτήθηκαν σωστά αξιοποιώντας τον αρμονικό μέσο των Precision Grouping Accuracy (PGA) και Recall Grouping Accuracy (RGA), εξισορροπώντας έτσι την ικανότητα του συστήματος να παράγει σωστές ομάδες χωρίς να εισάγει λανθασμένες ή ελλιπείς ομαδοποιήσεις. Σε αντίθεση με τις μετρικές που εξετάζουν μόνο αν το παραγόμενο template είναι λεκτικά κοντά στο ground truth, εδώ αξιολογείται αν όλα τα μηνύματα που ανήκουν πραγματικά στην ίδια ομάδα αποδίδονται μαζί μόνο αυτά.

Αναλυτικότερα, εάν για κάθε μήνυμα που ανήκει σε μία ομάδα του G ισχύει ότι, μέσω της διαδικασίας parsing, αυτά τα μηνύματα και μόνο αυτά έχουν αντιστοιχηθεί σε μία κοινή



(a) Full examples

(b) 8 examples



(c) 0 examples



Figure 5.2: Τιμές της μετρικής Soft Parsing Accuracy για τα εξεταζόμενα datasets και μοντέλα, μαζί με το υπόμνημα χρωμάτων και τύπων γραμμής.

ομάδα/πρότυπο του I, τότε η συγκεκριμένη ομάδα θεωρείται ότι έχει ανακτηθεί σωστά.

Έστω λοιπόν ότι με C_g συμβολίζεται το πλήθος των ομάδων που έχουν ανακτηθεί σωστά. Με βάση αυτό, η μετρική Precision Grouping Accuracy (PGA) ορίζεται ως:

$$\text{Precision Grouping Accuracy} = \frac{|C_g|}{|I|} \cdot 100 \tag{5.6}$$

και η μετρική Recall Grouping Accuracy (RGA) ορίζεται ως:

$$\text{Recall Grouping Accuracy} = \frac{|C_g|}{|G|} \cdot 100 \tag{5.7}$$

Τέλος, η μετρική F1 Grouping Accuracy (FGA) ορίζεται ως:

$$\text{F1 Grouping Accuracy} = \frac{2 \cdot PGA \cdot RGA}{PGA + RGA} \cdot 100 \tag{5.8}$$

Με τον τρόπο αυτό, η FGA αποτιμά κατά πόσο το σύστημα ανακτά ορθά τη συνολική δομή των ομάδων templates, χωρίς να ανταμείβει περιπτώσεις όπου ένα ground-truth template διασπάται σε πολλές inferred ομάδες ή όπου διαφορετικά ground-truth templates συγχωνεύονται λανθασμένα σε μία μόνο ομάδα.

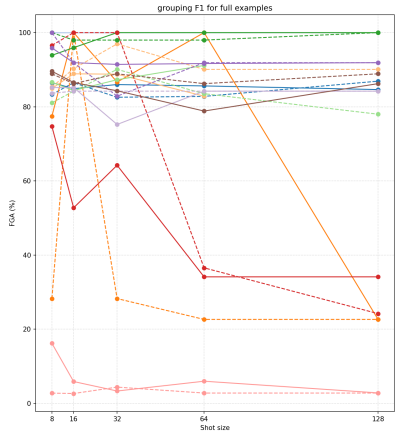
Συμπληρωματικά, χρησιμοποιείται και η μετρική Grouping Accuracy (GA), η οποία εξετάζει την ορθότητα της ομαδοποίησης σε επίπεδο μηνυμάτων. Έστω $C_{m-grouped}$ το σύνολο των μηνυμάτων που ανήκουν σε ομάδες οι οποίες έχουν ανακτηθεί πλήρως και σωστά. Τότε η μετρική Grouping Accuracy ορίζεται ως:

$$\text{Grouping Accuracy} = \frac{|C_{m-grouped}|}{|M|} \cdot 100 \tag{5.9}$$

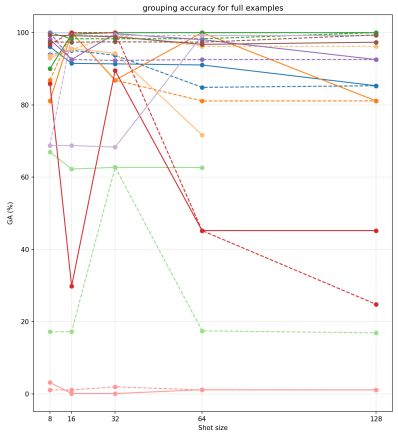
Η διαφορά των δύο μετρικών αυτών είναι ότι η FGA δίνει έμφαση στον αριθμό των σωστών ομάδων ως δομικών οντοτήτων, ενώ η GA αποτυπώνει το ποσοστό των επιμέρους μηνυμάτων που τελικά τοποθετούνται μέσα σε απολύτως σωστές ομάδες.

το Σχήμα 5.3 φαίνονται οι τιμές των μετρικών F1 Grouping Accuracy και Grouping Accuracy για τους εξεταζόμενους συνδυασμούς παραμέτρων. Τα αποτελέσματα δείχνουν ότι, στην πλειονότητα των datasets, το LILAC επιτυγχάνει υψηλή απόδοση ως προς την ομαδοποίηση των μηνυμάτων σε πρότυπα. Ειδικότερα, σε όλα τα σύνολα δεδομένων εκτός του Proxifier παρατηρείται τουλάχιστον μία ρύθμιση παραμέτρων που οδηγεί σε τιμές πάνω από 80% για την μετρική F1 Grouping Accuracy και άνω του 60% για την Grouping Accuracy. Παράλληλα, datasets όπως τα HealthApp, Spark, Windows και Zookeeper εμφανίζουν γενικά πιο σταθερά υψηλές επιδόσεις, ενώ άλλα, όπως τα HDFS, Hadoop και OpenStack, παρουσιάζουν μεγαλύτερη ευαισθησία στις παραμέτρους.

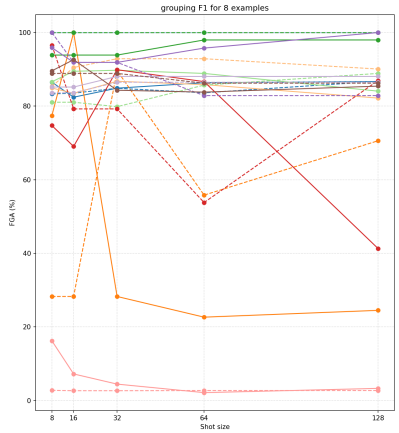
Η εικόνα αυτή, σε συνδυασμό με τις προηγούμενες μετρικές F1 Template Accuracy και Parsing Accuracy, δείχνει ότι παρατηρούνται περιπτώσεις στις οποίες και οι τέσσερις μετρικές λαμβάνουν υψηλές τιμές και, επομένως, το σύστημα επιτυγχάνει πολύ καλή συνολική απόδοση, καθώς ανακτά σωστά τα templates και αντιστοιχίζει σωστά τα μηνύματα στις ορθές ομάδες. Επομένως, οι υψηλές τιμές στις FGA και GA ενισχύουν την ερμηνεία ότι η καλή συμπεριφορά του parsing επεκτείνεται και στη σωστή δομική οργάνωση των δεδομένων.



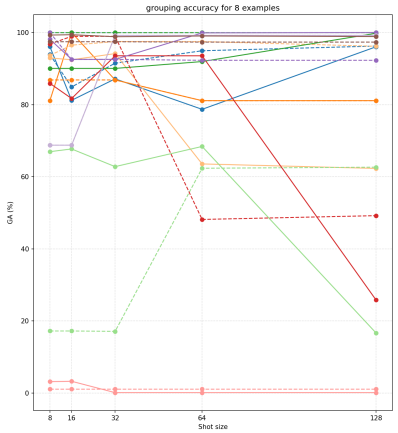
(a) F1 Grouping Accuracy, full examples



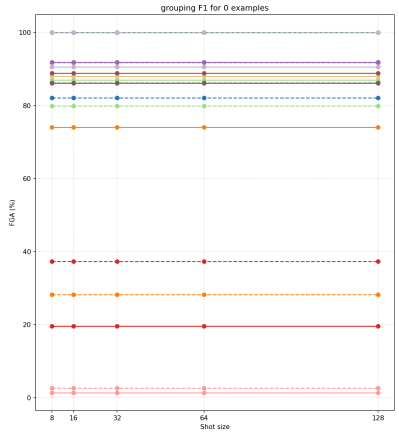
(b) Grouping Accuracy, full examples



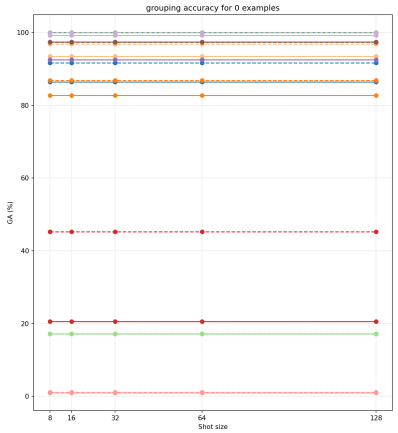
(c) F1 Grouping Accuracy, 8 examples



(d) Grouping Accuracy, 8 examples



(e) F1 Grouping Accuracy, 0 examples



(f) Grouping Accuracy, 0 examples



Figure 5.3: Αποτελέσματα αξιολόγησης του LILAC parsing για διαφορετικό αριθμό παρεχόμενων παραδειγμάτων μαζί με το υπόμνημα χρωμάτων και τύπων γραμμής(FGA & GA).

5.2.3 Βέλτιστος συνδυασμός παραμέτρων

Πέρα από την παρουσίαση των τιμών κάθε μετρικής ανά dataset, πραγματοποιείται και μία συγκεντρωτική ανάλυση των βέλτιστων συνδυασμών παραμέτρων. Πιο συγκεκριμένα, για κάθε σύνολο δεδομένων και για κάθε μετρική επιλέγεται ο συνδυασμός (examples, shot, model) που επιτυγχάνει τη μέγιστη τιμή της αντίστοιχης μετρικής. Με τον τρόπο αυτό, εντοπίζεται ο βέλτιστος συνδυασμός παραμέτρων που αντιστοιχεί σε κάθε ένα από αυτά.

Στη συνέχεια, εξετάζεται η συχνότητα εμφάνισης των βέλτιστων αυτών συνδυασμών στο σύνολο των datasets, ώστε να προσδιοριστεί ο majority best combination, δηλαδή ο συνδυασμός που εμφανίζεται τις περισσότερες φορές ως βέλτιστος για κάθε σύνολο δεδομένων. Η διαδικασία αυτή εφαρμόζεται ξεχωριστά για κάθε μετρική, καθώς διαφορετικές μετρικές ενδέχεται να αναδεικνύουν διαφορετικούς βέλτιστους συνδυασμούς. Επομένως, ο majority best combination ερμηνεύεται ως ο συνδυασμός που επικρατεί συχνότερα υπό το πρίσμα της συγκεκριμένης μετρικής. Τέλος, μετά την αποσπασματική αξιολόγηση ανά dataset, γίνεται μία συνολική εικόνα για το ποια ρύθμιση παρουσιάζει τη σταθερότερη και συχνότερα βέλτιστη συμπεριφορά.

Dataset	Best value	Examples	Shot	Model
Android	70.23809523809524	full	128	qwen2.5:7b
Apache	66.66666666666666	full	128	qwen2.5:7b
HDFS	0	full	128	qwen2.5:7b
Hadoop	38.05309734513274	full	16	qwen2.5:7b
HealthApp	90.66666666666667	full	64	llama3
Linux	60.33057851	full	32	qwen2.5:7b
OpenStack	62.79069767	full	16	qwen2.5:7b
Proxifier	0	full	128	qwen2.5:7b
Spark	68.49315068	full	8	llama3
Windows	64	0	8	qwen2.5:7b
Zookeeper	61.111111111111114	full	8	qwen2.5:7b

Table 5.1: Βέλτιστος συνδυασμός ανά dataset ως προς το F1 Template Accuracy.

Ξεκινώντας με τη μετρική F1 Template Accuracy, ο Πίνακας 5.1 δείχνει ότι οι καλύτερες τιμές εμφανίζονται συχνότερα με full examples και αρκετές φορές με το qwen2.5:b, ιδιαίτερα σε datasets όπως τα HealthApp, Spark και Android. Τα shots που είναι βέλτιστα δεν αντιστοιχούν σε μία μόνο τιμή, αλλά σε πολλές περιπτώσεις αντιστοιχεί στη μέγιστη τιμή, 128, γεγονός που δείχνει ότι η συγκεκριμένη μετρική επηρεάζεται από τα διαθέσιμα παράδειγματα. Υπάρχουν ακόμη και πιο ασταθείς περιπτώσεις, όπως τα HDFS και Proxifier, όπου οι βέλτιστες τιμές παραμένουν μηδενικές, κάτι που υποδηλώνει εγγενή δυσκολία του parsing στα συγκεκριμένα datasets. Όσον αφορά τα παραδείγματα φαίνεται ότι όταν αξιοποιούνται όλα τα διαθέσιμα, σχεδόν σε όλες τις περιπτώσεις, προκύπτει η καλύτερη περίπτωση.

Στον Πίνακα του Parsing Accuracy 5.2 παραμένει η παρουσία του qwen2.5:7b και

Dataset	Best value	Examples	Shot	Model
Android	88.55	8	128	qwen2.5:7b
Apache	69.95	full	128	qwen2.5:7b
HDFS	0	full	128	qwen2.5:7b
Hadoop	33.55	full	128	qwen2.5:7b
HealthApp	86.75	full	64	llama3
Linux	15.5	full	8	qwen2.5:7b
OpenStack	37.45	full	16	qwen2.5:7b
Proxifier	0	full	128	qwen2.5:7b
Spark	75.95	full	8	llama3
Windows	67.85	0	8	qwen2.5:7b
Zookeeper	65.55	full	8	qwen2.5:7b

Table 5.2: Βέλτιστος συνδυασμός ανά dataset ως προς το Parsing Accuracy.

η τάση προς το μεγαλύτερο shot size, στα 128 και με χρήση όλων των διαθέσιμων παραδειγμάτων. Αυτό υποδηλώνει ότι η συγκεκριμένη μετρική ευνοείται περισσότερο από ρυθμίσεις που ενισχύουν την ανάκτηση των πραγματικών templates, ακόμη και αν αυτό δεν συνεπάγεται πάντα την καλύτερη συνολική ποιότητα parsing. Παρ' όλα αυτά, η συμπεριφορά δεν είναι ομοιόμορφη για όλα τα datasets, καθώς σε ορισμένες περιπτώσεις, όπως τα HealthApp και Spark, το llama3 παραμένει βέλτιστο, ενώ σε άλλα datasets, όπως τα HDFS και Proxifier, οι τιμές δείχνουν ότι η ανάκτηση του ground truth παραμένει ιδιαίτερα δύσκολη και τα templates δεν καταφέρνουν να ανακτηθούν με την ακριβή πραγματική μορφή τους, όπως αναφέρεται και στο κεφάλαιο 5.2.1. Η δυσκολία αυτή φαίνεται να μην υπάρχει στην επόμενη μετρική την Soft Parsing Accuracy πράγμα που δείχνει ότι παρά την μηδενική τιμή της Parsing Accuracy το εργαλείο καταφέρνει ακόμα να εντοπίσει σωστά τις σχέσεις και για αυτά τα σύνολα.

Για την μετρική Soft Parsing Accuracy, οι βέλτιστοι συνδυασμοί εμφανίζονται στον Πίνακα 5.3 όπου παρατηρείται ότι υπερισχύει και πάλι η χρήση των full examples, ενώ τα shots συγκεντρώνονται περισσότερο σε υψηλότερες τιμές όπως 128, με κάποια σύνολα δεδομένων να εμφανίζουν καλύτερα αποτελέσματα για μικρότερη τιμή shot. Η εικόνα αυτή είναι συμβατή με το γεγονός ότι η μετρική είναι πιο ανεκτική από τις αυστηρές exact-match μετρικές και επομένως αναδεικνύει ρυθμίσεις όπου το μοντέλο συλλαμβάνει σωστά τη γενική μορφή των templates ακόμη και όταν δεν τα αναπαράγει ακριβώς. Επιπλέον, το μοντέλο qwen2.5:7b φαίνεται να είναι το βέλτιστο μοντέλο στην πλειοψηφία των περιπτώσεων. Εδώ το HDFS παρουσιάζει καλύτερα αποτελέσματα πράγμα που δείχνει ότι εν μέρει το parsing ανακτά το αντίστοιχο template. Αντίθετα, στο σύνολο δεδομένων Proxifier, ακόμη και αυτή η πιο επιεικής μετρική εμφανίζει χαμηλές τιμές, υποδεικνύοντας περιορισμένη επιτυχία του parsing.

Για τη μετρική F1 Grouping Accuracy, ο Πίνακας 5.4 δείχνει τους συνδυασμούς παραμέτρων που οδηγούν στη βέλτιστη ανάκτηση της δομής των ομάδων. Οι υψηλές τιμές της συγκεκριμένης μετρικής υποδηλώνουν ότι όντως τα μηνύματα που ανήκουν στην

Dataset	Best value	Examples	Shot	Model
Android	89.9	0	128	llama3
Apache	69.95	full	128	qwen2.5:7b
HDFS	50.849999999999994	full	128	qwen2.5:7b
Hadoop	49.45	full	128	qwen2.5:7b
HealthApp	86.75	full	64	llama3
Linux	15.75	full	8	qwen2.5:7b
OpenStack	44.1	full	16	qwen2.5:7b
Proxifier	3.35	full	32	qwen2.5:7b
Spark	77.85	8	32	qwen2.5:7b
Windows	99	0	8	qwen2.5:7b
Zookeeper	80.9	8	16	llama3

Table 5.3: Βέλτιστος συνδυασμός ανά dataset ως προς το Soft Parsing Accuracy.

ίδια πραγματική ομάδα τείνουν να ανακτώνται μαζί, χωρίς σημαντικές συγχωνεύσεις ή διασπάσεις ομάδων. Σε αυτήν την περίπτωση το σύνολο δεδομένων HDFS πετυχαίνει 100% f1-score, το οποίο δείχνει ότι το parsing αποδίδει ιδιαίτερα καλά. Το σύνολο δεδομένων Proxifier εξακολουθείνα εμφανίζει χαμηλές τιμές, ενώ για τα υπόλοιπα σύνολα δεδομένων προκύπτουν γενικά ικανοποιητικά αποτελέσματα. Οι βέλτιστες παράμετροι συνεχίζουν να κινούνται σε παρόμοιο πλαίσιο με τους προηγούμενους πίνακες.

Dataset	Best value	Examples	Shot	Model
Android	86.9047619	full	128	qwen2.5:7b
Apache	100	full	128	qwen2.5:7b
HDFS	100	full	64	llama3
Hadoop	96.96969697	full	32	qwen2.5:7b
HealthApp	100	full	128	qwen2.5:7b
Linux	91.13924051	full	64	llama3
OpenStack	100	full	32	qwen2.5:7b
Proxifier	16.21621622	full	8	llama3
Spark	100	full	8	qwen2.5:7b
Windows	100	0	8	qwen2.5:7b
Zookeeper	92.63157895	8	16	llama3

Table 5.4: Βέλτιστος συνδυασμός ανά dataset ως προς το F1 Grouping Accuracy.

Για τη μετρική Grouping Accuracy, ο Πίνακας 5.5 αποτυπώνει τους συνδυασμούς που οδηγούν στην ακριβέστερη αντιστοίχιση των επιμέρους μηνυμάτων στις σωστές ομάδες. Η συγκεκριμένη μετρική εστιάζει περισσότερο στο επίπεδο των μηνυμάτων και επομένως δείχνει πότε το parsing τοποθετεί τα ίδια τα μηνύματα στις αντίστοιχες πραγματικές ομάδες με μεγαλύτερη ακρίβεια. Με αυτόν τον τρόπο, οι βέλτιστοι συνδυασμοί που προκύπτουν

για τη μετρική Grouping Accuracy συμπληρώνουν την εικόνα της μετρικής F1 Grouping Accuracy και ενισχύουν την αποτίμηση της ποιότητας της δομικής οργάνωσης των δεδομένων. Συνολικά, τα περισσότερα σύνολα δεδομένων διατηρούν την ικανοποιητική και σχετικά σταθερή συμπεριφορά και ως προς αυτή τη μετρική. Εξαίρεση αποτελούν το Linux, το οποίο εμφανίζει πιο μέτρια απόδοση, και το Proxifier, το οποίο παρουσιάζει σταθερά τις χαμηλότερες επιδόσεις.

Dataset	Best value	Examples	Shot	Model
Android	96.25	8	128	qwen2.5:7b
Apache	100	full	128	qwen2.5:7b
HDFS	100	full	64	llama3
Hadoop	99.25	full	32	qwen2.5:7b
HealthApp	100	full	128	qwen2.5:7b
Linux	68.4	8	64	llama3
OpenStack	100	full	32	qwen2.5:7b
Proxifier	3.25	8	16	llama3
Spark	100	full	8	qwen2.5:7b
Windows	100	0	8	qwen2.5:7b
Zookeeper	99.4	8	16	llama3

Table 5.5: Βέλτιστος συνδυασμός ανά dataset ως προς το Grouping Accuracy.

Τελικά, ο Πίνακας 5.6 συνοψίζει ότι προκύπτει ένας ενιαίος συνδυασμός παραμέτρων που να κυριαρχεί σε όλες τις parsing μετρικές και αυτός παρατηρείται ότι είναι η χρήση full examples από τα μέγιστα δυνατά υποψήφια, 128, ,ε το μοντέλο qwen2.5:7b. Η συνθήκη αυτή ενισχύει τη διαπίστωση ότι η ύπαρξη πλουσιότερου παραδειγματικού πλαισίου βοηθά σταθερά την απόδοση και ότι τα μικρότερα shot sizes δεν επαρκούν συνήθως για σταθερά καλή συμπεριφορά. Βέβαια, αυτές οι παράμετροι δεν προκύπτουν συνολικά για όλα τα σύνολα δεδομένων. Η εναλλαγή των παραμέτρων στα αποτελέσματα δείχνει ότι η απόδοση του συστήματος εξαρτάται αρκετά από τα ιδιαίτερα χαρακτηριστικά κάθε dataset και δεν καθορίζεται μόνο από μία καθολικά καλύτερη επιλογή παραμέτρων, με κάποιες τιμές να αναδεικνύονται σχετικά καλύτερες.

Metric	Examples	Shot	Model	Supporting datasets
F1 Template Accuracy	full	128	qwen2.5:7b	4/11
Parsing Accuracy	full	128	qwen2.5:7b	4/11
Soft Parsing Accuracy	full	128	qwen2.5:7b	3/11
F1 Grouping Accuracy	full	128	qwen2.5:7b	3/11
Grouping Accuracy	full	128	qwen2.5:7b	2/11

Table 5.6: Επικρατέστεροι βέλτιστοι συνδυασμοί παραμέτρων ανά parsing μετρική.

5.3 Αξιολόγηση Δημιουργίας Ιχνών για τα Αρχεία XES

Η αξιολόγηση της διαδικασίας κατασκευής traces γίνεται κυρίως για να παρατηρηθεί πώς σχηματίζονται τα traces υπό διαφορετικές τιμές της παραμέτρου avgIntervals. Ο αριθμός των παραγόμενων traces από μόνος του δεν αρκεί για αυτήν την αποτίμηση, καθώς επηρεάζεται άμεσα από το μέγεθος του χρονικού παραθύρου. Αντιθέτως, μεγαλύτερη σημασία έχει αν η τμηματοποίηση οδηγεί σε traces με μήκος συμβατό προς την επιδιωκόμενη κλίμακα και αν αυτή η συμπεριφορά παραμένει σχετικά σταθερή ανά dataset. Για τον λόγο αυτό, η ανάλυση βασίζεται κυρίως στο μέσο μήκος των traces και στη διασπορά των μηκών τους, ώστε να εξεταστεί αν τα παραγόμενα traces ακολουθούν συστηματικά τη μεταβολή της παραμέτρου avgIntervals ή αν εμφανίζουν έντονη αστάθεια.

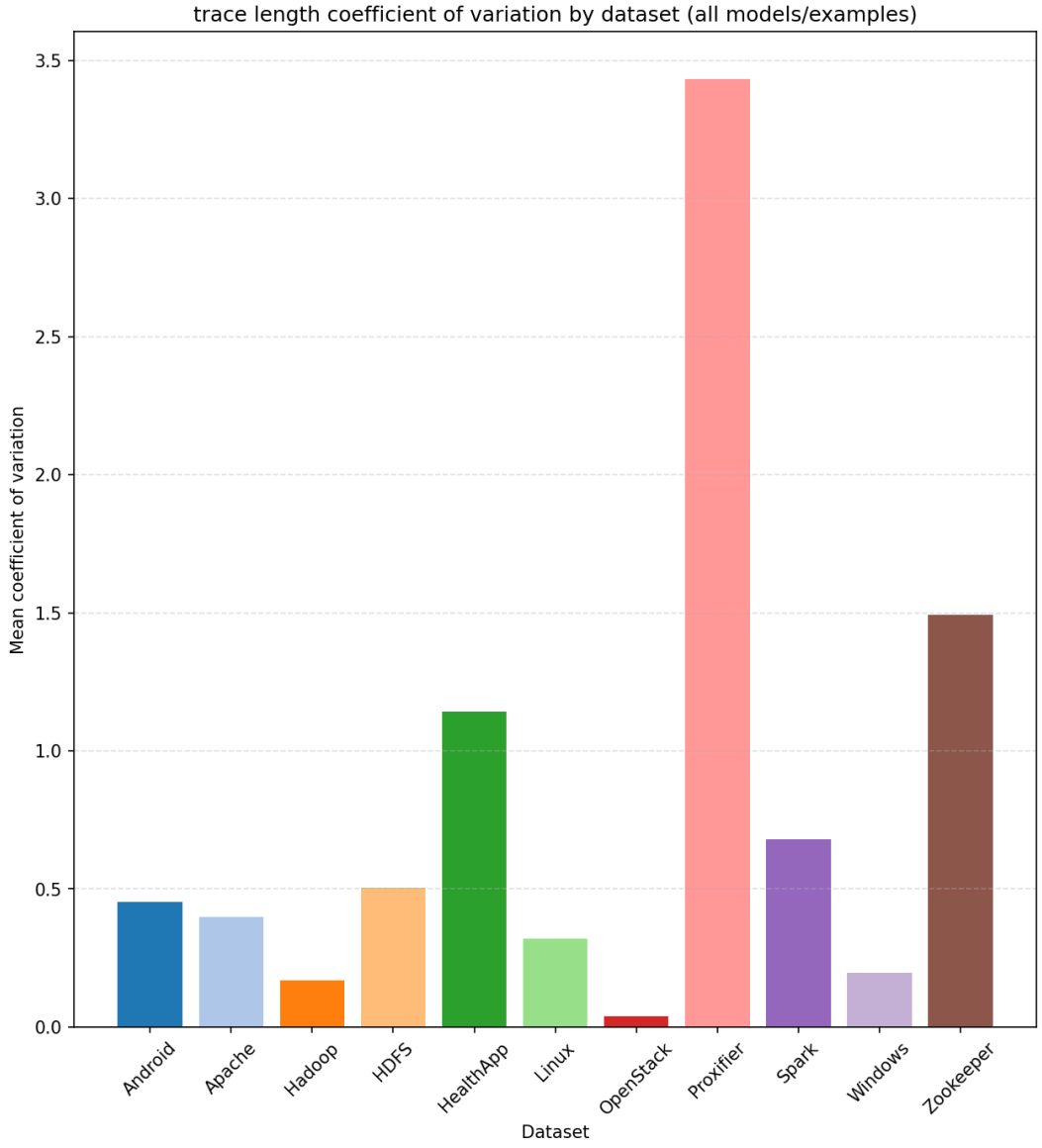


Figure 5.4: Μέσος συντελεστής μεταβλητότητας του μήκους των traces ανά dataset.

Το Σχήμα 5.4 δείχνει πόσο σταθερή είναι η διαδικασία τμηματοποίησης ανά dataset με βάση τον συντελεστή μεταβλητότητας του μήκους των traces. Χαμηλότερες τιμές σημαίνουν ότι τα μήκη των traces συγκεντρώνονται πιο κοντά σε μία αντιπροσωπευτική τιμή, άρα η τμηματοποίηση είναι πιο συνεπής. Αυτό φαίνεται ιδιαίτερα στα OpenStack, Hadoop και Windows, όπου η διασπορά παραμένει σχετικά περιορισμένη. Αντίθετα, τα Proxifier, Zookeeper και HealthApp εμφανίζουν πολύ υψηλότερη μεταβλητότητα, γεγονός που δείχνει ότι η ίδια τιμή της παραμέτρου avgIntervals μπορεί να οδηγεί σε traces με πολύ διαφορετικά μήκη.

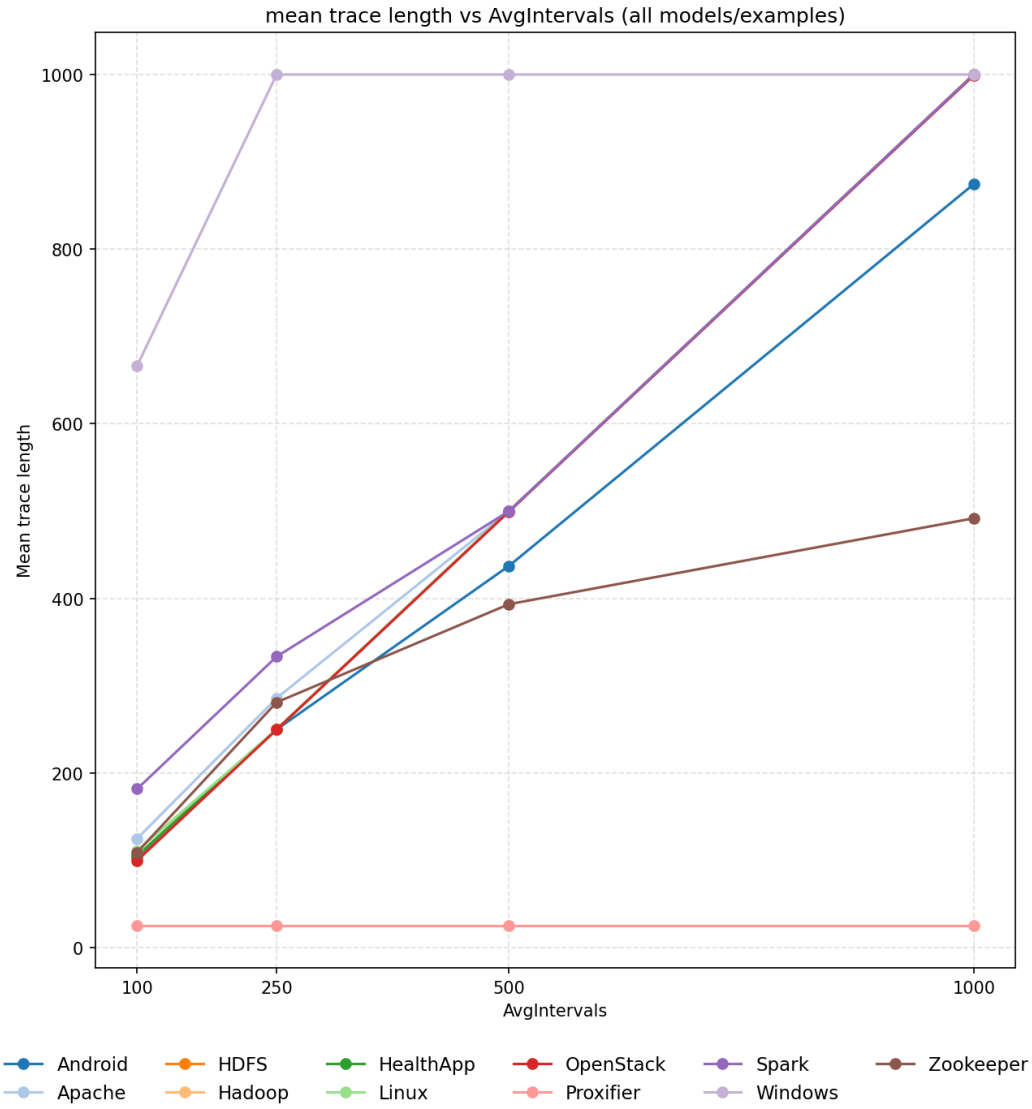


Figure 5.5: Μέσο μήκος traces ανά dataset ως προς την παράμετρο avgIntervals, συγκεντρωτικά για όλα τα μοντέλα και όλα τα examples.

Συμπληρωματικά, το Σχήμα 5.5 αποτυπώνει το μέσο μήκος των traces ανά dataset. Στο συγκεκριμένο γράφημα ορισμένες καμπύλες δεν ξεχωρίζουν εύκολα οπτικά, όχι επειδή λείπουν από τα δεδομένα, αλλά επειδή αλληλεπικαλύπτονται σχεδόν πλήρως. Πιο χαρακτηριστικά, τα HDFS και HealthApp έχουν ουσιαστικά ίδιες μέσες τιμές για όλες τις τιμές

του avgIntervals και επομένως η μία καμπύλη καλύπτει την άλλη. Επιπλέον, το Hadoop εμφανίζεται πολύ κοντά σε καμπύλες όπως του OpenStack και του Linux, με αποτέλεσμα η οπτική διάκριση να είναι δυσκολότερη όταν όλες οι σειρές παρουσιάζονται ταυτόχρονα στο ίδιο διάγραμμα.

Πέρα από την οπτική επικάλυψη, η μορφή των καμπυλών του μέσου μήκους δείχνει και τον τρόπο με τον οποίο ανταποκρίνεται κάθε dataset στην αύξηση του χρονικού παραθύρου. Όταν η καμπύλη αυξάνεται ομαλά και σχετικά μονοτονικά με το avgIntervals, η τμηματοποίηση ακολουθεί την αναμενόμενη κλίμακα και η μεταβολή του παραθύρου οδηγεί σε προβλέψιμη επιμήκυνση των traces. Ωστόσο, η ερμηνεία αυτή είναι αξιόπιστη μόνο όταν συνοδεύεται από χαμηλή μεταβλητότητα στο Σχήμα 5.4. Σε περιπτώσεις υψηλού συντελεστή μεταβλητότητας, ακόμη και αν η μέση τιμή αυξάνεται, το αποτέλεσμα υποδηλώνει ότι συνυπάρχουν πολύ διαφορετικά μήκη traces και επομένως η συμπεριφορά της τμηματοποίησης παραμένει ασταθής.

Συνεπώς, η ταυτόχρονη ανάγνωση μέσου μήκους και μεταβλητότητας είναι απαραίτητη καθώς ο μέσος όρος δείχνει την κεντρική τάση της κλίμακας, ενώ η μεταβλητότητα δείχνει πόσο συνεπής είναι αυτή η τάση. Επομένως, η αξιολόγηση της κατασκευής traces δεν αφορά μόνο το αν το μέσο μήκος αυξάνεται όσο μεγαλώνει το χρονικό παράθυρο, αλλά και το αν αυτή η αύξηση συνοδεύεται από επαρκή σταθερότητα. Πιο ουσιαστικά συμπεράσματα για τη σημασία αυτής της συμπεριφοράς προκύπτουν στην επόμενη ενότητα, όπου θα φανεί πώς η ποιότητα της τμηματοποίησης επηρεάζει τελικά την εξόρυξη των περιορισμών.

5.4 Αξιολόγηση DECLARE Mining

Η αξιολόγηση της διαδικασίας της δηλωτικής εξόρυξης βασίστηκε πρωτίστως στην ομοιότητα Jaccard μεταξύ του συνόλου των περιορισμών που εξορύχθηκαν από τα παραγόμενα ίχνη και του αντίστοιχου συνόλου περιορισμών που προέκυψε από τα ground-truth traces, ώστε να αποτιμηθεί τόσο ο βαθμός επικάλυψης όσο και η συνολική ποιότητα της ανακάλυψης. Αν G είναι το σύνολο των ground-truth constraints και D το σύνολο των discovered constraints, τότε η ομοιότητα Jaccard ορίζεται ως:

$$\text{Jaccard Similarity} = \frac{|G \cap D|}{|G \cup D|} \cdot 100 \tag{5.10}$$

Η μετρική αυτή χρησιμοποιείται ως κύριο κριτήριο σύγκρισης, επειδή αποτυπώνει άμεσα σε ποιον βαθμό τα δύο σύνολα συμφωνούν ως προς τους περιορισμούς που περιέχουν.

Συμπληρωματικά, για σφαιρικότερη αποτίμηση της συμπεριφοράς του mining, χρησιμοποιείται και μία πιο χαλαρή εκδοχή σύγκρισης, βασισμένη στην soft ομοιότητα μέσω του SequenceMatcher, όπως ακριβώς εφαρμόστηκε και στην αντίστοιχη αξιολόγηση του parsing. Ειδικότερα, οι περιορισμοί που ανακαλύπτονται δεν συγκρίνονται μόνο ως ακριβώς ίδιες λεκτικές αναπαραστάσεις, αλλά κάθε ground-truth constraint αντιστοιχίζεται με τον πλησιέστερο περιορισμό που έχει ανακαλυφθεί και υπολογίζεται η αντίστοιχη ομοιότητα συμβολοσειράς. Στη συνέχεια, λαμβάνεται ο μέσος όρος των πλησιέστερων αυτών ομοιοτήτων, ώστε να εξεταστεί αν το παραγόμενο αποτέλεσμα παραμένει ουσιαστικά κοντά στο

σωστό δηλωτικό μοντέλο ακόμη και όταν δεν υπάρχει ακριβής ταύτιση των constraints.

Ειδικότερα, οι ground-truth περιορισμοί εξετάστηκαν για όλους τους συνδυασμούς τιμών των παραμέτρων avgIntervals και support threshold και συγκρίθηκαν με τους περιορισμούς που παράχθηκαν από τα αρχεία `.csv` που δημιουργήθηκαν από το parsing για τις αντίστοιχες τιμές αυτών των παραμέτρων και όλες τις τιμές των υπόλοιπων. Δηλαδή, για κάθε ζευγάρι avgIntervals και support threshold έγινε σύγκριση με τα αποτελέσματα από όλη την διαδικασία για όλους τους συνδυασμούς shot, examples και models.

Η σύγκριση των παραγόμενων περιορισμών με το ground-truth για κάθε συνδυασμό παραμέτρων επιτρέπει την αντικειμενική αξιολόγηση της ποιότητας του mining, καθώς αποτυπώνει τόσο την ικανότητα του συστήματος να ανακαλύπτει τους πραγματικούς περιορισμούς όσο και την επίδραση των επιμέρους ρυθμίσεων στη συνολική απόδοση. Η αποτίμηση αυτή είναι κρίσιμη, διότι αναδεικνύει ποιες παραμετρικές επιλογές οδηγούν σε πιο αξιόπιστα και συνεπή αποτελέσματα, επιτρέποντας τη βελτιστοποίηση της διαδικασίας εξόρυξης δηλωτικών μοντέλων.

5.4.1 Επίδραση της παραμέτρου avgIntervals

Τα trace statistics της προηγούμενης ενότητας είναι χρήσιμα, αλλά όχι από μόνα τους επαρκή για να εξηγήσουν όλη τη συμπεριφορά της εξόρυξης. Για τον λόγο αυτό παρατίθεται αρχικά ο Πίνακας 5.7, ο οποίος δείχνει τη μέση ομοιότητα Jaccard των περιορισμών ανά τιμή της παραμέτρου avgIntervals. Η τιμή αυτή προκύπτει από τη σύγκριση του συνόλου των discovered constraints με το αντίστοιχο σύνολο ground-truth constraints για κάθε συνδυασμό παραμέτρων και στη συνέχεια από τον υπολογισμό του μέσου όρου ανά τιμή της παραμέτρου avgIntervals. Παράλληλα, για κάθε τιμή του avgIntervals παρουσιάζεται και η καλύτερη επιμέρους περίπτωση, δηλαδή ο συνδυασμός παραμέτρων που οδήγησε στη μεγαλύτερη τιμή της ομοιότητας Jaccard. Έτσι, ο παρακάτω πίνακας αποτυπώνει τόσο τη μέση συμπεριφορά κάθε χρονικού παραθύρου όσο και την καλύτερη επίδοση που μπορεί να επιτευχθεί για αυτό με βάση την κύρια μετρική αξιολόγησης της εξόρυξης.

AvgInt.	Mean Jacc.	Best Jacc.	Best std	Dataset	Support	Shot	Ex.	Model
100	15.786715327156504	100	157.04997362265308	Spark	0.3	8	0	llama3
250	26.31228956228956	100	302.1642305472675	Spark	0.3	8	0	llama3
500	11.675122592461301	100	520.6067998019236	Zookeeper	0.3	8	0	llama3
1000	27.311843504542995	100	128.5	Android	0.3	8	0	llama3

Table 5.7: Ομοιότητα Jaccard των περιορισμών ανά τιμή της παραμέτρου avgIntervals με τον βέλτιστο αντίστοιχο συνδυασμό παραμέτρων.

Τα αποτελέσματα του Πίνακα 5.7 δείχνουν ότι η καλύτερη μέση επίδοση εμφανίζεται για avgIntervals=1000, όπου η μέση ομοιότητα Jaccard φτάνει περίπου στο 27.31%. Η τιμή αυτή είναι αισθητά χαμηλότερη από τις αντίστοιχες τιμές της soft closest similarity, κάτι που είναι αναμενόμενο επειδή εδώ εξετάζεται η ακριβής επικάλυψη των συνόλων constraints και όχι η λεκτική εγγύτητα μεταξύ τους με αποτέλεσμα αν κατά το parsing ένα πρότυπο δεν έχει ανακτηθεί ακριβώς, τότε ακόμα και αν ο αντίστοιχος περι-

ορισμός υπάρχει, δεν προσμετράται. Επιπλέον, και στις τέσσερις τιμές της παραμέτρου avgIntervals εμφανίζεται τουλάχιστον μία περίπτωση πλήρους ταύτισης με το ground truth, δηλαδή Jaccard similarity 100%. Παρατηρείται επίσης ότι ο βέλτιστος συνδυασμός παραμέτρων παραμένει ιδιαίτερα σταθερός, με support = 0.3, shot = 8, examples = 0 και model = llama3 σε όλες τις περιπτώσεις, ενώ διαφοροποιείται μόνο το dataset στο οποίο επιτυγχάνεται η βέλτιστη τιμή.

Συμπληρωματικά, για σφαιρικότερη αποτίμηση της συμπεριφοράς της εξόρυξης, παρατίθεται και ο Πίνακας 5.8, ο οποίος δείχνει τη μέση πλησιέστερη ομοιότητα (mean closest similarity) των περιορισμών ανά τιμή της παραμέτρου avgIntervals. Η τιμή αυτή προκύπτει συγκρίνοντας, για κάθε ground-truth περιορισμό, τον πιο όμοιο περιορισμό στο ground truth με χρήση του αλγορίθμου SequenceMatcher και στη συνέχεια υπολογίζοντας τον μέσο όρο των αντίστοιχων ομοιοτήτων. Για κάθε τιμή του avgIntervals παρουσιάζεται και η καλύτερη επιμέρους περίπτωση, δηλαδή ο συνδυασμός παραμέτρων που οδήγησε στη μεγαλύτερη ομοιότητα. Έτσι, ο παρακάτω πίνακας αποτυπώνει τόσο τη μέση συμπεριφορά κάθε χρονικού παραθύρου όσο και την καλύτερη επίδοση που μπορεί να επιτευχθεί για αυτό υπό ένα πιο επιεικές κριτήριο σύγκρισης.

AvgInt.	Mean sim.	Best sim.	Best std	Dataset	Support	Shot	Ex.	Model
100	72.01490380857786	100	157.04997362265308	Spark	0.3	8	0	llama3
250	78.50066276066173	100	302.1642305472675	Spark	0.3	8	0	llama3
500	72.99817895	100	520.6067998019236	Zookeeper	0.3	8	0	llama3
1000	83.20449286731684	100	128.5	Android	0.3	8	0	llama3

Table 5.8: Closest similarity των περιορισμών ανά τιμή της παραμέτρου avgIntervals με τον βέλτιστο αντίστοιχο συνδυασμό παραμέτρων.

Τα αποτελέσματα του Πίνακα 5.8 δείχνουν ότι η καλύτερη μέση επίδοση εμφανίζεται για avgIntervals=1000, όπου η μέση ομοιότητα φτάνει περίπου στο 83.2%. Προφανώς οι καλύτερες περιπτώσεις μένουν ίδιες καθώς και οι 2 μετρικές είναι ίσες όταν το σύνολο των περιορισμών ανακτάται πλήρως. Βέβαια, αξίζει να σημειωθεί πως ενώ για όλες τις τιμές της παραμέτρου avgIntervals υπάρχουν περιπτώσεις όπου επιτυγχάνεται πλήρης ταύτιση των περιορισμών που ανακαλύπτονται από το parsing και το ground-truth, για avgIntervals ίσο με 250, η ταύτιση αφορά 16 περιορισμούς, ενώ για τις τιμές 100, 500 και 1000 οι περιορισμοί είναι 4, 2 και 1 αντίστοιχα.

5.4.2 Επίδραση του Support Threshold

Ο Πίνακας 5.9 εστιάζει αποκλειστικά στην παράμετρο του support threshold. Για κάθε συνδυασμό dataset, model, example, shot και avgIntervals, εξετάζονται διαδοχικά οι τιμές του support και εντοπίζεται η πρώτη τιμή στην οποία το πλήθος των constraints γίνεται μηδενικό, τόσο στο ground-truth mining όσο και στο discovered mining. Στη συνέχεια, οι περιπτώσεις συγκεντρώνονται ανά dataset και μετράται πόσες φορές το discovered αποτέλεσμα συμφωνεί ή διαφωνεί με το ground truth ως προς αυτό το πρώτο σημείο μηδενισμού. Με τον τρόπο αυτό, ο πίνακας δεν αποτιμά συνολικά την ποιότητα του

mining, αλλά απομονώνει τη σταθερότητα της συμπεριφοράς κάθε dataset όταν αυξάνεται η αυστηρότητα του support threshold.

dataset	Total	Matches	Mismatches
Android	64	64	0
Apache	64	64	0
HDFS	64	64	0
Hadoop	64	64	0
HealthApp	64	64	0
Linux	64	32	32
OpenStack	64	31	33
Proxifier	64	16	48
Spark	64	64	0
Windows	68	64	4
Zookeeper	64	64	0

Table 5.9: Συγκεντρωτική σύγκριση συμφωνίας και απόκλισης ως προς το πρώτο support μηδενισμού των constraints ανά dataset.

Ο πίνακας αυτός δείχνει ότι το support threshold δεν επηρεάζει όλα τα datasets με τον ίδιο τρόπο. Σε αρκετές περιπτώσεις υπάρχει πλήρης συμφωνία μεταξύ discovered και ground-truth συμπεριφοράς, γεγονός που σημαίνει ότι το discovered μοντέλο χάνει τους περιορισμούς περίπου στο ίδιο επίπεδο αυστηρότητας με το ground truth. Ωστόσο, σε datasets όπως τα Linux, OpenStack και κυρίως τα Proxifier εμφανίζονται πολλές αποκλίσεις, άρα το αποτέλεσμα είναι πιο εύθραυστο και μηδενίζεται σε διαφορετική τιμή support από αυτή που θα αναμενόταν. Συνεπώς, ο πίνακας αυτός αποτυπώνει την ευαισθησία της εξόρυξης ως προς το κατώφλι υποστήριξης και όχι τη συνολική ποιότητα του μοντέλου.

5.4.3 Συνολική Αποτίμηση της Εξόρυξης Περιορισμών

Για τη συνολική αποτίμηση της εξόρυξης των περιορισμών το ζητούμενο δεν είναι μόνο αν μια τιμή avgIntervals φαίνεται καλή κατά μέσο όρο ή αν το support threshold οδηγεί σε σταθερή συμπεριφορά, αλλά αν τελικά η εξόρυξη ανακαλύπτει constraints που ταυτίζονται πράγματι με το σωστό δηλωτικό μοντέλο. Για τον λόγο αυτό, ο Πίνακας 5.10 δίνει αρχικά μια συγκεντρωτική εικόνα ανά dataset για τη μέση ομοιότητα Jaccard μεταξύ των περιορισμών που ανακαλύφθηκαν από το ground truth και από τα αποτελέσματα του parsing, αλλά και για τη βέλτιστη επιλογή παραμέτρων που αναδείχθηκε τελικά για κάθε dataset.

Ο Πίνακας 5.10 δείχνει ότι το Android επιτυγχάνει πλήρη μέση ταύτιση με το ground truth, ενώ σχετικά υψηλές επιδόσεις εμφανίζονται και στα Zookeeper και HealthApp. Αντίθετα, τα Linux, Apache και κυρίως τα HDFS, Hadoop, OpenStack και Proxifier παρουσιάζουν χαμηλότερες τιμές ή ακόμη και μηδενική μέση ακριβή επικάλυψη, γεγονός που επιβεβαιώνει ότι η ομοιότητα Jaccard αποτελεί σαφώς αυστηρότερο κριτήριο από τη soft σύγκριση. Παράλληλα, η βέλτιστη ρύθμιση παραμένει σε μεγάλο βαθμό κοντά στις ίδιες τιμές που ανέδειξε και ο πίνακας ανά avgIntervals, δηλαδή σε μικρό support, μικρό shot size και χρήση του μοντέλου llama3. Επομένως, η συνολική εικόνα δείχνει

Dataset	Mean Jacc.	Best Jacc.	St.d.	AvgInt.	Support	Shot	Ex.	Model
Android	100	100	128.5	1000	0.3	8	0	llama3
HealthApp	58.333333333333336	100	311.6909527079668	250	0.3	8	0	llama3
Spark	43.143166357452074	100	157.04997362265308	100	0.3	8	0	llama3
Zookeeper	66.66666666666667	100	206.17786430263328	100	0.3	8	0	llama3
Linux	10.729166666666666	50	69.27842376959799	250	0.5	8	8	llama3
Apache	17.244625471065476	37.5	92.46891369536034	100	0.3	8	0	llama3
HDFS	0	0	82.47094678559178	100	0.3	8	0	llama3
Hadoop	0	0	0.5	1000	0.3	8	0	llama3
OpenStack	0	0	8.302860952707807	250	0.3	8	0	llama3
Proxifier	0	0	86.89503988754574	100	0.3	8	0	llama3
Windows	0	0	47	250	0.3	8	0	llama3

Table 5.10: Συγκεντρωτική αποτίμηση της ομοιότητας Jaccard ανά dataset με τον βέλτιστο αντίστοιχο συνδυασμό παραμέτρων.

ότι η ακριβής ανάκτηση των περιορισμών είναι εφικτή σε ορισμένα datasets, αλλά δεν γενικεύεται ομοιόμορφα σε όλα τα σύνολα δεδομένων.

Συμπληρωματικά, ο Πίνακας 5.11 δίνει την αντίστοιχη συγκεντρωτική εικόνα για τη soft closest similarity με SequenceMatcher, ώστε να αποτιμηθεί και πόσο κοντά παραμένουν τα discovered constraints στο ground truth ακόμη και όταν δεν υπάρχει ακριβής συντακτική ταύτιση.

Dataset	Mean sim.	Best sim.	St.d.	AvgInt.	Support	Shot	Ex.	Model
Android	100	100	128.5	1000	0.3	8	0	llama3
HealthApp	87.78735632183908	100	311.6909527079668	250	0.3	8	0	llama3
Spark	96.92929708747401	100	157.04997362265308	100	0.3	8	0	llama3
Zookeeper	98.98294215723996	100	206.17786430263328	100	0.3	8	0	llama3
Windows	74.34079126851042	98.78418329637842	47	250	0.5	8	0	llama3
OpenStack	66.71183538379981	96.30680965165679	8.302860952707807	250	0.5	8	8	qwen2.5:7b
HDFS	91.61195777599752	94.69699576645375	227.6148062	500	0.3	8	0	llama3
Linux	54.209087943976755	93.46052997328394	108.26587643389767	500	0.3	8	0	llama3
Apache	87.01770363745642	92.08931419457737	92.46891369536034	100	0.3	8	0	llama3
Hadoop	75.79142598825429	90.20469403065053	40.01386478459685	250	0.3	32	8	llama3
Proxifier	20.060377181582364	67.35247626107352	86.89503988754574	100	0.3	8	0	llama3

Table 5.11: Συγκεντρωτική αποτίμηση της closest similarity ανά dataset με τον βέλτιστο αντίστοιχο συνδυασμό παραμέτρων.

Ο πίνακας αυτός είναι σημαντικός, επειδή δείχνει τη μέση συμπεριφορά κάθε dataset στο σύνολο των δοκιμών και ποια συγκεκριμένη ρύθμιση αποδίδει καλύτερα σε κάθε περίπτωση. Έτσι, γίνεται φανερό ότι η ποιότητα του mining δεν εξαρτάται από μία μοναδική καθολική επιλογή παραμέτρων, αλλά από τα ιδιαίτερα χαρακτηριστικά κάθε συνόλου δεδομένων. Παράλληλα, η παράθεση των παραμέτρων ξεχωριστά αντί για ένα ενιαίο tuple επιτρέπει να φανεί πιο καθαρά ποιοι παράγοντες, όπως το avgIntervals, το support, τα shots, τα examples και το μοντέλο, συνδέονται συχνότερα με τις καλύτερες επιδόσεις.

Γενικότερα, τα σύνολα δεδομένων φαίνεται να έχουν αρκετά καλές επιδόσεις σε σχέση με τα αποτελέσματα της διαδικασίας. Ιδιαίτερα, τα dataset Linux και Proxifier παρουσιάζουν αισθητά χαμηλότερες αποδόσεις πράγμα που παρατηρείται σε όλα τα στάδια της διαδικασίας και επομένως είναι αναμενόμενο.

Ολοκληρώνοντας, προστίθεται και ο Πίνακας 5.12, ώστε να φανεί αν σε συνδυασμό με τη μετρική closest similarity, ανακτώνται σωστά τουλάχιστον οι τύποι των περιορισμών, ακόμη και όταν δεν επιτυγχάνεται ακριβής λεκτική ταύτιση ολόκληρου του constraint όπως αποτυπώνει η jaccard similarity.

Dataset	Mean F1	Best F1	St.d.	AvgInt.	Support	Shot	Ex.	Model
Android	100	100	128.5	1000	0.3	8	0	llama3
Apache	100	100	51	1000	0.3	8	0	llama3
HDFS	100	100	82.47094678559178	100	0.3	8	0	llama3
Hadoop	95.54781262098335	100	0.5	1000	0.3	8	0	llama3
HealthApp	100	100	311.6909527079668	250	0.3	8	0	llama3
Linux	72.83353010625738	100	69.27842376959799	250	0.3	8	8	llama3
OpenStack	85.52520765286722	100	8.302860952707807	250	0.3	8	8	llama3
Spark	100	100	157.04997362265308	100	0.3	8	0	llama3
Windows	100	100	47	250	0.5	8	0	llama3
Zookeeper	100	100	206.17786430263328	100	0.3	8	0	llama3
Proxifier	77.41258741258741	80	86.89503988754574	100	0.3	32	8	llama3

Table 5.12: Συγκεντρωτική αποτίμηση του F1-score ως προς τον τύπο των constraints ανά dataset με τον βέλτιστο αντίστοιχο συνδυασμό παραμέτρων.

Τα αποτελέσματα του Πίνακα 5.12 δείχνουν ότι, η ακριβής μετρική Jaccard similarity είναι αρκετά αυστηρή και επηρεάζεται άμεσα από τις αποκλίσεις του parsing στον εντοπισμό των προτύπων και κατ' επέκταση στις λεκτικές αναπαραστάσεις των περιορισμών και φαίνεται ότι σε αρκετά σύνολα δεδομένων ανακτάται ορθά τουλάχιστον το είδος των περιορισμών. Πιο συγκεκριμένα, στα Android, Apache, HDFS, HealthApp, Spark, Windows και Zookeeper η μέση τιμή του F1 ως προς τον τύπο των constraints είναι 100%, ενώ πολύ υψηλές τιμές εμφανίζονται και στα Hadoop και OpenStack. Αυτό σημαίνει ότι, ακόμη και όταν δεν επιτυγχάνεται ακριβής ταύτιση ολόκληρου του περιορισμού, το παραγόμενο αποτέλεσμα συχνά διατηρεί τη σωστή δηλωτική μορφή, δηλαδή τον σωστό τύπο περιορισμού. Η εικόνα αυτή είναι ιδιαίτερα χρήσιμη σε συνδυασμό με τη soft closest similarity, επειδή δείχνει ότι σε αρκετές περιπτώσεις οι αποκλίσεις του parsing δεν οδηγούν απαραίτητα σε πλήρη απώλεια της δηλωτικής δομής. Ακόμα και αν το σύστημα μπορεί να αποτυγχάνει στην αυστηρή σύγκριση μπορεί να εξακολουθεί να ανακτά σωστά τη γενική κατηγορία των περιορισμών που περιγράφουν τη συμπεριφορά του. Οι τιμές για τα σύνολα δεδομένων Linux και Proxifier οι τιμές παραμένουν αισθητά χαμηλότερες, γεγονός που επιβεβαιώνει ότι οι δυσκολίες των προηγούμενων σταδίων επηρεάζουν όχι μόνο την ακριβή μορφή αλλά και εν μέρει το ίδιο το είδος των constraints που ανακαλύπτονται.

5.5 Χρόνος Εκτέλεσης

Η αξιολόγηση του χρόνου εκτέλεσης εξετάζει ξεχωριστά τα δύο βασικά στάδια της προτεινόμενης ροής. Αφενός τη δημιουργία του αρχείου `.xes` από τα αποτελέσματα του parsing και αφετέρου την εξόρυξη δηλωτικού μοντέλου μέσω του MINERful. Ο διαχωρισμός αυτός είναι αναγκαίος, διότι επιτρέπει να αποτιμηθεί ποιο στάδιο επιβαρύνει πραγματικά το συνολικό υπολογιστικό κόστος, το επίπεδο της μετατροπής σε event log ή/και το

στάδιο του mining. Τα αποτελέσματα που παρουσιάζονται έχουν παραχθεί αξιοποιώντας την Υπολογιστική Συστοιχία και τις παρεχόμενες υπηρεσίες υποστήριξης του Κέντρου Ηλεκτρονικής Διακυβέρνησης του Α.Π.Θ.

Για κάθε dataset και για κάθε μοντέλο χρησιμοποιούνται ο μέσος όρος (mean), η τυπική απόκλιση (standard deviation) και η διάμεσος (median) για τις εκτελέσεις της διαδικασίας. Ο μέσος όρος παρέχει μία συνοπτική εικόνα της επιβάρυνσης, η τυπική απόκλιση αποτυπώνει τη σταθερότητα των επαναλήψεων, ενώ η διάμεσος είναι ιδιαίτερα χρήσιμη σε περιπτώσεις όπου υπάρχουν μεμονωμένες ακραίες τιμές που μπορούν να αλλοιώσουν τον μέσο χρόνο.

Ο Πίνακας 5.13 παρουσιάζει τους χρόνους δημιουργίας των event log ανά dataset και μοντέλο. Είναι εμφανές ότι το στάδιο αυτό παραμένει εξαιρετικά ελαφρύ υπολογιστικά, καθώς οι χρόνοι βρίσκονται σταθερά σε κλάσματα του δευτερολέπτου. Παράλληλα, οι τιμές της δείχνουν ότι ότι η διαδικασία δεν επηρεάζεται ιδιαίτερα από το κάθε μοντέλο και σύνολο δεδομένων.

Dataset	Model	Samples	Mean (s)	Std (s)	Median (s)
Android	llama3	5	0.292	0.008	0.287
Android	qwen2.5:7b	5	0.286	0.003	0.285
Apache	llama3	5	0.249	0.024	0.239
Apache	qwen2.5:7b	5	0.230	0.003	0.232
HDFS	llama3	5	0.270	0.037	0.254
HDFS	qwen2.5:7b	5	0.256	0.011	0.251
Hadoop	llama3	5	0.290	0.030	0.276
Hadoop	qwen2.5:7b	5	0.286	0.020	0.276
HealthApp	llama3	5	0.372	0.134	0.336
HealthApp	qwen2.5:7b	5	0.258	0.023	0.251
Linux	llama3	5	0.309	0.031	0.296
Linux	qwen2.5:7b	5	0.299	0.003	0.299
OpenStack	llama3	5	0.361	0.040	0.344
OpenStack	qwen2.5:7b	5	0.329	0.002	0.329
Proxifier	llama3	5	0.291	0.118	0.234
Proxifier	qwen2.5:7b	5	0.235	0.001	0.235
Spark	llama3	5	0.242	0.002	0.241
Spark	qwen2.5:7b	5	0.250	0.011	0.242
Windows	llama3	5	0.280	0.021	0.277
Windows	qwen2.5:7b	5	0.264	0.004	0.264
Zookeeper	llama3	5	0.292	0.021	0.283
Zookeeper	qwen2.5:7b	5	0.283	0.004	0.281

Table 5.13: Χρόνος δημιουργίας XES ανά dataset και μοντέλο.

Η εικόνα διαφοροποιείται αισθητά στο στάδιο του declarative mining. Όπως φαίνεται στον Πίνακα 5.14, ο χρόνος του MINERful είναι κατά τάξεις μεγέθους μεγαλύτερος από τον χρόνο παραγωγής των αρχείων `.xes` και αποτελεί το κυρίαρχο μέρος του συνολικού κόστους. Για τα 2 μοντέλα παρατηρείται γενικά παρόμοια τάξη μεγέθους στους χρόνους με εξαίρεση το σύνολο OpenStack για το μοντέλο qwen2.5:7b όπου εμφανίζεται πολύ μεγάλη επιβάρυνση, με μέσο χρόνο 67.5 δευτερολέπτων. Η συγκεκριμένη περίπτωση επηρεάζει σημαντικά τη συνολική εικόνα του μοντέλου και αναδεικνύει ότι η χρονική συμπεριφορά του mining εξαρτάται από το σύνολο δεδομένων και το parsing.

Τα παραπάνω στοιχεία επιβεβαιώνουν τη διάκριση ανάμεσα στα δύο στάδια. Στη δημιουργία αρχείων `.xes` τα δύο LLMs είναι πρακτικά ισοδύναμα, αφού αυτή η διαδικασία

Dataset	Model	Samples	Mean (s)	Std (s)	Median (s)
Android	llama3	5	12.590	0.354	12.724
Android	qwen2.5:7b	5	11.824	0.333	11.923
Apache	llama3	5	1.522	0.163	1.596
Apache	qwen2.5:7b	5	1.522	0.183	1.591
HDFS	llama3	5	4.886	0.281	5.014
HDFS	qwen2.5:7b	5	4.221	0.219	4.309
Hadoop	llama3	5	4.748	0.232	4.847
Hadoop	qwen2.5:7b	5	6.737	0.292	6.828
HealthApp	llama3	5	4.421	0.208	4.486
HealthApp	qwen2.5:7b	5	4.334	0.244	4.453
Linux	llama3	5	6.049	0.285	6.126
Linux	qwen2.5:7b	5	8.092	0.207	8.157
OpenStack	llama3	5	10.977	0.364	10.991
OpenStack	qwen2.5:7b	5	67.521	1.260	67.292
Proxifier	llama3	5	6.667	0.054	6.645
Proxifier	qwen2.5:7b	5	7.399	0.044	7.398
Spark	llama3	5	2.534	0.193	2.577
Spark	qwen2.5:7b	5	1.999	0.194	2.119
Windows	llama3	5	2.868	0.055	2.890
Windows	qwen2.5:7b	5	2.698	0.052	2.721
Zookeeper	llama3	5	3.079	0.032	3.083
Zookeeper	qwen2.5:7b	5	3.098	0.046	3.116

Table 5.14: Χρόνος εκτέλεσης του MINERful ανά dataset και μοντέλο.

δεν επιβαρύνεται χρονικά από τα αποτελέσματα του parsing. Αντίθετα, στη διαδικασία της εξόρυξης διαδικασιών με το MINERful το σύνολο δεδομένων και το μοντέλο επηρεάζουν τον χρόνο εκτέλεσης καθώς επηρεάζουν και τα αντίστοιχα αποτελέσματα. Αυτό σημαίνει ότι οποιαδήποτε μελλοντική βελτιστοποίηση του χρόνου εκτέλεσης θα πρέπει να εστιάσει πρωτίστως είτε στη ρύθμιση του MINERful.

5.6 Αποτελέσματα και Συζήτηση

Συνολικά, τα αποτελέσματα της πειραματικής αξιολόγησης δείχνουν ότι η προτεινόμενη ροή μπορεί να λειτουργήσει αποτελεσματικά σε ένα ευρύ σύνολο datasets, αλλά η τελική της ποιότητα εξαρτάται έντονα από τις επιδόσεις σε όλα τα επιμέρους στάδια. Η ποιότητα του parsing επηρεάζει άμεσα τη δημιουργία των event logs, η ποιότητα των ιχνών επηρεάζει τη σταθερότητα του mining και, τελικά, η αξιοπιστία των εξαγόμενων περιορισμών προκύπτει από τον συνδυασμό αυτών των παραγόντων και όχι από μία μόνο καλή επιλογή παραμέτρου. Με αυτή την έννοια, η εργασία επιβεβαιώνει ότι η end-to-end αξιολόγηση είναι ουσιαστικότερη από την απομονωμένη αποτίμηση κάθε σταδίου ξεχωριστά.

Γίνεται επίσης φανερό ότι υπάρχουν σύνολα δεδομένων στα οποία η συνολική διαδικασία παραμένει σταθερή και έχει καλά αποτελέσματα, όπως στο Android, Spark και Zookeeper, ενώ σε άλλα σύνολα, όπως το Linux, OpenStack και το Proxifier, οι αδυναμίες που εμφανίζονται ήδη στο parsing μεταφέρονται και στο τελικό αποτέλεσμα της εξόρυξης. Αυτό δείχνει ότι η προτεινόμενη μεθοδολογία έχει μεν ικανοποιητική δυνατότητα γενίκευσης, αλλά δεν είναι ομοιόμορφα ανθεκτική σε όλα τα είδη δεδομένων. Επομένως, η κύρια συμβολή της αφορά τη δημιουργία ενός συνεκτικού πλαισίου μέσα στο

οποίο οι επιδράσεις των επιμέρους επιλογών μπορούν να μελετηθούν συστηματικά και να αξιοποιηθούν για περαιτέρω βελτιστοποίηση.

Η συνολική εικόνα των αποτελεσμάτων δείχνει ότι η προτεινόμενη μεθοδολογία καταφέρνει σε μεγάλο βαθμό να διατηρεί την ουσιώδη πληροφορία των αρχείων καταγραφής κατά τα διαδοχικά στάδια επεξεργασίας. Υπάρχουν ορισμένες περιπτώσεις όπου εμφανίζονται αποκλίσεις ως προς την πλήρη αντιστοίχιση των παραγόμενων αποτελεσμάτων με τα αναμενόμενα, όμως φαίνεται ότι οι βασικές κατηγορίες συμπεριφοράς και οι κύριες σχέσεις μεταξύ γεγονότων εξακολουθούν να αποτυπώνονται ικανοποιητικά. Επομένως, τα ευρήματα ενισχύουν την εκτίμηση ότι η προτεινόμενη ροή μπορεί να λειτουργήσει ως επαρκής βάση για πιο σύνθετες διαδικασίες ανάλυσης και εξαγωγής γνώσης από ετερογενή log δεδομένα.

Η συμπερίληψη 11 διαφορετικών datasets στην τελική ποσοτική αξιολόγηση ενισχύει την ένδειξη ότι η προτεινόμενη ροή διαθέτει ουσιαστική, αν και όχι απεριόριστη, δυνατότητα γενίκευσης. Η εξαίρεση των υπολοίπων συνόλων οφείλεται σε συγκεκριμένα προβλήματα ερμηνείας των χρονικών πεδίων, όπως αμφίσημες χρονικές αναπαραστάσεις ή πεδία χρόνου τα οποία ορίζονται στη μορφοποίηση με άλλες ονομασίες με αποτέλεσμα να μην μπορούν να αντιστοιχηθούν σε ένα event timestamp. Το στοιχείο αυτό είναι σημαντικό, διότι δείχνει ότι η γενίκευση της μεθόδου εξαρτάται και από το κατά πόσο το ίδιο το log παρέχει χρονική πληροφορία με επαρκή λειτουργική σημασία για τη δημιουργία ιχνών.

Τέλος, η αξιολόγηση της εξόρυξης διαδικασιών αποτελεί μια ιδιαίτερα σύνθετη διαδικασία, η οποία δεν μπορεί να βασιστεί αποκλειστικά στην αντιστοίχιση συνόλων constraints. Η ορθή ερμηνεία των αποτελεσμάτων απαιτεί εξειδικευμένες γνώσεις για τη σημασία των δηλωτικών περιορισμών για το εκάστοτε dataset, ώστε οι εξαγόμενοι περιορισμοί να ερμηνεύονται σε συνάρτηση με τη λειτουργία του συγκεκριμένου συστήματος, τη σημασία των γεγονότων του και τις ιδιαιτερότητες του αντίστοιχου περιβάλλοντος καταγραφής. Κατά συνέπεια, η παρούσα εργασία προτείνει μια ροή επεξεργασίας και ένα πλαίσιο αξιολόγησης το οποίο προσπαθεί να αποτιμήσει με μια γενικότερη προσέγγιση τα συνολικά αποτελέσματα.

6 Συμπεράσματα, Περιορισμοί και Μελλοντική Έρευνα

6.1 Σύνοψη

Η παρούσα εργασία προτείνει μια καινοτόμα ενιαία ροή που συνδέει λειτουργικά το log parsing, τη μετατροπή των αποτελεσμάτων σε event logs και την εξόρυξη δηλωτικών μοντέλων. Η βασική της συνεισφορά δεν βρίσκεται μόνο στην επιμέρους αξιοποίηση του LILAC και του MINERful, αλλά κυρίως στην ενοποίηση σταδίων που συνήθως απαιτούν αποσπασματική υλοποίηση, σημαντικό χρόνο προσαρμογής και εξειδικευμένη γνώση από διαφορετικά πεδία, όπως η επεξεργασία μη δομημένων logs, ο μετασχηματισμός δεδομένων και η εξόρυξη διαδικασιών. Υπό αυτή την έννοια, η προτεινόμενη διαδικασία συνιστά ένα βήμα προς την αυτοματοποίηση μιας κατά τα άλλα δύσκολα χρονοβόρας ανάλυσης που δεν αναπαράγεται εύκολα, ενώ παρέχει ένα συνεκτικό πειραματικό πλαίσιο για τη μελέτη της σχέσης μεταξύ parsing ποιότητας, κατασκευής ιχνών και της δηλωτικής εξόρυξης αποτελεσμάτων.

Στο πλαίσιο αυτό, το log parsing και η εξόρυξη διαδικασιών αντιμετωπίζονται ως διαδοχικά και αλληλοεξαρτώμενα στάδια της ίδιας αναλυτικής διαδικασίας. Ενώ πολλές προσεγγίσεις σταματούν είτε στην εξαγωγή templates είτε στην ανάλυση ήδη έτοιμων event logs, η παρούσα εργασία επιχειρεί να καλύψει το χάσμα ανάμεσα στα ακατέργαστα log μηνύματα και στην παραγωγή declarative knowledge, συνδέοντας τη μη δομημένη είσοδο με ένα τελικό, αναλύσιμο μοντέλο. Η end-to-end αυτή οπτική έχει μεθοδολογικό ενδιαφέρον, επειδή επιτρέπει να μελετηθεί συστηματικά πώς σφάλματα, παραδοχές και επιλογές των πρώιμων σταδίων επηρεάζουν το τελικό αποτέλεσμα της εξόρυξης, μειώνοντας παράλληλα την απαίτηση για αποσπασματική εξειδικευμένη γνώση κατά την εφαρμογή των επιμέρους διαδικασιών.

Η συνδυαστική αξιοποίηση του LILAC για parsing, του ενδιάμεσου σταδίου δημιουργίας ιχνών και event log και του MINERful για declarative mining διαμορφώνει ένα πλαίσιο που μπορεί να επαναληφθεί σε διαφορετικά σύνολα δεδομένων χωρίς να απαιτείται κάθε φορά πλήρης επανασχεδιασμός της ροής. Η σημασία αυτού του στοιχείου είναι ιδιαίτερη σε ένα πεδίο όπου η αναπαραγωγή αποτελεσμάτων παραμένει δύσκολη, επειδή δεν προϋποθέτει μόνο πρόσβαση στα κατάλληλα εργαλεία αλλά και πρακτική γνώση για το πώς αυτά συνδυάζονται λειτουργικά. Παράλληλα, η εργασία αξιολογεί τις συνέπειές του log parsing στην ανακάλυψη περιορισμών και στον χρόνο εκτέλεσης, υιοθετώντας έτσι μια ολιστική αποτίμηση της συνολικής ροής αντί μιας αποσπασματικής εξέτασης μεμονωμένων υποσυστημάτων.

Τέλος, η συνεισφορά της εργασίας διαμορφώνει μια βάση για περαιτέρω εξέλιξη. Προτείνεται ένα λειτουργικό πλαίσιο πάνω στο οποίο μπορούν να δοκιμαστούν εναλλακτικοί parsers, διαφορετικές στρατηγικές δημιουργίας ιχνών, νέες τεχνικές συσχέτισης γεγονότων και διαφορετικά εργαλεία εξόρυξης. Με αυτή την έννοια, μπορεί να αποτελέσει αφετηρία τόσο για πιο ώριμες μορφές αυτοματοποίησης όσο και για μελλοντικές βελτιώσεις σε κάθε στάδιο της διαδικασίας.

6.2 Περιορισμοί και Αδυναμίες

Η παρούσα εργασία παρουσιάζει ορισμένους ουσιώδεις περιορισμούς. Αρχικά, η προτεινόμενη ροή δεν είναι πλήρως αυτοματοποιημένη, καθώς σε κρίσιμα σημεία εξακολουθούν να απαιτούνται χειροκίνητες παρεμβάσεις ή αποφάσεις από τον χρήστη. Αυτό ισχύει ιδιαίτερα στα στάδια της ερμηνείας του log format, της επιλογής των κατάλληλων χρονικών γνωρισμάτων και της διαμόρφωσης των απαιτούμενων μετασχηματισμών, όπου η ανθρώπινη επίβλεψη παραμένει αναγκαία για την ορθή λειτουργία της διαδικασίας.

Ένα ακόμη βασικό σημείο είναι ότι η δυνατότητα γενίκευσης της μεθόδου ως προς την αναγνώριση και αξιοποίηση χρονικών πεδίων παραμένει περιορισμένη όταν τα δεδομένα ακολουθούν ιδιαίτερα εξειδικευμένες ή μη συμβατικές μορφές χρονικής αναπαράστασης. Παρότι η εργασία διαχειρίζεται σε σημαντικό βαθμό την ετερογένεια των datasets, η επιτυχία του μετασχηματισμού σε αρχεία `.xes` εξακολουθεί να εξαρτάται από το αν τα διαθέσιμα χρονικά γνωρίσματα μπορούν να ερμηνευθούν με αξιόπιστο τρόπο. Σε ιδιόμορφα logs, όπου η χρονική πληροφορία είναι ασαφής, ελλιπής ή ενσωματωμένη σε πιο σύνθετες σημασιολογικές δομές, η προσέγγιση δεν γενικεύει με την ίδια ευχέρεια.

Επιπλέον, η διαχείριση της ετερογένειας των logs είναι ουσιαστική αλλά όχι καθολική. Η χρήση του LILAC και η συνολική ροή που υλοποιήθηκε βελτιώνουν αισθητά τη δυνατότητα επεξεργασίας διαφορετικών συνόλων δεδομένων, χωρίς όμως να εξαλείφουν την ανάγκη ειδικού χειρισμού σε όλες τις περιπτώσεις. Συνεπώς, η μεθοδολογία διατηρεί εν μέρει την ανάγκη προσαρμογών ανά dataset, προσφέρει όμως ένα πιο σταθερό πλαίσιο μέσα στο οποίο αυτές μπορούν να ενταχθούν συστηματικά.

Τέλος, ένας σημαντικός περιορισμός αφορά τον τρόπο κατασκευής των traces. Στην παρούσα υλοποίηση, τα ίχνη δημιουργούνται κυρίως βάσει της χρονικής σειράς των γεγονότων, επιλογή λειτουργική και χρήσιμη ιδίως όταν δεν υπάρχει σαφές case identifier. Ωστόσο, η προσέγγιση αυτή ενδέχεται να παραλείπει συσχετίσεις διεργασιών που δεν εκφράζονται πρωτίστως χρονικά, αλλά μέσω άλλων γνωρισμάτων, όπως αναγνωριστικά χρηστών, διεργασιών, sessions ή αντικειμένων του συστήματος. Ως αποτέλεσμα, ορισμένες πραγματικές σχέσεις διαδικασιών μπορεί να μην αποτυπώνονται επαρκώς στο τελικό event log και, κατ' επέκταση, ούτε στο μοντέλο που παράγεται από την εξόρυξη.

6.3 Κατευθύνσεις Μελλοντικής Έρευνας

Οι παραπάνω περιορισμοί δείχνουν ότι η φυσική συνέχεια της παρούσας εργασίας βρίσκεται στην περαιτέρω ωρίμανση και αυτοματοποίηση της συνολικής ροής. Ένα επόμενο βήμα θα ήταν η ανάπτυξη πιο αυτόνομων μηχανισμών που να περιορίζουν τις χειροκίνητες παρεμβάσεις στην επιλογή πεδίων, στους μετασχηματισμούς και στη ρύθμιση των επιμέρους σταδίων. Στο ίδιο πλαίσιο εντάσσεται και η ανάγκη για καλύτερη γενίκευση ως προς τα χρονικά γνωρίσματα, ιδιαίτερα σε σύνολα δεδομένων όπου η χρονική πληροφορία εμφανίζεται με σύνθετες, ασαφείς ή ιδιαίτερα εξειδικευμένες μορφές. Προς αυτή την κατεύθυνση, θα είχε ενδιαφέρον η αξιοποίηση πιο εξελιγμένων τεχνικών αναγνώρισης και κανονικοποίησης χρονικής πληροφορίας, καθώς και η διερεύνηση υβριδικών μηχανισμών

που να συνδυάζουν κανόνες, στατιστικές ενδείξεις και LLM-based reasoning για την πιο αξιόπιστη ερμηνεία τέτοιων πεδίων. Μία ακόμη πιο εξελιγμένη έκδοση θα μπορούσε να είναι ο εντοπισμός όλων των πεδίων ενός μηνύματος με αυτές τις τεχνικές, έτσι ώστε να μην χρειάζεται να δίνεται συγκεκριμένη μορφοποίηση για κάθε σύνολο δεδομένων.

Εξίσου σημαντική είναι και η επέκταση του τρόπου με τον οποίο δημιουργούνται τα traces. Η παρούσα προσέγγιση βασίζεται κυρίως σε χρονικά παράθυρα πράγμα που δεν είναι πάντοτε επαρκές για την αποτύπωση πιο σύνθετων σχέσεων μεταξύ γεγονότων. Για τον λόγο αυτό, μελλοντικές επεκτάσεις θα μπορούσαν να εξετάσουν και μη χρονικά κριτήρια, όπως processIDs, sessionIDs, userIDs ή άλλες λογικές οντότητες που συνδέουν γεγονότα της ίδιας διαδικασίας. Μια τέτοια κατεύθυνση θα μπορούσε να οδηγήσει σε μια πιο αξιόπιστη ανακατασκευή των πραγματικών διαδικασιών, ιδίως σε πολύπλοκα περιβάλλοντα.

A Παραδείγματα Χρήσης Εργαλείων Log Parsing

Στο παρόν παράρτημα παρατίθενται ενδεικτικά παραδείγματα χρήσης αντιπροσωπευτικών εργαλείων log parsing, με στόχο τη συμπληρωματική πρακτική κατανόηση της λειτουργίας τους. Σε αντίθεση με το κεφάλαιο 3 της σχετικής βιβλιογραφίας , όπου δόθηκε έμφαση στα χαρακτηριστικά, στη θεωρητική ανάλυση και στους περιορισμούς των εργαλείων, εδώ η παρουσίαση επικεντρώνεται σε αντιπροσωπευτικά σενάρια εκτέλεσης και σε ενδεικτικά παραδείγματα εξόδου.

A.1 Ενδεικτική Χρήση του Logstash

Το ακόλουθο παράδειγμα παρουσιάζει μία απλή αλλά πλήρη ροή επεξεργασίας στο Logstash για τη μετατροπή ακατέργαστων αρχείων καταγραφής σε δομημένη μορφή. Συγκεκριμένα, ως είσοδος χρησιμοποιείται ένα τοπικό αρχείο log, στη συνέχεια εφαρμόζεται κανόνας `grok` για την εξαγωγή επιλεγμένων πεδίων από κάθε γραμμή, ενώ στο τελικό στάδιο τα αποτελέσματα εξάγονται σε αρχείο `.csv`. Το παράδειγμα αποσκοπεί στην ανάδειξη ενός τυπικού σεναρίου παραμετροποίησης και της μορφής της παραγόμενης εξόδου.

Algorithm 7 Logstash Pipeline Configuration

```
1: input {
2:   file {
3:     path: "etc/raw_logs.log"
4:     start_position: "beginning"
5:     sincedb_path: "NUL"
6:   }
7: }

8: filter {
9:   grok {
10:    match: {"message" => "%{TIMESTAMP : timestamp}%{LOGLEVEL : loglevel}%{WORD : component}%{GREEDYDATA : content}"}
11:  }
12:  mutate {
13:    remove_field: [ "component" ]
14:  }
15: }

16: output {
17:   csv {
18:     path: "etc/log_parsed.csv"
19:     fields: ["timestamp","loglevel","content"]
20:   }
21: }
```

Στην παραπάνω διαμόρφωση, κάθε εγγραφή μετατρέπεται σε μία περισσότερο δομημένη αναπαράσταση που περιλαμβάνει μόνο τα πεδία `timestamp`, `loglevel` και `content`. Το αποτέλεσμα είναι ένα αρχείο κατάλληλο για περαιτέρω ανάλυση ή για τροφοδότηση επόμενων σταδίων επεξεργασίας.

A.2 Ενδεικτική Χρήση του Drain

Το επόμενο παράδειγμα παρουσιάζει μία ενδεικτική εκτέλεση του Drain σε δεδομένα του συνόλου HDFS με συγκεκριμένη παραμετροποίηση. Στόχος δεν είναι η περαιτέρω θεωρητική ανάλυση της εσωτερικής λειτουργίας του αλγορίθμου, αλλά η πρακτική αποτύπωση του τρόπου με τον οποίο μία ροή log μηνυμάτων αντιστοιχίζεται σε πρότυπα μηνυμάτων (templates) υπό ένα δεδομένο σύνολο παραμέτρων.

Algorithm 8 Drain: Online Log Parsing

```
Set dataset = "HDFS"
Set depth = 5, max_child = 150, sim_threshold = 0.48, max_clusters = 1000
parser ← Drain(depth, max_child, sim_threshold)
for each log in log_stream do
    template ← parser.parse(log)
    print f"Log: {log} → Template: {template}"
```

Στο συγκεκριμένο παράδειγμα παρουσιάζονται δύο αντιπροσωπευτικές περιπτώσεις, ώστε να καταδειχθεί τόσο η δυνατότητα επιτυχούς γενίκευσης όσο και οι περιπτώσεις στις οποίες η εξαγωγή προτύπου γίνεται υπερβολικά γενική.

Για το log μήνυμα:

```
081109 204655 556 INFO dfs.DataNode$PacketResponder:  Received block
blk_3587508140051953248 of size 67108864 from /10.251.42.84
```

το εξαγόμενο template είναι:

```
<*>:  Received block <*> of size <*> from <*>
```

Η εξαγωγή αυτή μπορεί να θεωρηθεί επιτυχής, καθώς διατηρεί τη σταθερή δομή του μηνύματος και ταυτόχρονα αντικαθιστά τα μεταβλητά στοιχεία με κατάλληλους συμβολισμούς γενίκευσης.

Αντίθετα, για το μήνυμα:

```
081110 103343 27 INFO dfs.FSNamesystem:  BLOCK*
NameSystem.allocateBlock:  /user/root/rand/_temporary/_task_200811101
024_0001_m_000206_0/part-00206.  blk_-2230892279625104430
```

παράγεται το template:

```
<*>:  <*> <*>
```

Στην περίπτωση αυτή, το παραγόμενο αποτέλεσμα είναι υπερβολικά αφαιρετικό και δεν διατηρεί επαρκή πληροφορία για τη σημασιολογική δομή του αρχικού μηνύματος. Το παράδειγμα αυτό αναδεικνύει ότι, ακόμη και όταν η διαδικασία parsing ολοκληρώνεται επιτυχώς από υπολογιστική άποψη, η ποιότητα του προκύπτοντος template μπορεί να διαφέρει σημαντικά.

Τα τελικά αποτελέσματα μπορούν να αποθηκευτούν σε αρχείο `.csv`, το οποίο περιλαμβάνει το αρχικό log μήνυμα, το αντίστοιχο template και το αναγνωριστικό του cluster, διευκολύνοντας τόσο την ποιοτική όσο και την ποσοτική αξιολόγηση της εξόδου του parser.

A.3 Ενδεικτική Χρήση του LogPPT

Το ακόλουθο παράδειγμα παρουσιάζει μία τυπική ροή χρήσης του LogPPT με αξιοποίηση προεκπαιδευμένου μοντέλου σε log μηνύματα του συνόλου HDFS. Η παρουσίαση εστιάζει στην πρακτική ακολουθία βημάτων που οδηγεί από το αρχικό μήνυμα στην παραγωγή του αντίστοιχου template, χωρίς να επαναλαμβάνονται οι θεωρητικές λεπτομέρειες της προσέγγισης.

Algorithm 9 LogPPT: Semantic Log Parsing via Token Classification

```
Set dataset = "HDFS"
Set model_name = "roberta-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForTokenClassification.from_pretrained(model_name)
for each log in log_stream do
    tokens ← tokenizer(log)
    labels ← model(tokens)
    Replace variable tokens with '*'
    Store (log, template)
```

Στο πλαίσιο αυτού του παραδείγματος, κάθε log μήνυμα υποβάλλεται αρχικά σε tokenization, στη συνέχεια διέρχεται από το μοντέλο για την παραγωγή ετικετών ανά token, και τέλος μετατρέπεται σε template μέσω αντικατάστασης των μεταβλητών στοιχείων. Το αποτέλεσμα της διαδικασίας είναι ένα σύνολο αντιστοιχίσεων μεταξύ αρχικών log μηνυμάτων και εξαγόμενων templates, το οποίο μπορεί να αξιοποιηθεί σε επόμενα στάδια ανάλυσης ή σύγκρισης.

Συνολικά, τα παραπάνω παραδείγματα δεν αποσκοπούν στην εξαντλητική αξιολόγηση των εργαλείων, αλλά στη συμπληρωματική πρακτική αποσαφήνιση του τρόπου χρήσης τους. Με τον τρόπο αυτό, το παράρτημα λειτουργεί υποστηρικτικά προς την κύρια ανάλυση του κεφαλαίου 3, παρέχοντας συγκεκριμένα σενάρια εφαρμογής χωρίς να επαναλαμβάνει εκτενώς τις εννοιολογικές και συγκριτικές περιγραφές που έχουν ήδη προηγηθεί.

Β Παράδειγμα Ροής της Υλοποίησης

Το Σχήμα Β.1 παρουσιάζει ένα αναλυτικό παράδειγμα της ροής της υλοποίησης, ξεκινώντας από ένα μη δομημένο αρχείο log με μορφή `<Time>|<Component>|<Pid>|<Content>` από το σύνολο δεδομένων HealthApp. Σκοπός του σχήματος είναι να γίνει σαφέστερη η συνολική επεξεργασία του log αυτού και πως μοιάζουν στην πράξη τα στάδια επεξεργασίας του. Με τον τρόπο αυτό, το παράρτημα λειτουργεί συμπληρωματικά προς την περιγραφή της κύριας μεθοδολογίας, παρέχοντας το παρακάτω παράδειγμα το στο οποίο φαίνονται αναλυτικά.

Στο πρώτο στάδιο της ροής, κάθε log μήνυμα αναλύεται έτσι ώστε να βρεθεί ο τύπος γεγονότος στον οποίο ανήκει μέσω του parsing. Ακολουθεί το στάδιο της μετατροπής του αρχείου σε δομημένη αναπαράσταση όπου τα πεδία του συνόλου δεδομένων αντιστοιχίζονται σε κάθε κομμάτι της καταγραφής. Το επόμενο στάδιο αφορά την δημιουργία των traces από τα οποία θα προκύψει το event log που αναπαρίσταται στο `.xes` αρχείο. Τα πεδία που δεν έχουν καθοριστικό ρόλο αποθηκεύονται ως attributes έτσι ώστε να μην χαθεί η πληροφορία και μαζί με τα γεγονότα μαζί με τα αναγνωριστικά τους απαρτίζουν το trace. Η δομημένη αυτή μορφή επιτρέπει την προετοιμασία των δεδομένων για το επόμενο στάδιο, δηλαδή εξόρυξη περιορισμών. Η μετάβαση αυτή από το μη δομημένο στο δομημένο επίπεδο είναι πολύ σημαντική, καθώς αποτελεί τη γέφυρα ανάμεσα στο log parsing και στο process mining. Στο τελικό στάδιο, το παραγόμενο αρχείο `.xes` χρησιμοποιείται ως είσοδος στο MINERful, το οποίο εφαρμόζει declarative process mining και εξάγει τους αντίστοιχους περιορισμούς. Οι περιορισμοί αυτοί περιγράφουν σχέσεις και εξαρτήσεις μεταξύ των δραστηριοτήτων που εντοπίστηκαν στα προηγούμενα στάδια και αποτελούν το τελικό γνώρισμα της ανάλυσης. Επομένως, το σχήμα δεν αποτυπώνει απλώς μία τεχνική ακολουθία βημάτων, αλλά καταδεικνύει τη λογική μετάβαση από το ακατέργαστο log μήνυμα στη σημασιολογικά και διαδικαστικά αξιοποιήσιμη γνώση.

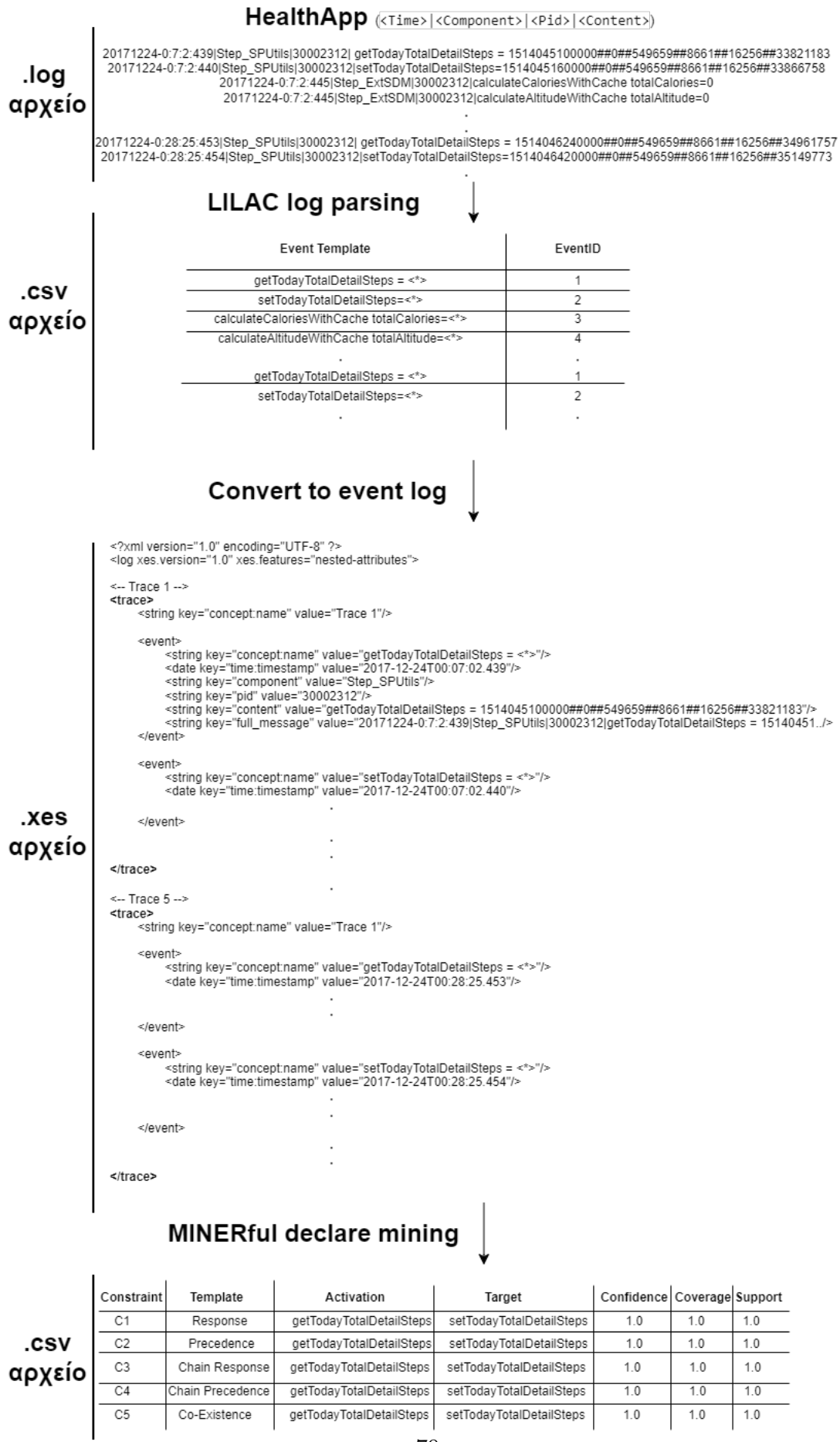


Figure B.1: Παραδειγμα της συνολικής ροής της προσέγγισης

References

[1] Anti Alman, Claudio Di Ciccio, Dominik Haas, Fabrizio Maria Maggi, and Alexander Nolte. Rule mining with rum. In *2020 2nd International Conference on Process Mining (ICPM)*, pages 121–128. IEEE, 2020.

[2] Cambridge University Press and Assessment. Log file.

[3] Francesco Chiariello, Fabrizio Maria Maggi, and Fabio Patrizi. Asp-based declarative process mining. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5539–5547, 2022.

[4] Anton A. Chuvakin and Kevin J. Schmidt. *Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management*. Syngress, Waltham, MA, 1 edition, 2013.

[5] Claudio Di Ciccio and Cecilia Iacometta. Minerful: A tool for declarative process mining, 2025.

[6] Claudio Di Ciccio and Massimo Mecella. Minerful, a mining algorithm for declarative process constraints in mailofmine. 01 2012.

[7] Ivan Donadello, Francesco Riva, Fabrizio Maria Maggi, and Aladdin Shikhizada. Declare4py: A python library for declarative process mining. *BPM (PhD/Demos)*, 3216:117–121, 2022.

[8] Ivan Donadello, Francesco Riva, Fabrizio Maria Maggi, and Aladdin Shikhizada. Declare4py: A python package for declarative process mining. `https://github.com/francxx96/declare4py`, 2022. Accessed: 2026-03-17.

[9] Elastic. Logstash on elastic. `https://www.elastic.co/docs/deploy-manage/deploy/cloud-on-k8s/logstash`, 2026.

[10] Graylog, Inc. Graylog documentation, 2026.

[11] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services (ICWS)*, pages 33–40. IEEE, 2017.

[12] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R Lyu. A survey on automated log analysis for reliability engineering. *ACM computing surveys (CSUR)*, 54(6):1–37, 2021.

[13] IEEE Task Force on Process Mining. Openxes: Reference implementation of the xes standard, 2024.

[14] IEEE Task Force on Process Mining. Xes standard. `https://www.xes-standard.org/`, 2024.

[15] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R. Lyu. LILAC: Log parsing using llms with adaptive parsing cache (code repository). `https://github.com/logpai/LILAC`, 2024.

[16] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R. Lyu. A large-scale evaluation for log parsing techniques: How far are we? In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2024.

[17] Nikita Kathare, O Vinati Reddy, and Vishalakshi Prabhu. A comprehensive study of elasticsearch. *International journal of science and research (IJSR)*, 2020.

[18] Łukasz Korzeniowski and Krzysztof Goczyła. Landscape of automated log analysis: A systematic literature review and mapping study. *IEEE Access*, 10:21892–21913, 2022.

[19] Van-Hoang Le and Hongyu Zhang. Log parsing with prompt-based few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2438–2449. IEEE, 2023.

[20] Zeyan Li, Jie Song, Tieying Zhang, Tao Yang, Xiongjun Ou, Yingjie Ye, Pengfei Duan, Muchen Lin, and Jianjun Chen. Adaptive and efficient log parsing as a cloud service. In *Companion of the 2025 International Conference on Management of Data*, pages 512–524, 2025.

[21] Yudong Liu, Xu Zhang, Shilin He, Hongyu Zhang, Liqun Li, Yu Kang, Yong Xu, Minghua Ma, Qingwei Lin, Yingnong Dang, et al. Uniparser: A unified log parser for heterogeneous log data. In *Proceedings of the ACM Web Conference 2022*, pages 1893–1901, 2022.

[22] LogIntelligence. Logppt: Log parsing with prompt-based few-shot learning. `https://github.com/LogIntelligence/LogPPT`, 2023.

[23] Ioannis Mavroudopoulos, Konstantinos Varvoutas, Georgia Kougka, Anastasios Gounaris, and Marco Comuzzi. Exploiting general purpose big-data frameworks in process mining: The case of declarative process discovery. In *International Conference on Business Process Management*, pages 185–202. Springer, 2024.

[24] Suyash Mishra and Anuranjan Misra. Structured and unstructured big data analytics. In *2017 International Conference on Current Trends in Computer, Electrical, Electronics and Communication (CTCEEC)*, pages 740–746. IEEE, 2017.

[25] Juliet Gathoni Muchori and Mwangi Peter Maina. A comprehensive review of system log significance and preparation techniques for experimental studies. *Int. J. Sci. Res. in Computer Science and Engineering Vol*, 13(6), 2025.

[26] Adam Oliner, Archana Ganapathi, and Wei Xu. Advances and challenges in log analysis: Logs contain a wealth of information for help in managing systems. *Queue*, 9(12):30–40, 2011.

[27] Maja Pesic, Helen Schonenberg, and Wil MP Van der Aalst. Declare: Full support for loosely-structured processes. In *11th IEEE international enterprise distributed object computing conference (EDOC 2007)*, pages 287–287. IEEE, 2007.

[28] SigNoz. Instrumentation Overview. `https://signoz.io/docs/instrumentation/overview/`.

[29] SigNoz. Metrics Management and User Guide. `https://signoz.io/docs/userguide/metrics/`.

[30] SigNoz Contributors. SigNoz: Open-source observability platform. `https://github.com/SigNoz/signoz`.

[31] V Sushma, Chinmay J Patil, P Swetha, et al. Secure log ingestion and visualization using dockerized graylog with multiple log shippers. In *2025 IEEE International Conference on Recent Advances in Computing and Systems (REACS)*, pages 1–5. IEEE, 2025.

[32] Pang-Ning Tan, Michael Steinbach, and Vipin Kumar. *Introduction to Data Mining*. Pearson Education India, 2016.

[33] Wil van der Aalst. *Process Mining: Data Science in Action*. Springer, 2016.

[34] Wil M. P. van der Aalst. Process mining: A 360 degree overview. In *Process Mining Handbook / or relevant proceedings depending on source*, Aachen, Germany, 2019. Springer.

[35] Yongheng Wang, Wensheng Gan, and Philip S Yu. Ai-driven log analysis: Advances and challenges.

[36] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. Divlog: Log parsing with prompt enhanced in-context learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–12, 2024.

[37] Zhuangbin Chen Yichen Li Junjie Huang Yintong Huo Pinjia He Jiazhen Gu Michael R. Lyu Zhihan Jiang, Jinyang Liu. Lilac: Log parsing using llms with adaptive parsing cache. *Proceedings of the ACM on Software Engineering*, 2024.

[38] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. Loghub: A large collection of system log datasets for ai-driven log analytics. In *Proceedings of the International Symposium on Software Reliability Engineering (ISSRE)*, 2023.

[39] Jieming Zhu, Shilin He, Jinyang Liu, Pinjia He, Qi Xie, Zibin Zheng, and Michael R Lyu. Tools and benchmarks for automated log parsing. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 121–130. IEEE, 2019.

[40] Βασίλειος Βερύκιος, Βασίλειος Καγκλής, and Ηλίας Σταυρόπουλος. *Η επιστήμη των δεδομένων μέσα από τη γλώσσα R*. Κάλλιπος, Ανοικτές Ακαδημαϊκές Εκδόσεις, Αθήνα, Ελλάδα, 2016.